

Reinforcement Learning Notes

A guided companion to
Mathematical Foundations of Reinforcement Learning
by Shiyu Zhao (Springer, 2025) — Chapters 1–10

Minghao Zhao

Spring 2026

Updated on: 26th July 2026

Textbook author / lecturer: Dr. Shiyu Zhao
Institution: University of California, San Diego

Every definition, theorem and example below is cross-referenced to the exact section and page of the textbook, and to the matching English lecture video.

Contents

How to use these notes	6
The four objects: state, action, policy, value	7
1 Basic Concepts	13
1.1 The grid world, once and for all	13
1.2 State and action	14
1.3 State transition	14
1.4 Policy	15
1.5 Reward	16
1.6 Trajectories, returns and episodes	18
1.7 Markov decision processes	20
1.8 Chapter 1 self-check	21
2 State Values and the Bellman Equation	22
2.1 Motivating example 1: why returns are the right yardstick	22
2.2 Motivating example 2: bootstrapping	23
2.3 State values	24
2.4 The Bellman equation	25
2.5 Two fully worked Bellman examples	26
2.6 Matrix–vector form	27
2.7 Solving for state values	28
2.7.1 Method 1: closed form	29
2.7.2 Method 2: iteration	29
2.7.3 What state values look like	30
2.8 From state values to action values	30
2.8.1 The Bellman equation in action values	32
2.9 Chapter 2 self-check	32
3 Optimal State Values and the Bellman Optimality Equation	34
3.1 Motivating example: how do you improve a policy?	34
3.2 Defining optimality	35
3.3 The Bellman optimality equation	36
3.3.1 Solving the maximisation on the right-hand side	36
3.3.2 Matrix–vector form	37
3.4 The contraction mapping theorem	37

3.4.1	The right-hand side of the BOE is a contraction	38
3.5	Solving the BOE	39
3.5.1	Uniqueness and stochasticity	39
3.6	What actually determines the optimal policy	40
3.7	Chapter 3 self-check	43
4	Value Iteration and Policy Iteration	44
4.1	Value iteration	44
4.1.1	Elementwise form — what you actually code	45
4.1.2	A complete numerical run	45
4.2	Policy iteration	46
4.2.1	Answer 1: how to evaluate	47
4.2.2	Answer 2: improvement really improves	47
4.2.3	Answer 3: convergence	47
4.2.4	A two-state example done by hand	48
4.2.5	A 5×5 example: how learning spreads	49
4.3	Truncated policy iteration	49
4.3.1	The two algorithms are the same algorithm	49
4.3.2	Does truncation break convergence?	51
4.4	Generalised policy iteration	51
4.5	Chapter 4 self-check	52
5	Monte Carlo Methods	53
5.1	Mean estimation: the whole idea in miniature	53
5.2	Making policy iteration model-free	54
5.3	MC Basic	55
5.3.1	A hand computation	56
5.3.2	Episode length and the sparse-reward problem	56
5.4	MC Exploring Starts	57
5.4.1	Idea 1: reuse every visit in an episode	57
5.4.2	Idea 2: update the policy after every episode	59
5.5	MC ε -Greedy	59
5.5.1	ε -greedy policies	60
5.5.2	The algorithm	60
5.5.3	Learning from a single very long episode	61
5.6	Exploration versus exploitation, quantified	61
5.7	Summary and self-check	63

6	Stochastic Approximation	66
6.1	From batch to incremental: mean estimation again	66
6.2	The Robbins–Monro algorithm	67
6.2.1	Why does it work?	69
6.2.2	The convergence theorem	69
6.2.3	Mean estimation is an RM algorithm	70
6.3	Dvoretzky’s theorem (the general convergence machinery)	71
6.4	Stochastic gradient descent	72
6.4.1	Three algorithms, one problem	72
6.4.2	Mean estimation is an SGD algorithm	73
6.4.3	The convergence pattern of SGD	74
6.4.4	The deterministic formulation	74
6.4.5	BGD, MBGD and SGD	75
6.4.6	Convergence of SGD	75
6.5	Chapter 6 summary and self-check	76
7	Temporal-Difference Methods	78
7.1	TD learning of state values	78
7.1.1	Where does this update come from?	79
7.1.2	TD target and TD error	80
7.1.3	TD versus Monte Carlo	80
7.1.4	Convergence	81
7.2	Sarsa	81
7.2.1	Finding optimal policies with Sarsa	82
7.2.2	Expected Sarsa	83
7.3	n -step Sarsa	84
7.4	Q-learning	84
7.4.1	On-policy versus off-policy	85
7.4.2	Two implementations	86
7.4.3	What the experiments show	86
7.5	A unified viewpoint	88
7.6	Chapter 7 self-check	89
8	Value Function Methods	91
8.1	From table to function	91
8.2	The objective function	93
8.2.1	The stationary distribution	94
8.3	The algorithm	95

8.4	Choosing features, with numbers	96
8.5	What is TD with function approximation actually minimising?	97
8.5.1	Convergence	97
8.5.2	Three objective functions	99
8.5.3	Least-squares TD	100
8.6	Sarsa and Q-learning with function approximation	100
8.7	Deep Q-learning (DQN)	101
8.7.1	Trick 1: two networks	102
8.7.2	Trick 2: experience replay	103
8.7.3	Two experiments, one encouraging and one sobering	103
8.8	Chapter 8 summary and self-check	104
9	Policy Gradient Methods	107
9.1	From a table of probabilities to a function	107
9.2	Two metrics	108
9.2.1	Metric 1: average state value	108
9.2.2	Metric 2: average reward	109
9.2.3	The two metrics are the same metric	109
9.3	The policy gradient theorem	110
9.3.1	Sketch of the derivation, discounted case	111
9.4	REINFORCE	112
9.4.1	Reading the update	112
9.4.2	How the samples should be drawn	113
9.5	Chapter 9 summary and self-check	114
10	Actor-Critic Methods	115
10.1	QAC: the simplest actor-critic	115
10.2	Advantage actor-critic (A2C)	116
10.2.1	Baseline invariance	116
10.2.2	The advantage function	117
10.2.3	One network, not two	118
10.3	Off-policy actor-critic	118
10.3.1	Importance sampling	118
10.3.2	The off-policy policy gradient theorem	119
10.3.3	The algorithm	120
10.4	Deterministic actor-critic (DPG)	120
10.5	Chapter 10 summary	121
10.6	Chapter 10 self-check	122

Where everything fits**123**

How to use these notes

Who this is for. These notes assume you are comfortable with calculus, basic linear algebra, and an undergraduate probability/statistics sequence of the kind an economics major takes — expectations, conditional expectations, the law of large numbers, and ordinary least squares. No prior exposure to reinforcement learning (RL) is assumed. Where the textbook moves quickly, these notes slow down and explain the step.

Three kinds of cross-reference appear throughout.

- **► Book:** §2.4, pp. 20–22 points at the exact section and page range of S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025). Page numbers refer to the printed book pagination, which is the same pagination used in the free PDF chapters distributed on the book’s GitHub page.
- **► L2 P3 — Bellman equation derivation** links to the corresponding English lecture video from the author’s YouTube playlist.
- Equation numbers in parentheses such as “(2.7)” are the textbook’s own equation numbers, so you can find the same line in the book.

Notation used throughout. A quick reference:

Symbol	Meaning	Symbol	Meaning
$\mathcal{S}$	state space	$\mathcal{A}(s)$	action space at state s
s, s'	a state and the next state	a	an action
r	a reward	$\gamma \in (0, 1)$	discount rate
$p(s' s, a)$	state transition probability	$p(r s, a)$	reward probability
$\pi(a s)$	policy	G_t	discounted return from time t
$v_\pi(s)$	state value under π	$q_\pi(s, a)$	action value under π
$v^*(s)$	optimal state value	$q^*(s, a)$	optimal action value
P_π	transition matrix under π	r_π	expected-reward vector under π
α_t	learning rate / step size	ε	exploration parameter
$\hat{v}(s, w)$	approximate state value	w, θ	parameter vectors

The shape of the subject. The book has two halves. Chapters 1–3 build the *objects*: what a state, a policy, and a value are, and what “optimal” means. Chapters 4–10 build the *algorithms* that find optimal policies, progressively relaxing what we are allowed to assume:

Chapter	Needs the model?	Values stored as
4 — Value / policy iteration	Yes (dynamic programming)	a table
5 — Monte Carlo	No (needs complete episodes)	a table
6 — Stochastic approximation	(mathematical toolkit)	—
7 — Temporal difference	No (learns online, step by step)	a table
8 — Value function methods	No	a parametric function
9 — Policy gradient	No	policy is a parametric function
10 — Actor-critic	No	both value and policy are functions

The four objects: state s , action a , policy π , value v_π

Almost every difficulty people have with reinforcement learning comes from blurring four objects together. This section defines them one at a time, as concretely as possible, and then shows how they link up. Everything here is developed properly in §1 and §2; this is the orientation you should read first.

Symbol	Name	In one sentence
s	state	<i>Where you are.</i> A complete description of the agent’s situation.
a	action	<i>What you do.</i> A choice available to the agent at a state.
$\pi(a s)$	policy	<i>Your rule for deciding.</i> It assigns, to every state, a probability for each action.
$v_\pi(s)$	state value	<i>How good it is to be at s if you follow π forever.</i> The expected total discounted reward.

Intuition (The one-line story). The environment puts you in a **state**. Your **policy** tells you which **action** to take. The action earns a reward and moves you to a new state, and the whole process repeats. The **state value** is the score that tells you whether your policy is any good. *Reinforcement learning is the search for the policy with the highest values.*

1. The state s

► Book: §1.2, p. 2

Definition. A **state** s describes the agent’s status with respect to the environment. The set of all possible states is the **state space**, written $\mathcal{S}$.

In the grid world the agent’s situation is fully described by *which cell it is standing in*. Nine cells, so nine states, $\mathcal{S} = \{s_1, \dots, s_9\}$, numbered left to right and top to bottom:

s_1	s_2	s_3	white = ordinary cell (reward 0)
s_4	s_5	s_6	orange = forbidden cell (penalty -1)
s_7	s_8	s_9	blue = target cell (reward $+1$)

Remark (What makes something a state?). A state must contain *everything relevant about the present*. Here the cell index is enough: knowing you are at s_5 tells you all you need in order to decide what to do next — the route you took to arrive there is irrelevant. That property has a name, the **Markov property** [Book §1.7, p. 11], and without it none of the mathematics in this book would work.

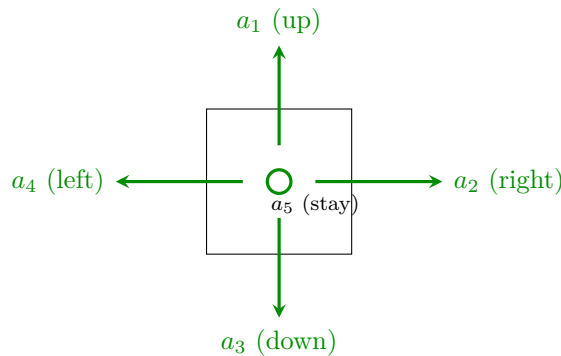
A counterexample makes it concrete. If the grid had a door that opens only once you have picked up a key, then “which cell I am in” would *not* be a valid state — two agents in the same cell would face different futures depending on whether they hold the key. The fix is to enlarge the state to (cell, key held or not).

2. The action a

► Book: §1.2, p. 2

Definition. An **action** a is a choice available to the agent. The set of actions available at state s is the **action space** $\mathcal{A}(s)$.

In the grid world there are five actions at every state:



Common pitfall (Three things that trip people up). **(i) Actions are attempts, not outcomes.** Taking a_1 at s_1 (the top-left corner) does not move you out of the grid — you are bounced back to s_1 and collect a reward of -1 [Book §1.3, p. 3]. The action is what you *try*; the environment decides what happens.

(ii) Every action is available at every state. You might think “left” should be removed at s_1 since it hits a wall. The book deliberately keeps $\mathcal{A}(s) = \{a_1, \dots, a_5\}$ everywhere and punishes wall-bumping with a negative reward instead [Book §1.2, p. 2]. This keeps all the tables rectangular.

(iii) Actions can be taken into forbidden cells. Orange cells are enterable, just unpleasant [Book §1.3, p. 3]. Whether entering one is worth it is exactly the kind of question values are designed to answer.

3. The policy π

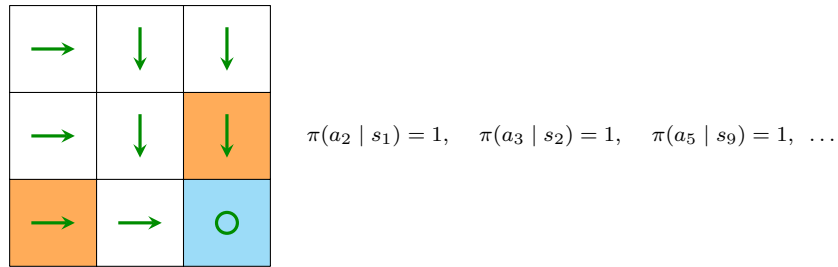
► Book: §1.4, p. 4

Definition. A **policy** π tells the agent which action to take at every state. Formally, $\pi(a \mid s)$ is the probability of taking action a when in state s , and it must satisfy

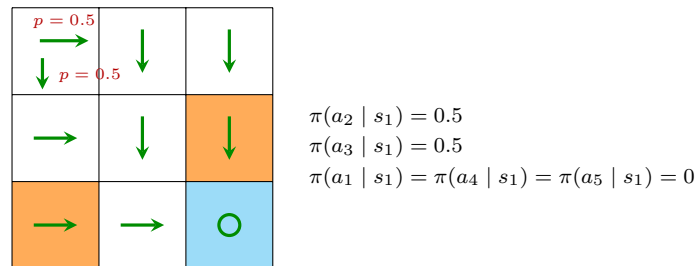
$$\pi(a \mid s) \geq 0 \quad \text{for all } a, \quad \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1 \quad \text{for every } s.$$

Intuition. A policy is a *complete contingency plan*: it specifies what to do at *every* state, not just the one you happen to occupy. Think of it as a strategy handed to you before the game starts, covering every situation you might find yourself in. It is not a route of the form “go right, then down, then right”; it is a lookup rule, “if at s_5 , go down”.

A deterministic policy. Every state has exactly one arrow, so $\pi(a \mid s) = 1$ for that action and 0 for the other four:



A stochastic policy. At s_1 the agent now tosses a fair coin between going right and going down; everywhere else it is unchanged:



Remark (Policy versus trajectory). Running a policy produces a **trajectory** — an actual sequence of states, actions and rewards, such as $s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_3} s_8 \xrightarrow{a_2} s_9$. The policy is the *rule*; the trajectory is one *realisation* of it. A deterministic policy in a deterministic environment always produces the same trajectory from a given start; a stochastic policy produces a different one each time [Book §1.6, p. 8].

4. The state value $v_\pi(s)$

► Book: §2.3, p. 19

This is the one that needs the most care, so we build it in three steps.

Step 1: reward r — what you get right now. After taking an action you receive a number. In the grid world: -1 for hitting the boundary, -1 for entering a forbidden cell, $+1$ for reaching the target, 0 otherwise [Book §1.5, p. 7].

Step 2: return G_t — what you get in total. A single reward is myopic; what matters is the whole future stream, with later rewards discounted [Book §1.6, p. 9]:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots, \quad \gamma \in (0, 1).$$

The discount rate γ plays exactly the role an economist expects: it is a one-period discount factor, and $1/(1 - \gamma)$ is the present value of a perpetuity paying 1 per period.

Step 3: state value $v_\pi(s)$ — what you get on average. The return is a random variable, because the policy and the environment may both be stochastic. Average it:

Definition (State value).

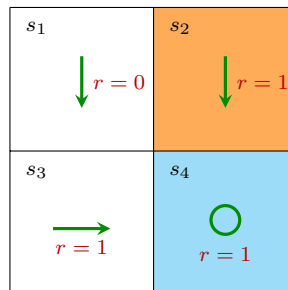
$$v_{\pi}(s) = \mathbb{E}[G_t | S_t = s]$$

the expected discounted return obtained by starting at s and following π forever afterwards.

Common pitfall (Read the subscript). v_{π} carries a π for a reason. **A state has no value on its own** — only a value *under a given policy*. The same cell s_1 is worth a lot under a good policy and very little under a bad one. Asking “what is the value of s_1 ?” is like asking “what is this chess position worth?” without saying who is playing.

A fully worked value computation

Take the smallest example that shows everything [Book §2.5, pp. 22–25]. A 2×2 grid: s_2 is forbidden, s_4 is the target, $\gamma = 0.9$. Consider the policy drawn below — at s_1 go down, at s_2 go down, at s_3 go right, at s_4 stay put.



Write down one equation per state. Each says: value = immediate reward + $\gamma \times$ value of wherever you land.

$$\begin{aligned} v_{\pi}(s_1) &= 0 + \gamma v_{\pi}(s_3), & v_{\pi}(s_2) &= 1 + \gamma v_{\pi}(s_4), \\ v_{\pi}(s_3) &= 1 + \gamma v_{\pi}(s_4), & v_{\pi}(s_4) &= 1 + \gamma v_{\pi}(s_4). \end{aligned}$$

Solve. The last equation involves only $v_{\pi}(s_4)$:

$$v_{\pi}(s_4) = 1 + \gamma v_{\pi}(s_4) \implies v_{\pi}(s_4)(1 - \gamma) = 1 \implies v_{\pi}(s_4) = \frac{1}{1 - \gamma} = \frac{1}{0.1} = 10.$$

Then back-substitute:

$$v_{\pi}(s_3) = 1 + 0.9(10) = 10, \quad v_{\pi}(s_2) = 1 + 0.9(10) = 10, \quad v_{\pi}(s_1) = 0 + 0.9(10) = 9.$$

9.0	10.0
10.0	10.0

the state values v_{π} of this policy, with $\gamma = 0.9$

Intuition (Why s_1 is worth less). s_1 scores 9 rather than 10 purely because it sits one step further from the reward: the +1 stream starts one period later and is therefore discounted once more. Nothing bad happens at s_1 — it is simply further away. This is the ordinary logic of a present-value calculation.

Values rank policies — which is the whole point

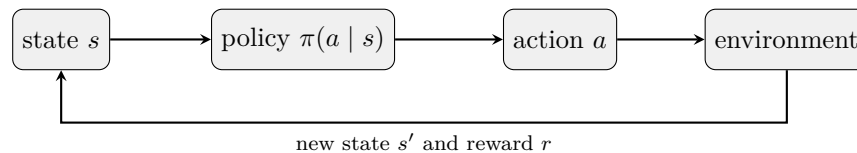
Now change one thing: at s_1 , send the agent *right* into the forbidden cell ($r = -1$) instead of down. Redoing the arithmetic [Book §3.1, p. 36]:

$$v_\pi(s_1) = -1 + \gamma v_\pi(s_2) = -1 + 0.9(10) = 8.$$

	$v_\pi(s_1)$	$v_\pi(s_2)$	$v_\pi(s_3)$	$v_\pi(s_4)$
Policy A (at s_1 go down)	9	10	10	10
Policy B (at s_1 go right)	8	10	10	10

Policy A is at least as good at every state and strictly better at s_1 , so **A is the better policy** — and we reached that verdict by arithmetic, not by eyeballing the picture. That is what state values are *for* [Book §2.5, p. 25].

How the four objects chain together



repeat forever $\Rightarrow$ a stream of rewards $\Rightarrow$ discounted return $G_t \Rightarrow$ average it $\Rightarrow$ the state value $v_\pi(s)$

Formally the chain is [Book §2.4, p. 21]

$$v_\pi(s) = \underbrace{\sum_a \pi(a | s)}_{\text{which action — from the policy}} \left[\underbrace{\sum_r p(r | s, a) r}_{\text{reward now}} + \gamma \underbrace{\sum_{s'} p(s' | s, a) v_\pi(s')}_{\text{value of the next state}} \right],$$

which is the Bellman equation — and you can now read every symbol in it.

Six confusions worth heading off

1. **Reward vs. return vs. value.** *Reward* is one number received once. *Return* is the discounted sum of all future rewards along one trajectory — still random. *Value* is the expectation of the return — a fixed number. Reward is myopic and cannot rank policies; value can [Book §1.5, p. 7].
2. **A policy is not a path.** It is a rule defined at every state, so the agent still knows what to do if it is blown off course.
3. **Value depends on the policy.** Always v_π , never v alone — except for v^* , the value of an *optimal* policy [Book §3.2, p. 38].
4. **State value vs. action value.** $v_\pi(s)$ answers “how good is it to be here, behaving as usual?” The **action value** $q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$ answers “how good is it to be here and do a *once*, then behave as usual?” They are linked by $v_\pi(s) = \sum_a \pi(a | s) q_\pi(s, a)$ [Book §2.8, p. 30]. Improving a policy means comparing q values, which is why q takes over from v later in the book.

5. $\pi(a \mid s)$ and $p(s' \mid s, a)$ are **different objects**. The policy belongs to the *agent* and is what we optimise. The transition probability belongs to the *environment* and is not ours to choose — indeed from Chapter 5 onward we do not even know it.
6. **Action values exist for actions your policy never takes**. If π never picks a at s , $q_\pi(s, a)$ is still perfectly well defined, and it may well be the largest of them all — which is precisely how a policy gets improved [Book §2.8.1, p. 32].

Object	Type	Grid-world instance
s	element of $\mathcal{S}$	s_5 , the middle cell
a	element of $\mathcal{A}(s)$	a_3 , “move down”
$\pi(a \mid s)$	a probability distribution over a , for each s	$\pi(a_3 \mid s_5) = 1$
r	a real number	-1 for entering a forbidden cell
G_t	a random variable	$0 + \gamma 0 + \gamma^2 1 + \dots$
$v_\pi(s)$	a real number, one per state	$v_\pi(s_1) = 9$ when $\gamma = 0.9$
$q_\pi(s, a)$	a real number, one per state–action pair	$q_\pi(s_1, a_3) = 9$

to find a rule for choosing moves that gets the agent to the target from any starting cell, without entering forbidden cells, without pointless detours, and without banging into the outer wall.

The reason the book uses one running example for ten chapters is worth stating plainly: it means that when an algorithm changes, you can see exactly what changed, because the problem did not. Get comfortable with this grid; you will see it a hundred times.

Intuition. If the agent had a map, this would be a shortest-path problem and you would solve it with Dijkstra’s algorithm in an afternoon. RL is what you do when the agent has *no map*: it must discover the layout by trying things and observing what happens. Everything hard about RL follows from that one missing ingredient [Book §1.1, p. 1].

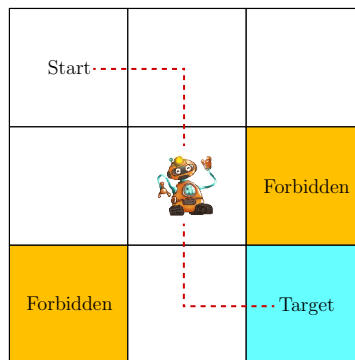


Figure 1.2. The grid world example is used throughout the book.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§1.2 State and action

► Book: §1.2, p. 2; Figure 1.3, p. 3

Definition. A **state** s describes the agent’s status with respect to the environment. The set of all states is the **state space** $\mathcal{S}$.

In the 3×3 grid there are nine cells and therefore nine states, $\mathcal{S} = \{s_1, \dots, s_9\}$, indexed left to right, top to bottom.

Definition. An **action** a is a choice available to the agent. The set of actions at state s is the **action space** $\mathcal{A}(s)$.

The grid world has five actions: $a_1 = \text{up}$, $a_2 = \text{right}$, $a_3 = \text{down}$, $a_4 = \text{left}$, $a_5 = \text{stay still}$.

Remark. In principle the action space can differ across states. For example, at the top-left corner s_1 both “up” and “left” would run into the wall, so you could declare $\mathcal{A}(s_1) = \{a_2, a_3, a_5\}$. The textbook deliberately does *not* do this: it keeps $\mathcal{A}(s_i) = \mathcal{A} = \{a_1, \dots, a_5\}$ for every state, and instead punishes wall-bumping with a negative reward [Book §1.2, p. 2]. This is the more general setup and it keeps all the tables rectangular.

§1.3 State transition

► Book: §1.3, p. 3; Table 1.1, p. 4

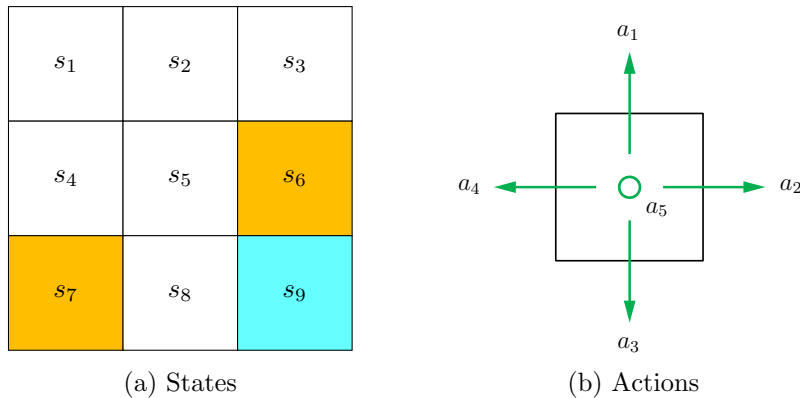


Figure 1.3. Illustrations of the state and action concepts. (a) There are nine states $\{s_1, \dots, s_9\}$. (b) Each state has five possible actions $\{a_1, a_2, a_3, a_4, a_5\}$.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Taking an action moves the agent from one state to another. Write $s_1 \xrightarrow{a_2} s_2$ for “taking a_2 at s_1 leads to s_2 ”.

Two edge cases are settled once and for all in the textbook:

- **Walking into the outer boundary.** The agent is bounced back: $s_1 \xrightarrow{a_1} s_1$. It cannot leave the state space.
- **Walking into a forbidden cell.** Two modelling choices exist: the forbidden cell is a wall (agent bounces back) or the forbidden cell is enterable but unpleasant. *The book chooses the second:* forbidden cells are accessible, entering them earns a penalty [Book §1.3, p. 3].

Definition. The *state transition probability* is

$$p(s' | s, a) = \mathbb{P}(S_{t+1} = s' | S_t = s, A_t = a), \quad \sum_{s' \in \mathcal{S}} p(s' | s, a) = 1 \quad \forall (s, a).$$

For the deterministic grid, $p(s_2 | s_1, a_2) = 1$ and $p(s' | s_1, a_2) = 0$ for every other s' . A tabular listing (the book’s Table 1.1) captures this perfectly. But a table can only express *deterministic* transitions. The conditional probability notation is strictly more general: with random wind gusts, taking a_2 at s_1 might blow the agent to s_5 , i.e. $p(s_5 | s_1, a_2) > 0$ [Book §1.3, p. 4].

Worked example: reading the transition table

From the book’s Table 1.1 (p. 4), row s_5 reads: $a_1 \rightarrow s_2$, $a_2 \rightarrow s_6$, $a_3 \rightarrow s_8$, $a_4 \rightarrow s_4$, $a_5 \rightarrow s_5$. Note $a_2 \rightarrow s_6$ even though s_6 is a forbidden cell — consistent with the “forbidden cells are accessible” choice above. Row s_9 reads $a_1 \rightarrow s_6$, $a_2 \rightarrow s_9$, $a_3 \rightarrow s_9$, $a_4 \rightarrow s_8$, $a_5 \rightarrow s_9$: three of the five actions leave the agent where it is, because s_9 is the bottom-right corner.

§1.4 Policy

► Book: §1.4, p. 4; Figures 1.4–1.5, pp. 5–6; Table 1.2, p. 6

	a_1 (up)	a_2 (right)	a_3 (down)	a_4 (left)	a_5 (stay)
s_1	s_1	s_2	s_4	s_1	s_1
s_2	s_2	s_3	s_5	s_1	s_2
s_3	s_3	s_3	s_6	s_2	s_3
s_4	s_1	s_5	s_7	s_4	s_4
s_5	s_2	s_6	s_8	s_4	s_5
s_6	s_3	s_6	s_9	s_5	s_6
s_7	s_4	s_8	s_7	s_7	s_7
s_8	s_5	s_9	s_8	s_7	s_8
s_9	s_6	s_9	s_9	s_8	s_9

Table 1.1. Tabular representation of the state transition process: each cell gives the next state reached by taking that action at that state [Book §1.3, p. 4].

Definition. A *policy* π tells the agent which action to take at every state. Formally $\pi(a \mid s)$ is the probability of taking a at s , with $\sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1$ for every s .

A policy is drawn as a field of arrows, one arrow per cell. Two kinds:

- **Deterministic:** $\pi(a^* \mid s) = 1$ for one action a^* . Example from the book: $\pi(a_2 \mid s_1) = 1$ and $\pi(a \mid s_1) = 0$ for $a \neq a_2$ [Book §1.4, p. 5].
- **Stochastic:** several actions get positive probability. The book’s Figure 1.5 (p. 6) has $\pi(a_2 \mid s_1) = \pi(a_3 \mid s_1) = 0.5$, drawn as two arrows each labelled $p = 0.5$.

A policy stored as a table with one row per state and one column per action is called a **tabular representation** [Book §1.4, p. 6]. Keep this word in mind: Chapters 1–7 all use tabular representations, and Chapters 8–10 are precisely about replacing the table with a parameterised function.

Remark. Note the asymmetry in notation: $p(s' \mid s, a)$ belongs to the *environment* and is not ours to choose; $\pi(a \mid s)$ belongs to the *agent* and is exactly the thing we are trying to optimise.

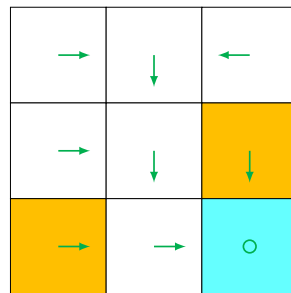
	a_1 (up)	a_2 (right)	a_3 (down)	a_4 (left)	a_5 (stay)
s_1	0	0.5	0.5	0	0
s_2	0	0	1	0	0
s_3	0	0	0	1	0
s_4	0	1	0	0	0
s_5	0	0	1	0	0
s_6	0	0	1	0	0
s_7	0	1	0	0	0
s_8	0	1	0	0	0
s_9	0	0	0	0	1

Table 1.2. Tabular representation of the stochastic policy of Figure 1.5: entry (i, j) is the probability of taking action a_j at state s_i [Book §1.4, p. 6].

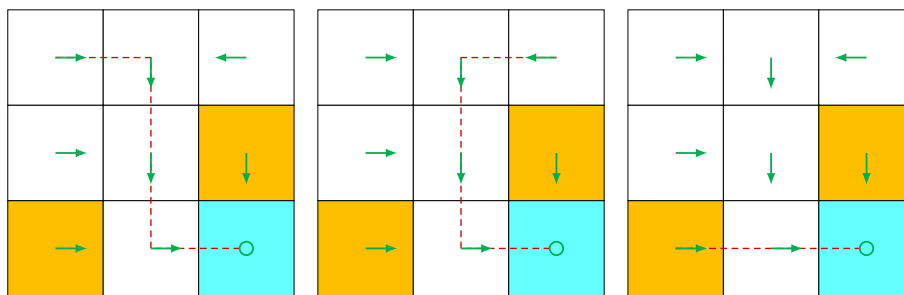
§1.5 Reward

► Book: §1.5, p. 6; Table 1.3, p. 8

Definition. After executing action a at state s , the agent receives a scalar **reward** r , a real number that may be positive, negative or zero. In general it is described by the conditional distribution $p(r \mid s, a)$ with $\sum_r p(r \mid s, a) = 1$.



(a) A deterministic policy



(b) Trajectories obtained from the policy

Figure 1.4. A policy represented by arrows and some trajectories obtained by starting from different initial states.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

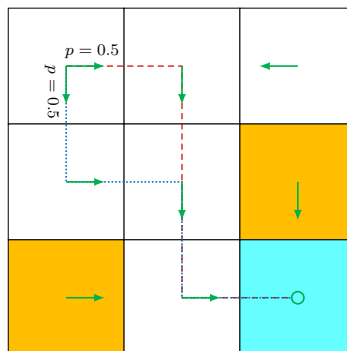


Figure 1.5. A stochastic policy. In state s_1 , the agent may move rightward or downward with equal probabilities of 0.5.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

The book’s standard grid-world reward design [Book §1.5, p. 7]:

$$r_{\text{boundary}} = -1, \quad r_{\text{forbidden}} = -1, \quad r_{\text{target}} = +1, \quad r_{\text{other}} = 0.$$

Three points that beginners routinely get wrong.

Common pitfall. (i) **Rewards are not the objective.** If you hand a student the reward table and ask them to build a policy by picking, at each state, the action with the largest immediate reward, they will build a bad policy. Immediate reward is myopic; what matters is the *total* reward accumulated in the long run. The book states this explicitly [Book §1.5, p. 7]. This is the entire reason state values exist.

(ii) **The reward at the target does not stop the process.** At s_9 the process continues. Taking a_5 (stay) at s_9 returns you to s_9 with $r = +1$ again; taking a_2 at s_9 also returns you to s_9 but with $r = r_{\text{boundary}} = -1$, because you tried to leave the grid [Book §1.5, p. 7].

(iii) **Only relative reward values matter.** Adding the same constant to every reward leaves the optimal policy unchanged. Setting $r_{\text{boundary}} = r_{\text{forbidden}} = -3$, $r_{\text{target}} = -1$, $r_{\text{other}} = -2$ — all negative — gives exactly the same optimal policy [Book §1.9 Q&A, p. 12]. The formal statement is Theorem 3.6 (optimal policy invariance), [Book §3.5, p. 51], proved in Chapter 3 of these notes.

Remark (Does the reward depend on the next state?). Intuitively, yes: you get $+1$ *because* you landed on the target. So why does the book write $p(r \mid s, a)$ rather than $p(r \mid s, a, s')$? Because s' is itself generated by (s, a) , so you can always marginalise it out:

$$p(r \mid s, a) = \sum_{s'} p(r \mid s, a, s') p(s' \mid s, a).$$

The two formulations are equivalent, and $p(r \mid s, a)$ makes the Bellman equation cleaner [Book §1.9 Q&A, p. 13].

	a_1 (up)	a_2 (right)	a_3 (down)	a_4 (left)	a_5 (stay)
s_1	r_{bnd}	0	0	r_{bnd}	0
s_2	r_{bnd}	0	0	0	0
s_3	r_{bnd}	r_{bnd}	r_{fbd}	0	0
s_4	0	0	r_{fbd}	r_{bnd}	0
s_5	0	r_{fbd}	0	0	0
s_6	0	r_{bnd}	r_{tgt}	0	r_{fbd}
s_7	0	0	r_{bnd}	r_{bnd}	r_{fbd}
s_8	0	r_{tgt}	r_{bnd}	r_{fbd}	0
s_9	r_{fbd}	r_{bnd}	r_{bnd}	0	r_{tgt}

Table 1.3. Tabular representation of the reward process [Book §1.5, p. 8]. Here $r_{\text{bnd}} = r_{\text{boundary}}$, $r_{\text{fbd}} = r_{\text{forbidden}}$, $r_{\text{tgt}} = r_{\text{target}}$.

§1.6 Trajectories, returns and episodes

► Book: §1.6, p. 8; Figure 1.6, p. 8

A **trajectory** is a state–action–reward chain. Following the book’s Figure 1.6(a) (p. 8):

$$s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=0]{a_3} s_5 \xrightarrow[r=0]{a_3} s_8 \xrightarrow[r=1]{a_2} s_9.$$

Definition. The (undiscounted) **return** of a trajectory is the sum of rewards collected along it.

For the trajectory above, $\text{return} = 0 + 0 + 0 + 1 = 1$ (the book’s equation (1.1), p. 8). Compare with the alternative policy of Figure 1.6(b):

$$s_1 \xrightarrow[r=0]{a_3} s_4 \xrightarrow[r=-1]{a_3} s_7 \xrightarrow[r=0]{a_2} s_8 \xrightarrow[r=+1]{a_2} s_9, \quad \text{return} = 0 - 1 + 0 + 1 = 0,$$

the book’s equation (1.2), p. 9. So $1 > 0$: the first policy is better. The mathematics agrees with the intuition that the second policy is worse because it cuts through a forbidden cell. *This tiny calculation is the seed of the entire subject*: returns turn “that policy looks better” into a number.

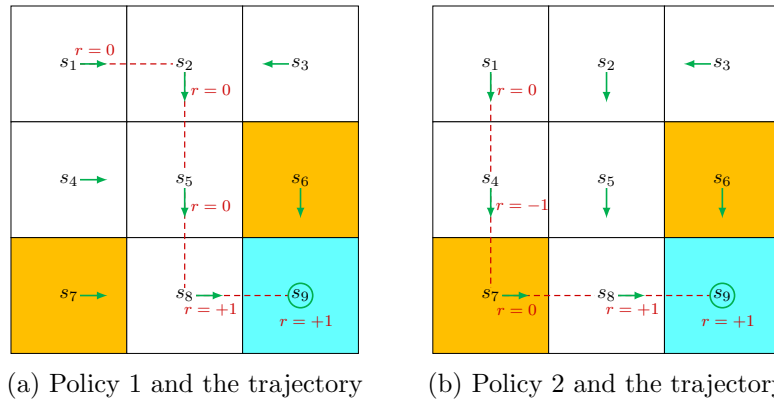


Figure 1.6. Trajectories obtained by following two policies. The trajectories are indicated by red dashed lines.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Why we must discount

► [Book: §1.6, pp. 9–10, equation \(1.3\)](#)

If the agent stays at s_9 forever it keeps collecting $+1$:

$$\text{return} = 0 + 0 + 0 + 1 + 1 + 1 + \dots = \infty,$$

which diverges and so cannot rank policies. Fix: discount.

Definition. The *discounted return* starting at time t is

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad \gamma \in (0, 1),$$

where γ is the *discount rate*.

For the trajectory above, the book computes [\[Book §1.6, p. 10\]](#)

$$\text{discounted return} = 0 + \gamma 0 + \gamma^2 0 + \gamma^3 (1 + \gamma + \gamma^2 + \dots) = \frac{\gamma^3}{1 - \gamma}.$$

The discount rate does two jobs [\[Book §1.6, p. 10\]](#):

1. **Convergence.** A bounded reward sequence discounted by $\gamma < 1$ always sums to a finite number, so infinitely long trajectories are legal.

2. **Myopia control.** $\gamma \rightarrow 0$ makes the agent short-sighted (only the next reward matters); $\gamma \rightarrow 1$ makes it far-sighted and willing to accept near-term losses for later gains. Chapter 3 of these notes shows this changing the optimal policy in a concrete grid [Book §3.5, pp. 49–51].

Remark (The bootstrapping identity). Split off the first term:

$$G_t = R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) = R_{t+1} + \gamma G_{t+1}.$$

This one line is the engine of the entire book. It is the reason the Bellman equation exists [Book §2.4, p. 20], the reason temporal-difference learning works [Book §7.1, p. 126], and it is used again in every chapter after that. Learn it.

Definition. An *episode* is a trajectory that terminates at a terminal state. Tasks with terminal states are *episodic*; tasks without are *continuing*.

The book unifies the two by converting episodic tasks into continuing ones [Book §1.6, p. 10]. Two devices are available:

- Make the terminal state **absorbing**: set $\mathcal{A}(s_g) = \{a_5\}$ or set $p(s_g | s_g, a_i) = 1$ for all i , so the agent can never leave.
- Treat the terminal state as an ordinary state — the agent may leave and return. Because $r = +1$ is collected each time s_g is reached, the agent will *learn* to stay. This requires discounting to avoid divergence.

The book takes the second option throughout [Book §1.6, p. 10], so keep that in mind whenever you see s_g in an example.

§1.7 Markov decision processes

► Book: §1.7, p. 11; equation (1.4), p. 11; Figure 1.7, p. 12

Everything above assembles into one object.

Definition (MDP). A *Markov decision process* consists of:

- **Sets:** state space $\mathcal{S}$; action spaces $\mathcal{A}(s)$; reward sets $\mathcal{R}(s, a)$.
- **Model (dynamics):** $p(s' | s, a)$ and $p(r | s, a)$, with $\sum_{s'} p(s' | s, a) = 1$ and $\sum_r p(r | s, a) = 1$.
- **Policy:** $\pi(a | s)$ with $\sum_a \pi(a | s) = 1$.
- **Markov property:** for all t ,

$$p(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(s_{t+1} | s_t, a_t),$$

$$p(r_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(r_{t+1} | s_t, a_t).$$

The pair $\{p(s' | s, a), p(r | s, a)\}$ is called the **model** or **dynamics** [Book §1.7, p. 11]. A model is **stationary** if it does not change over time. If forbidden regions could appear and disappear, the model would be nonstationary; **the book considers only stationary models**, and only **finite** MDPs (finitely many states and actions) [Book §1.7, p. 12].

Intuition (What the Markov property is really buying us). The Markov property says the current state is a *sufficient statistic* for the future: given where you are now, how you got here tells you nothing extra. Without it, the value of a state would depend on the entire history, the number of unknowns would explode, and there would be no Bellman equation. The Markov property is exactly what makes the recursion $G_t = R_{t+1} + \gamma G_{t+1}$ turn into a solvable finite system of equations [Book §1.7, p. 11].

Remark (MDP versus Markov process). Fix a policy π in an MDP and the actions disappear — the agent no longer chooses anything, it just moves according to $p_\pi(s' | s) = \sum_a \pi(a | s) p(s' | s, a)$. What remains is a **Markov process** (a Markov chain, when discrete-time with finitely many states). The book draws this explicitly in Figure 1.7, p. 12: the grid world under a fixed policy becomes a graph of nine nodes with transition probabilities on the edges. This observation is what makes the matrix form of the Bellman equation possible in the next chapter.

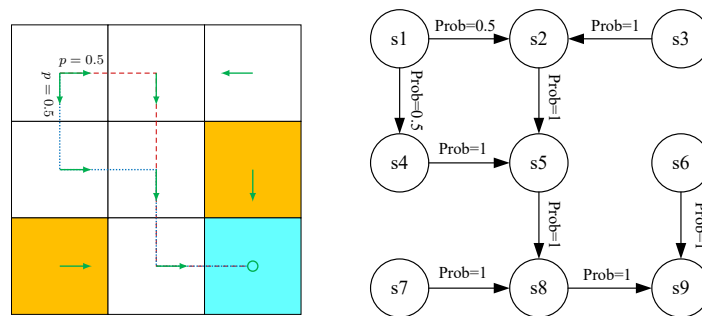


Figure 1.7. Abstraction of the grid world example as a Markov process. Here, the circles represent states and the links with arrows represent state transitions.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§1.8 Chapter 1 self-check

1. Why can adding -2 to every reward leave the optimal policy unchanged, but change every state value? (*Answer: Theorem 3.6 and equation (3.8), [Book §3.5, p. 51].*)
2. In the grid world, the agent bumps the wall at s_1 by taking a_1 . What are s' , r , and $p(s_1 | s_1, a_1)$? (*Answer: $s' = s_1$, $r = -1$, probability 1; [Book §1.3, p. 3] and Table 1.3, p. 8.)*)
3. Write G_t in terms of G_{t+1} and explain in one sentence why this is the most important identity in the book.
4. Why does the book insist that a return, not a reward, is the right thing to maximise? Give the two-policy numerical comparison from p. 8–9.

§2 State Values and the Bellman Equation

READING MAP — Chapter 2 of the textbook

Textbook: Chapter 2, “State Values and Bellman Equation”, pp. 15–34. Sections: §2.1 why returns matter (p. 16), §2.2 how to calculate returns (p. 17), §2.3 state values (p. 19), §2.4 the Bellman equation (p. 20), §2.5 worked examples (p. 22), §2.6 matrix–vector form (p. 25), §2.7 solving for state values (p. 27; closed form p. 27, iterative p. 28, examples p. 28), §2.8 from state value to action value (p. 30; examples p. 31; Bellman equation in action values p. 32), §2.9 summary (p. 33), §2.10 Q&A (p. 33).

Videos: ► L2 P1 — Motivating examples; ► L2 P2 — State value; ► L2 P3 — Bellman equation derivation; ► L2 P4 — Matrix–vector form and solution; ► L2 P5 — Action value.

What to take away: the state value $v_\pi(s) = \mathbb{E}[G_t \mid S_t = s]$ is the scoreboard for policies; the Bellman equation is a *linear system* that computes that scoreboard.

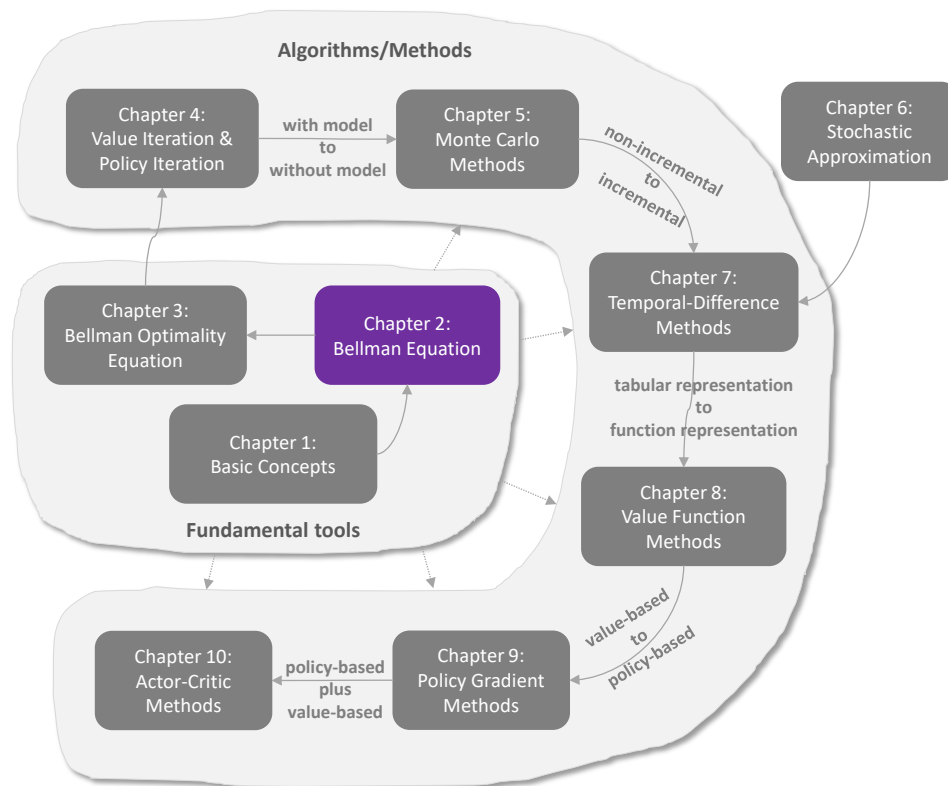


Figure 2.1. Where we are in this book.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§2.1 Motivating example 1: why returns are the right yardstick

► **Book:** §2.1, pp. 16–17; Figure 2.2, p. 16; inequality (2.1), p. 17

The textbook opens with three policies that differ only at s_1 in a 2×2 grid (Figure 2.2, p. 16). s_2 is a forbidden cell, s_4 is the target.

Worked example: ranking three policies by their returns [Book §2.1, pp. 16–17]

Policy 1 (at s_1 go down). Trajectory $s_1 \rightarrow s_3 \rightarrow s_4 \rightarrow s_4 \rightarrow \dots$, with rewards $0, 1, 1, 1, \dots$:

$$\text{return}_1 = 0 + \gamma 1 + \gamma^2 1 + \dots = \gamma(1 + \gamma + \gamma^2 + \dots) = \frac{\gamma}{1 - \gamma}.$$

Policy 2 (at s_1 go right, into the forbidden cell). Trajectory $s_1 \rightarrow s_2 \rightarrow s_4 \rightarrow s_4 \rightarrow \dots$, rewards $-1, 1, 1, \dots$:

$$\text{return}_2 = -1 + \gamma 1 + \gamma^2 1 + \dots = -1 + \frac{\gamma}{1 - \gamma}.$$

Policy 3 (at s_1 go right or down, each with probability 0.5). Now there are two possible trajectories, each with probability 0.5, so we average:

$$\text{return}_3 = 0.5 \left(-1 + \frac{\gamma}{1 - \gamma} \right) + 0.5 \left(\frac{\gamma}{1 - \gamma} \right) = -0.5 + \frac{\gamma}{1 - \gamma}.$$

Conclusion. For every $\gamma \in (0, 1)$,

$$\text{return}_1 > \text{return}_3 > \text{return}_2,$$

which is the book's inequality (2.1), p. 17. Policy 1 is best, policy 2 is worst, and the stochastic policy 3 sits exactly in between. This matches the eyeball verdict: policy 2 walks into the forbidden cell, policy 3 does so half the time.

Remark. The book flags something important about return_3 : it is not really a return at all, because it averages over two different trajectories. It is an *expected* return. That object is precisely what we are about to name a **state value** [Book §2.1, p. 17].

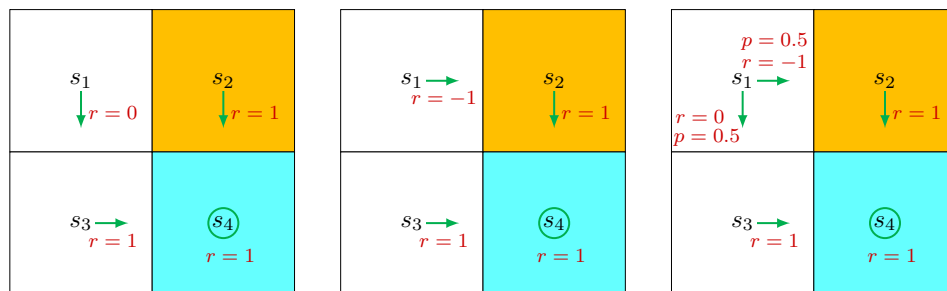


Figure 2.2. Examples for demonstrating the importance of returns. The three examples have different policies for s_1 .

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§2.2 Motivating example 2: bootstrapping

► Book: §2.2, pp. 17–19; Figure 2.3, p. 18; equations (2.2)–(2.3)

Consider a 2×2 grid in which the policy sends the agent round a cycle $s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4 \rightarrow s_1 \rightarrow \dots$ collecting rewards r_1, r_2, r_3, r_4 along the way (the book's Figure 2.3, p. 18; there are no target or forbidden cells here). Let v_i be the return starting from s_i .

Method A — brute force (the book’s equation (2.2), p. 18):

$$\begin{aligned} v_1 &= r_1 + \gamma r_2 + \gamma^2 r_3 + \cdots, & v_2 &= r_2 + \gamma r_3 + \gamma^2 r_4 + \cdots, \\ v_3 &= r_3 + \gamma r_4 + \gamma^2 r_1 + \cdots, & v_4 &= r_4 + \gamma r_1 + \gamma^2 r_2 + \cdots. \end{aligned}$$

Method B — bootstrapping (equation (2.3), p. 18). Factor out γ :

$$v_1 = r_1 + \gamma v_2, \quad v_2 = r_2 + \gamma v_3, \quad v_3 = r_3 + \gamma v_4, \quad v_4 = r_4 + \gamma v_1.$$

Intuition. “Bootstrapping” means computing something from itself. At first this looks like circular reasoning: to get v_1 you need v_2 , which needs v_3 , which needs v_4 , which needs v_1 . It is not circular — it is a *simultaneous system of linear equations*, and you solve it the way you solved simultaneous equations in high school. An economist should find this familiar: it is the same structure as a Leontief input–output system $x = Ax + d$, or a linear rational-expectations model where today’s price depends on the expected price tomorrow.

Stack the four equations [Book §2.2, p. 18]:

$$\underbrace{\begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{pmatrix}}_v = \underbrace{\begin{pmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{pmatrix}}_r + \gamma \underbrace{\begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}}_P \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{pmatrix}, \quad \text{i.e.} \quad v = r + \gamma P v,$$

whose solution is $v = (I - \gamma P)^{-1}r$. That $I - \gamma P$ is always invertible is proved in §2.7.1 (p. 27) and reproduced below.

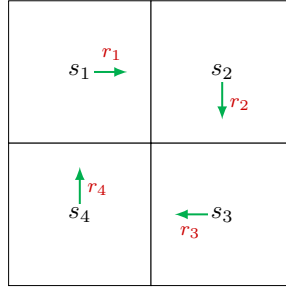


Figure 2.3. An example for demonstrating how to calculate returns. There are no target or forbidden cells in this example.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§2.3 State values

► Book: §2.3, pp. 19–20

Notation. At time t the agent is in state S_t , takes action A_t under π , moves to S_{t+1} and collects R_{t+1} : $S_t \xrightarrow{A_t} S_{t+1}, R_{t+1}$. All four objects are random variables, with $S_t, S_{t+1} \in \mathcal{S}$, $A_t \in \mathcal{A}(S_t)$, $R_{t+1} \in \mathcal{R}(S_t, A_t)$ [Book §2.3, p. 19].

Definition (State value).

$$v_\pi(s) \doteq \mathbb{E}[G_t \mid S_t = s], \quad G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

v_π is called the **state-value function**, or simply the **state value** of s under π .

Three properties the book emphasises [Book §2.3, pp. 19–20]:

1. $v_\pi(s)$ depends on s , because it is a *conditional* expectation.
2. $v_\pi(s)$ depends on π , because the trajectories are generated by π . Different policy, different number.
3. $v_\pi(s)$ does *not* depend on t . Once the policy and the (stationary) model are fixed, the number is determined.

Remark (Return versus state value). If both the policy and the model are deterministic, starting from s always produces the same trajectory, so the return equals the state value — one number, no averaging. If either is stochastic, different trajectories arise and the state value is the *mean* of their returns [Book §2.3, p. 20]. This is why return₃ in §2.1 above was already secretly a state value.

The ranking rule. π_1 is better than π_2 if $v_{\pi_1}(s) \geq v_{\pi_2}(s)$ for *all* $s \in \mathcal{S}$. Note this is a partial order: two policies can be incomparable if each is better at some states. Chapter 3 shows, remarkably, that a policy exists which dominates *every* other policy at *every* state simultaneously [Book Definition 3.1, §3.2, p. 38].

§2.4 The Bellman equation

► Book: §2.4, pp. 20–22; equations (2.4)–(2.7) ► L2 P3 — derivation

Start from the bootstrapping identity $G_t = R_{t+1} + \gamma G_{t+1}$ and take conditional expectations [Book equation (2.4), p. 20]:

$$v_\pi(s) = \mathbb{E}[G_t \mid S_t = s] = \mathbb{E}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}[G_{t+1} \mid S_t = s].$$

Now expand each piece.

The immediate-reward term. By the law of total expectation [Book equation (2.5), p. 21]:

$$\mathbb{E}[R_{t+1} \mid S_t = s] = \sum_{a \in \mathcal{A}} \pi(a \mid s) \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{r \in \mathcal{R}} p(r \mid s, a) r.$$

Read it right to left: given the action, average over rewards; then average over actions using the policy.

The future-reward term. [Book equation (2.6), p. 21]

$$\begin{aligned} \mathbb{E}[G_{t+1} \mid S_t = s] &= \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1} \mid S_t = s, S_{t+1} = s'] p(s' \mid s) \\ &= \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1} \mid S_{t+1} = s'] p(s' \mid s) && \text{(Markov property)} \\ &= \sum_{s' \in \mathcal{S}} v_\pi(s') p(s' \mid s) = \sum_{s' \in \mathcal{S}} v_\pi(s') \sum_{a \in \mathcal{A}} p(s' \mid s, a) \pi(a \mid s). \end{aligned}$$

The second line is the *only* place the Markov property is used: knowing you arrived at s' makes the route you took irrelevant.

Putting them together.

Definition (Bellman equation, textbook equation (2.7), p. 21).

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left[\underbrace{\sum_{r \in \mathcal{R}} p(r | s, a) r}_{\text{mean immediate reward}} + \gamma \underbrace{\sum_{s' \in \mathcal{S}} p(s' | s, a) v_{\pi}(s')}_{\text{mean future reward}} \right], \quad \forall s \in \mathcal{S}.$$

Four things to notice [Book §2.4, p. 22]:

- $v_{\pi}(s)$ and $v_{\pi}(s')$ are the **unknowns**. It is not one equation — it is $|\mathcal{S}|$ equations, one per state, and they are *linear* in the unknowns.
- $\pi(a | s)$ is **given**. Solving the system for a given π is called **policy evaluation**, and it is a subroutine inside almost every algorithm in the rest of the book.
- $p(r | s, a)$ and $p(s' | s, a)$ are the **model**. Chapters 5–10 are about what to do when you do not have them.
- Two equivalent forms appear in the literature [Book §2.4, p. 22]: $v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma v_{\pi}(s')]$ (used by Sutton & Barto), and, when the reward depends only on the next state, $v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s'} p(s' | s, a) [r(s') + \gamma v_{\pi}(s')]$.

§2.5 Two fully worked Bellman examples

► Book: §2.5, pp. 22–25; Figures 2.4–2.5

Worked example: a deterministic policy [Book Figure 2.4, p. 23]

2×2 grid, s_2 forbidden, s_4 target, $\gamma = 0.9$. The policy sends s_1 down, s_2 down, s_3 right, and s_4 stays.

Step 1: state s_1 . Under this policy $\pi(a_3 | s_1) = 1$ and $\pi(a \neq a_3 | s_1) = 0$. Transitions: $p(s_3 | s_1, a_3) = 1$. Rewards: $p(r = 0 | s_1, a_3) = 1$. Substituting into (2.7) collapses the triple sum to a single term:

$$v_{\pi}(s_1) = 0 + \gamma v_{\pi}(s_3).$$

Note how a formidable-looking formula becomes trivial once the policy and model are deterministic — this is exactly the book's point.

Step 2: the other states.

$$v_{\pi}(s_2) = 1 + \gamma v_{\pi}(s_4), \quad v_{\pi}(s_3) = 1 + \gamma v_{\pi}(s_4), \quad v_{\pi}(s_4) = 1 + \gamma v_{\pi}(s_4).$$

Step 3: solve. The last equation gives $v_{\pi}(s_4) = 1/(1 - \gamma)$ and the rest follow by back-substitution:

$$v_{\pi}(s_4) = v_{\pi}(s_3) = v_{\pi}(s_2) = \frac{1}{1 - \gamma}, \quad v_{\pi}(s_1) = \frac{\gamma}{1 - \gamma}.$$

Step 4: put in $\gamma = 0.9$.

$$v_{\pi}(s_4) = v_{\pi}(s_3) = v_{\pi}(s_2) = \frac{1}{0.1} = 10, \quad v_{\pi}(s_1) = \frac{0.9}{0.1} = 9.$$

Worked example: a stochastic policy [Book Figure 2.5, p. 24]

Same grid, but at s_1 the agent goes right or down with probability 0.5 each: $\pi(a_2 | s_1) = \pi(a_3 | s_1) = 0.5$. Going right lands on the forbidden cell s_2 and earns $r = -1$; going down lands on s_3 and earns $r = 0$.

The Bellman equation at s_1 now has two terms:

$$v_\pi(s_1) = 0.5[0 + \gamma v_\pi(s_3)] + 0.5[-1 + \gamma v_\pi(s_2)].$$

The other three states are unchanged, so again $v_\pi(s_2) = v_\pi(s_3) = v_\pi(s_4) = 1/(1 - \gamma)$ and

$$v_\pi(s_1) = -0.5 + \frac{\gamma}{1 - \gamma}.$$

With $\gamma = 0.9$: $v_\pi(s_2) = v_\pi(s_3) = v_\pi(s_4) = 10$ and $v_\pi(s_1) = -0.5 + 9 = 8.5$.

Comparison. $9 > 8.5$, and the two policies tie everywhere else, so $v_{\pi_1}(s_i) \geq v_{\pi_2}(s_i)$ for all i : the deterministic policy is better [Book §2.5, p. 25]. State values have delivered a verdict that agrees with intuition, and they did so by arithmetic rather than by inspection.

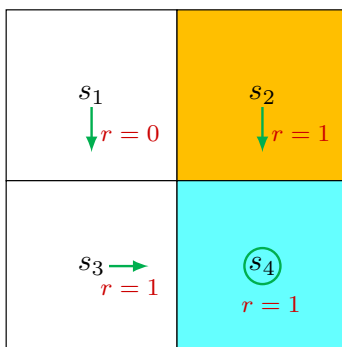


Figure 2.4. An example for demonstrating the Bellman equation. The policy in this example is deterministic.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

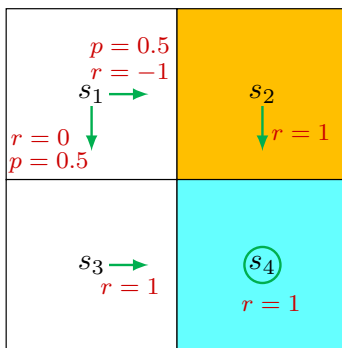


Figure 2.5. An example for demonstrating the Bellman equation. The policy in this example is stochastic.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§2.6 Matrix–vector form

► Book: §2.6, pp. 25–27; equations (2.8)–(2.10) ► L2 P4

Define, for each state, the policy-averaged reward and the policy-induced transition probability [Book §2.6, p. 26]:

$$r_\pi(s) \doteq \sum_a \pi(a | s) \sum_r p(r | s, a) r, \quad p_\pi(s' | s) \doteq \sum_a \pi(a | s) p(s' | s, a).$$

Then the Bellman equation (2.7) becomes $v_\pi(s) = r_\pi(s) + \gamma \sum_{s'} p_\pi(s' | s) v_\pi(s')$ [Book equation (2.8)], and stacking over $s_1, \dots, s_n$ with $n = |S|$:

Definition (Matrix–vector Bellman equation, equation (2.10), p. 26).

$$v_\pi = r_\pi + \gamma P_\pi v_\pi$$

where $v_\pi, r_\pi \in \mathbb{R}^n$ and $[P_\pi]_{ij} = p_\pi(s_j | s_i)$.

Fact (Properties of P_π , [Book §2.6, p. 26]). $P_\pi \geq 0$ elementwise, and P_π is a **stochastic matrix**: every row sums to one, i.e. $P_\pi \mathbf{1} = \mathbf{1}$.

Worked example: the matrix form of the stochastic example [Book Figure 2.6, p. 27]

For the four-state example with $\pi(a_2 | s_1) = \pi(a_3 | s_1) = 0.5$:

$$\begin{pmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{pmatrix} = \begin{pmatrix} 0.5(0) + 0.5(-1) \\ 1 \\ 1 \\ 1 \end{pmatrix} + \gamma \begin{pmatrix} 0 & 0.5 & 0.5 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{pmatrix}.$$

Every row of P_π sums to 1, as promised. Row 1 reads: from s_1 , go to s_2 with probability 0.5 and to s_3 with probability 0.5.

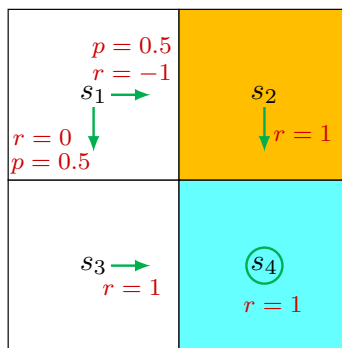


Figure 2.6. An example for demonstrating the matrix-vector form of the Bellman equation.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§2.7 Solving for state values

► Book: §2.7, pp. 27–30

§2.7.1 Method 1: closed form

► Book: §2.7.1, p. 27

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi.$$

Theorem 2.1 (Invertibility). $I - \gamma P_\pi$ is invertible for every $\gamma \in (0, 1)$ and every policy π .

Proof sketch (the book's argument, p. 27). Use the Gershgorin circle theorem: every eigenvalue lies in at least one disc centred at a diagonal entry $[I - \gamma P_\pi]_{ii} = 1 - \gamma p_\pi(s_i | s_i)$ with radius $\sum_{j \neq i} \gamma p_\pi(s_j | s_i)$. Because rows of P_π sum to one and $\gamma < 1$,

$$\sum_{j \neq i} \gamma p_\pi(s_j | s_i) = \gamma(1 - p_\pi(s_i | s_i)) < 1 - \gamma p_\pi(s_i | s_i),$$

so no disc contains the origin, so no eigenvalue is zero. □

Two further properties the book records [Book §2.7.1, p. 27], both useful later:

- $(I - \gamma P_\pi)^{-1} = I + \gamma P_\pi + \gamma^2 P_\pi^2 + \dots \geq I \geq 0$ (a Neumann series with nonnegative terms).
- If $r \geq 0$ then $(I - \gamma P_\pi)^{-1} r \geq r \geq 0$; and if $r_1 \geq r_2$ then $(I - \gamma P_\pi)^{-1} r_1 \geq (I - \gamma P_\pi)^{-1} r_2$. Monotonicity of this operator is what makes “a better reward gives a better value” true.

Common pitfall. The closed form is a theoretical device, not an algorithm. Inverting an $n \times n$ matrix costs $O(n^3)$, and real problems have n in the millions. The book says so explicitly [Book §2.7.2, p. 28].

§2.7.2 Method 2: iteration

► Book: §2.7.2, p. 28; equations (2.11)–(2.12); Box 2.1, p. 28

Pick any $v_0 \in \mathbb{R}^n$ and iterate

$$v_{k+1} = r_\pi + \gamma P_\pi v_k, \quad k = 0, 1, 2, \dots$$

Then $v_k \rightarrow v_\pi = (I - \gamma P_\pi)^{-1} r_\pi$ as $k \rightarrow \infty$.

Convergence proof (the book's Box 2.1, p. 28). Let $\delta_k = v_k - v_\pi$. Substituting $v_k = \delta_k + v_\pi$ into the iteration and using $v_\pi = r_\pi + \gamma P_\pi v_\pi$,

$$\delta_{k+1} + v_\pi = r_\pi + \gamma P_\pi(\delta_k + v_\pi) \implies \delta_{k+1} = \gamma P_\pi \delta_k = \dots = \gamma^{k+1} P_\pi^{k+1} \delta_0.$$

Every entry of P_π^k lies in $[0, 1]$ for all k (products of stochastic matrices are stochastic), while $\gamma^k \rightarrow 0$. Hence $\delta_k \rightarrow 0$. □

Intuition. The error shrinks by a factor of at least γ every sweep. With $\gamma = 0.9$ you lose one digit of error roughly every 22 iterations ($0.9^{22} \approx 0.10$); with $\gamma = 0.5$, every 3.3 iterations. Small γ means fast convergence but a short-sighted agent — the first of many discount-rate trade-offs.

§2.7.3 What state values look like

► Book: §2.7.3, pp. 28–30; Figure 2.7, p. 29

The book applies the iteration to a 5×5 grid with $r_{\text{boundary}} = r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$, $\gamma = 0.9$, and shows four policies (Figure 2.7, p. 29). Two lessons:

- **Good policies have large values.** The two “good” policies both produce the value grid whose row 1 reads 3.5, 3.9, 4.3, 4.8, 5.3 and whose target cell holds 10.0 [Book Figure 2.7(a), p. 29].
- **Different policies can share the same state values.** The two good policies in Figure 2.7(a) differ at the top two states of column 4 yet have identical values. *Values do not determine the policy uniquely.* This prefigures the non-uniqueness of optimal policies in Chapter 3 [Book §3.4, p. 48].
- **Bad policies have small, often negative values.** The “bad” policies in Figure 2.7(b) produce values like $-6.6, -7.3, -8.1, -9.0, -10.0$ across row 1.

§2.8 From state values to action values

► Book: §2.8, pp. 30–33 ► L2 P5

Definition (Action value).

$$q_{\pi}(s, a) \doteq \mathbb{E}[G_t \mid S_t = s, A_t = a].$$

It is the expected return from committing to action a at s and following π forever after. Strictly it should be called a state–action value; everyone says “action value” or “ Q -value”.

Two identities link v and q — the book calls them “two sides of the same coin” [Book §2.8, p. 31].

Theorem 2.2 (The v – q relations, equations (2.13)–(2.14), pp. 30–31).

$$(2.13) \quad v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) q_{\pi}(s, a),$$

$$(2.14) \quad q_{\pi}(s, a) = \sum_{r \in \mathcal{R}} p(r \mid s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) v_{\pi}(s').$$

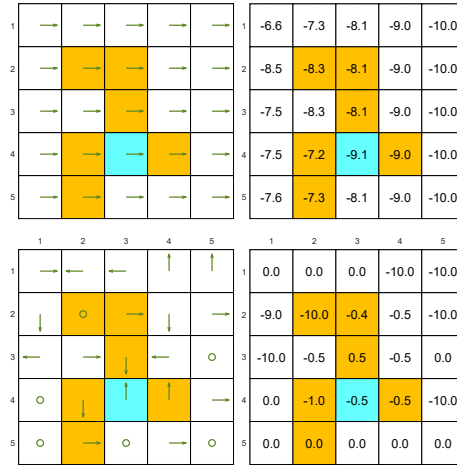
Equation (2.13) says a state value is the policy-weighted average of that state’s action values. Equation (2.14) says an action value is the mean immediate reward plus the discounted mean of the *next* states’ values. Substituting (2.14) into (2.13) recovers the Bellman equation — which is a good consistency check.

Worked example: action values at s_1 , and a mistake to avoid [Book §2.8.1, pp. 31–32; Figure 2.8]

Take the stochastic policy of Figure 2.8 (p. 31): at s_1 the agent goes right (a_2 , into forbidden s_2 , $r = -1$) or down (a_3 , into s_3 , $r = 0$), each with probability 0.5. The values computed earlier were $v_{\pi}(s_1) = 8.5$, $v_{\pi}(s_2) = v_{\pi}(s_3) = v_{\pi}(s_4) = 10$, $\gamma = 0.9$.



(a) Two “good” policies and their state values. The state values of the two policies are the same, but the two policies are different at the top two states in the fourth column.



(b) Two “bad” policies and their state values. The state values are smaller than those of the “good” policies.

Figure 2.7. Examples of policies and their corresponding state values.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

The two actions the policy does take:

$$q_{\pi}(s_1, a_2) = -1 + \gamma v_{\pi}(s_2) = -1 + 9 = 8, \quad q_{\pi}(s_1, a_3) = 0 + \gamma v_{\pi}(s_3) = 9.$$

The three actions the policy never takes. A beginner might set these to zero, or refuse to compute them. *Both are wrong* [Book §2.8.1, p. 31]. They are perfectly well defined: take the action once, then follow π afterwards. At s_1 , actions a_1 (up) and a_4 (left) bounce off the wall, so the agent returns to s_1 with $r = -1$; action a_5 stays at s_1 with $r = 0$:

$$q_{\pi}(s_1, a_1) = -1 + \gamma v_{\pi}(s_1) = -1 + 0.9(8.5) = 6.65,$$

$$q_{\pi}(s_1, a_4) = -1 + \gamma v_{\pi}(s_1) = 6.65, \quad q_{\pi}(s_1, a_5) = 0 + \gamma v_{\pi}(s_1) = 7.65.$$

Consistency check via (2.13):

$$v_{\pi}(s_1) = 0.5 q_{\pi}(s_1, a_2) + 0.5 q_{\pi}(s_1, a_3) = 0.5(8) + 0.5(9) = 8.5. \checkmark$$

Common pitfall (Why we care about actions the policy will not take). This is the single most important remark in Chapter 2 and the reason the whole book works [Book §2.8.1, p. 32; §2.10 Q&A, p. 34]. If a policy never selects action a at s , that is not evidence that a is bad — it may be evidence that the *policy* is bad and is missing the best action. In this very example, $q_\pi(s_1, a_3) = 9$ is the largest of the five, so a better policy would put all its probability on a_3 . **Policy improvement is nothing but comparing action values, including the ones you are not currently using.** That is why exploration matters (Chapter 5) and why model-free methods estimate q rather than v (Chapter 5, §5.2.1, p. 80).

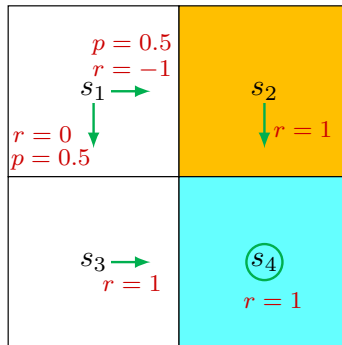


Figure 2.8. An example for demonstrating the process of calculating action values.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§2.8.1 The Bellman equation in action values

► Book: §2.8.2, pp. 32–33; equation (2.15)

Substituting (2.13) into (2.14) eliminates v entirely:

$$q_\pi(s, a) = \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) \sum_{a' \in \mathcal{A}(s')} \pi(a' | s') q_\pi(s', a').$$

In matrix form [Book equation (2.15), p. 32]: $q_\pi = \tilde{r} + \gamma P \Pi q_\pi$, where q_π and $\tilde{r}$ are indexed by state–action pairs, $[P]_{(s,a),s'} = p(s' | s, a)$, and Π is block diagonal with $\Pi_{s',(s',a')} = \pi(a' | s')$.

Remark. This version has a feature that matters in Chapters 7–8: $\tilde{r}$ and P depend only on the *environment*, and the policy is quarantined inside Π . This separation is what makes off-policy learning conceivable [Book §2.8.2, p. 33].

§2.9 Chapter 2 self-check

1. State the Bellman equation from memory, then say in words what each of the four sums is averaging over.
2. Where exactly in the derivation is the Markov property used, and what would break without it? (*Answer: equation (2.6), p. 21.*)
3. In the $\gamma = 0.9$ stochastic example, verify by hand that $v_\pi(s_1) = 8.5$ satisfies its own Bellman equation.

4. Two policies have identical state values everywhere. Must they be the same policy? (*No — Figure 2.7(a), p. 29.*)
5. Why must a model-free algorithm estimate q_π rather than v_π ? (*Look at (2.14): converting v to q needs $p(r \mid s, a)$ and $p(s' \mid s, a)$; [Book §5.2.1, p. 81].*)

§3 Optimal State Values and the Bellman Optimality Equation

READING MAP — Chapter 3 of the textbook

Textbook: Chapter 3, pp. 35–55. Sections: §3.1 motivating example — how to improve policies (p. 36), §3.2 optimal state values and optimal policies (p. 37), §3.3 the Bellman optimality equation (p. 38; maximisation of the RHS p. 39, matrix–vector form p. 40, contraction mapping theorem p. 40, contraction property of the RHS p. 44), §3.4 solving an optimal policy from the BOE (p. 46), §3.5 factors that influence optimal policies (p. 49), §3.6 summary (p. 53), §3.7 Q&A (p. 54).

Videos: ► L3 P1 — Motivating example; ► L3 P2 — Optimal policy; ► L3 P3 — More on the BOE; ► L3 P4 — Interesting properties.

Warning from the author: this is the most mathematically intensive chapter of the first half, and it is worth it, because it answers the four questions — does an optimal policy exist, is it unique, is it stochastic, how do we find it [Book §3.2, p. 38].

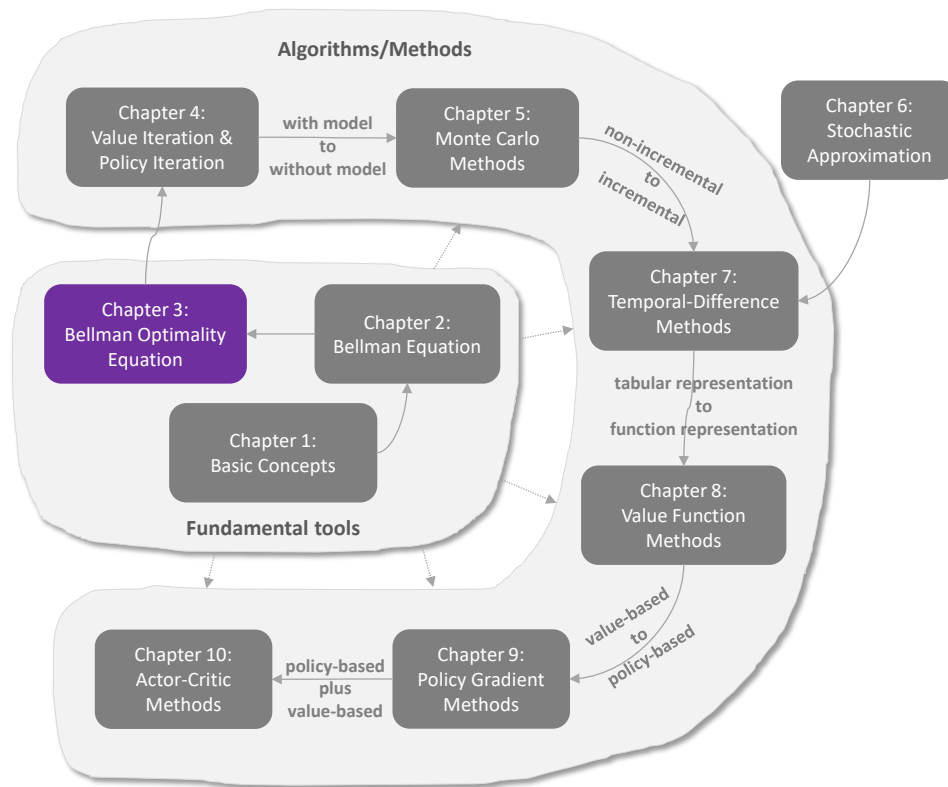


Figure 3.1. Where we are in this book.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§3.1 Motivating example: how do you improve a policy?

► Book: §3.1, pp. 36–37; Figure 3.2, p. 36

Take the 2×2 grid (s_2 forbidden, s_4 target, $\gamma = 0.9$) with a policy that stupidly sends the agent *right* at s_1 , straight into the forbidden cell. Can we fix it, and can we justify the fix with arithmetic rather than with eyeballs?

Worked example: one step of policy improvement [Book §3.1, pp. 36–37]

Step 1 — evaluate the current policy. Its Bellman equations are

$$v_\pi(s_1) = -1 + \gamma v_\pi(s_2), \quad v_\pi(s_2) = +1 + \gamma v_\pi(s_4), \quad v_\pi(s_3) = +1 + \gamma v_\pi(s_4), \quad v_\pi(s_4) = +1 + \gamma v_\pi(s_4).$$

With $\gamma = 0.9$: $v_\pi(s_4) = v_\pi(s_3) = v_\pi(s_2) = 10$ and $v_\pi(s_1) = -1 + 9 = 8$.

Step 2 — compute all five action values at s_1 using (2.14):

$$\begin{aligned} q_\pi(s_1, a_1) &= -1 + \gamma v_\pi(s_1) = -1 + 7.2 = 6.2 && \text{(up: bounces off wall)} \\ q_\pi(s_1, a_2) &= -1 + \gamma v_\pi(s_2) = -1 + 9 = 8 && \text{(right: into forbidden cell)} \\ q_\pi(s_1, a_3) &= 0 + \gamma v_\pi(s_3) = 9 && \text{(down: the good move)} \\ q_\pi(s_1, a_4) &= -1 + \gamma v_\pi(s_1) = 6.2 && \text{(left: bounces off wall)} \\ q_\pi(s_1, a_5) &= 0 + \gamma v_\pi(s_1) = 7.2 && \text{(stay)} \end{aligned}$$

Step 3 — act greedily. $q_\pi(s_1, a_3) = 9$ is the largest, so set $\pi_{\text{new}}(a_3 | s_1) = 1$. The new policy avoids the forbidden cell.

Intuition. This three-line recipe — evaluate, compute q , take the argmax — is the skeleton of *every* algorithm in Chapters 4, 5, 7 and 8. What changes across chapters is only *how* you get the q values: by solving equations (Chapter 4), by averaging sampled returns (Chapter 5), by one-step bootstrapped updates (Chapter 7), or by fitting a function (Chapter 8).

The example leaves open the hard questions: what if the policy is bad at *many* states? Does greedy improvement always work? Does an optimal policy even exist? Chapter 3 answers all of them.

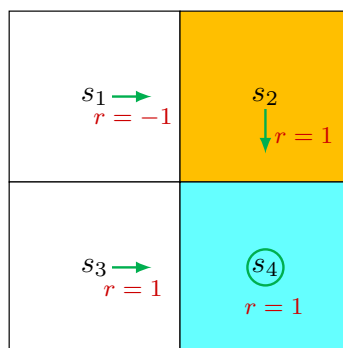


Figure 3.2. An example for demonstrating policy improvement.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§3.2 Defining optimality

► Book: §3.2, pp. 37–38; Definition 3.1, p. 38

Definition (Optimal policy and optimal state value, Definition 3.1, p. 38). A policy π^* is **optimal** if

$$v_{\pi^*}(s) \geq v_{\pi}(s) \quad \text{for all } s \in \mathcal{S} \text{ and for every other policy } \pi.$$

Its state values $v^* = v_{\pi^*}$ are the **optimal state values**.

Common pitfall. Read the quantifiers carefully. This is a very strong demand: π^* must beat every rival at *every state simultaneously*. Nothing in the definition guarantees such a policy exists — *a priori* you would expect trade-offs, where policy A is better at some states and policy B at others. The content of Chapter 3 is that for finite discounted MDPs no such trade-off arises. That is a genuinely surprising theorem, not a definition.

Remark. The book also flags a scope limitation [Book §3.7 Q&A, p. 54]: this definition of optimality is valid for *tabular* RL. Once values or policies are approximated by functions (Chapters 8–9), you cannot achieve the maximum at every state at once, and optimality must be redefined via a scalar objective [Book §9.2, p. 193].

§3.3 The Bellman optimality equation

► Book: §3.3, pp. 38–39; equation (3.1), p. 38

Definition (BOE, elementwise, equation (3.1), p. 38). For every $s \in \mathcal{S}$,

$$v(s) = \max_{\pi(s) \in \Pi(s)} \sum_{a \in \mathcal{A}} \pi(a | s) \left[\sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v(s') \right] = \max_{\pi(s) \in \Pi(s)} \sum_a \pi(a | s) q(s, a),$$

where $q(s, a) \doteq \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v(s')$.

Compare with the ordinary Bellman equation: the policy-weighted average $\sum_a \pi(a | s)(\cdot)$ has become a *maximisation over policies*. That one change makes the equation **nonlinear** — you can no longer invert a matrix — and introduces a second unknown: both $v(s)$ and π are unknown.

§3.3.1 Solving the maximisation on the right-hand side

► Book: §3.3.1, pp. 39–40; Examples 3.1–3.2

Worked example: Example 3.1, p. 39 — two unknowns, one equation

Solve $x = \max_{y \in \mathbb{R}} (2x - 1 - y^2)$ for $x, y \in \mathbb{R}$.

Step 1: eliminate y . Whatever x is, $\max_y (2x - 1 - y^2) = 2x - 1$, attained at $y = 0$.

Step 2: now solve x . The equation becomes $x = 2x - 1$, so $x = 1$.

Answer: $y = 0, x = 1$. Two unknowns, one equation — solved by doing the inner optimisation first. **The BOE is solved the same way.**

Worked example: Example 3.2, p. 39 — how to maximise a weighted average

Given fixed numbers q_1, q_2, q_3 , choose weights $c_1, c_2, c_3 \geq 0$ with $c_1 + c_2 + c_3 = 1$ to maximise $\sum_i c_i q_i$.

Suppose $q_3 \geq q_1, q_2$. Then

$$q_3 = (c_1 + c_2 + c_3)q_3 = c_1q_3 + c_2q_3 + c_3q_3 \geq c_1q_1 + c_2q_2 + c_3q_3$$

for any admissible weights. So the maximum is q_3 , attained at $c_3^* = 1$, $c_1^* = c_2^* = 0$.

Moral: to maximise a weighted average, put all the weight on the largest item. Not half on the largest and half on the second largest — *all* of it.

Applying Example 3.2 with $c_a = \pi(a \mid s)$ and $q_a = q(s, a)$ [Book §3.3.1, p. 40]:

$$\sum_a \pi(a \mid s) q(s, a) \leq \max_{a \in \mathcal{A}} q(s, a), \quad \text{with equality iff } \pi(a \mid s) = \begin{cases} 1, & a = a^*, \\ 0, & a \neq a^*, \end{cases}$$

where $a^* = \arg \max_a q(s, a)$. So the BOE simplifies to

$$v(s) = \max_{a \in \mathcal{A}(s)} q(s, a),$$

and the maximiser is the **greedy deterministic** policy.

§3.3.2 Matrix–vector form

► Book: §3.3.2, p. 40; equations (3.2)–(3.3)

$$v = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v) \doteq f(v), \quad \text{so the BOE is } v = f(v),$$

where the max is taken elementwise and r_π, P_π are exactly as in Chapter 2. We have reduced the whole problem to a *fixed-point equation*.

§3.4 The contraction mapping theorem

► Book: §3.3.3, pp. 40–44; Theorem 3.1, p. 41; proof in Box 3.1, pp. 42–44

Definition. x^* is a **fixed point** of $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ if $f(x^*) = x^*$. The map f is a **contraction mapping** if there is $\gamma \in (0, 1)$ with

$$\|f(x_1) - f(x_2)\| \leq \gamma \|x_1 - x_2\| \quad \forall x_1, x_2 \in \mathbb{R}^d.$$

Worked example: three contractions [Book Example 3.3, p. 41]

1. $f(x) = 0.5x$ on $\mathbb{R}$. Fixed point $x = 0$; and $|0.5x_1 - 0.5x_2| = 0.5|x_1 - x_2|$, so it contracts with γ any number in $[0.5, 1)$.
2. $f(x) = Ax$ on $\mathbb{R}^n$ with $\|A\| \leq \gamma < 1$. Fixed point $x = 0$, and $\|Ax_1 - Ax_2\| \leq \|A\| \|x_1 - x_2\| \leq \gamma \|x_1 - x_2\|$.
3. $f(x) = 0.5 \sin x$ on $\mathbb{R}$. Fixed point $x = 0$. By the mean value theorem $\left| \frac{0.5 \sin x_1 - 0.5 \sin x_2}{x_1 - x_2} \right| = |0.5 \cos x_3| \leq 0.5$ for some x_3 between x_1, x_2 , so it contracts with $\gamma = 0.5$.

Theorem 3.1 (Contraction mapping / Banach fixed-point theorem, Theorem 3.1, p. 41). *If f is a contraction mapping then:*

1. **Existence:** a fixed point x^* with $f(x^*) = x^*$ exists;
2. **Uniqueness:** it is the only one;
3. **Algorithm:** for any starting point x_0 , the iterates $x_{k+1} = f(x_k)$ converge to x^* , and they do so exponentially fast.

Proof sketch (the book's Box 3.1, pp. 42–44). Part 1 — the sequence is Cauchy. Contraction gives $\|x_{k+1} - x_k\| \leq \gamma \|x_k - x_{k-1}\| \leq \dots \leq \gamma^k \|x_1 - x_0\|$. That alone is not enough (the book warns: $x_n = \sqrt{n}$ has $x_{n+1} - x_n \rightarrow 0$ but diverges). So bound the general gap, for $m > n$:

$$\|x_m - x_n\| \leq \sum_{i=n}^{m-1} \|x_{i+1} - x_i\| \leq (\gamma^{m-1} + \dots + \gamma^n) \|x_1 - x_0\| \leq \frac{\gamma^n}{1-\gamma} \|x_1 - x_0\|,$$

the book's inequality (3.4), p. 43. Given any $\varepsilon > 0$, choose N making this $< \varepsilon$: the sequence is Cauchy, hence convergent, to some x^* .

Part 2 — the limit is a fixed point. $\|f(x_k) - x_k\| = \|x_{k+1} - x_k\| \leq \gamma^k \|x_1 - x_0\| \rightarrow 0$, so $f(x^*) = x^*$.

Part 3 — uniqueness. If $f(x') = x'$ too, then $\|x' - x^*\| = \|f(x') - f(x^*)\| \leq \gamma \|x' - x^*\|$, which with $\gamma < 1$ forces $\|x' - x^*\| = 0$.

Part 4 — rate. Letting $m \rightarrow \infty$ in the bound above, $\|x^* - x_n\| \leq \frac{\gamma^n}{1-\gamma} \|x_1 - x_0\|$: geometric decay. $\square$

§3.4.1 The right-hand side of the BOE is a contraction

► Book: §3.3.4, pp. 44–46; Theorem 3.2, p. 44; proof in Box 3.2, pp. 44–45

Theorem 3.2 (Theorem 3.2, p. 44). $f(v) = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v)$ satisfies

$$\|f(v_1) - f(v_2)\|_\infty \leq \gamma \|v_1 - v_2\|_\infty \quad \forall v_1, v_2 \in \mathbb{R}^{|S|},$$

where $\|\cdot\|_\infty$ is the maximum absolute entry.

Proof sketch (Box 3.2, pp. 44–45). Let $\pi_1^* = \arg \max_{\pi} (r_\pi + \gamma P_\pi v_1)$ and $\pi_2^* = \arg \max_{\pi} (r_\pi + \gamma P_\pi v_2)$. Since π_1^* is optimal for v_1 but merely feasible for v_2 (and vice versa),

$$f(v_1) - f(v_2) \leq (r_{\pi_1^*} + \gamma P_{\pi_1^*} v_1) - (r_{\pi_1^*} + \gamma P_{\pi_1^*} v_2) = \gamma P_{\pi_1^*} (v_1 - v_2),$$

and symmetrically $f(v_2) - f(v_1) \leq \gamma P_{\pi_2^*} (v_2 - v_1)$. Each row $p^\top$ of a stochastic matrix has nonnegative entries summing to one, so $|p^\top (v_1 - v_2)| \leq p^\top |v_1 - v_2| \leq \|v_1 - v_2\|_\infty$. Combining gives the claim. $\square$

Intuition (Why ℓ^∞ and not ℓ^2 ?). Because P_π is a stochastic matrix, applying it to a vector produces averages of that vector's entries, and an average is never larger in absolute value than the largest entry. That is a statement about the max-norm, and it is exactly why the max-norm is the natural metric for MDPs. Multiplying by γ then strictly shrinks it.

§3.5 Solving the BOE

► Book: §3.4, pp. 46–48; Theorems 3.3–3.5

Theorem 3.3 (Existence, uniqueness, algorithm — Theorem 3.3, p. 46). *The BOE $v = f(v) = \max_{\pi \in \Pi}(r_\pi + \gamma P_\pi v)$ has exactly one solution v^* , which can be computed by*

$$v_{k+1} = f(v_k) = \max_{\pi \in \Pi}(r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, \dots$$

from any initial guess v_0 , and $v_k \rightarrow v^$ exponentially fast.*

This iteration has a name: **value iteration**. Chapter 4 is about implementing it [Book §3.4, p. 46 and §4.1, p. 58].

Once v^* is known, the policy follows immediately: $\pi^* = \arg \max_{\pi \in \Pi}(r_\pi + \gamma P_\pi v^*)$ [Book equation (3.6), p. 46]. Substituting back gives $v^* = r_{\pi^*} + \gamma P_{\pi^*} v^*$, i.e. $v^* = v_{\pi^*}$:

Remark. The BOE is just the ordinary Bellman equation of the optimal policy. That is the sentence to remember from this chapter [Book §3.4, p. 46].

Theorem 3.4 (Optimality — Theorem 3.4, p. 47). *The solution v^* of the BOE is the optimal state value and π^* is an optimal policy: $v^* = v_{\pi^*} \geq v_\pi$ elementwise, for every policy π .*

Proof (Box 3.3, p. 47). For any π , $v_\pi = r_\pi + \gamma P_\pi v_\pi$, while $v^* = \max_{\pi'}(r_{\pi'} + \gamma P_{\pi'} v^*) \geq r_\pi + \gamma P_\pi v^*$. Subtracting,

$$v^* - v_\pi \geq \gamma P_\pi(v^* - v_\pi) \geq \gamma^2 P_\pi^2(v^* - v_\pi) \geq \dots \geq \gamma^n P_\pi^n(v^* - v_\pi) \xrightarrow{n \rightarrow \infty} 0,$$

because $\gamma^n \rightarrow 0$ and P_π^n is stochastic (entries in $[0, 1]$). □

Theorem 3.5 (Greedy optimal policy — Theorem 3.5, p. 47; equation (3.7)). *For every s , the deterministic greedy policy*

$$\pi^*(a \mid s) = \begin{cases} 1, & a = a^*(s), \\ 0, & a \neq a^*(s), \end{cases} \quad a^*(s) = \arg \max_a q^*(s, a),$$

with $q^(s, a) = \sum_r p(r \mid s, a)r + \gamma \sum_{s'} p(s' \mid s, a)v^*(s')$, is optimal.*

§3.5.1 Uniqueness and stochasticity

► Book: §3.4, p. 48; Figure 3.3, p. 48

- v^* is **unique** (fixed point of a contraction).
- π^* **need not be unique**. Figure 3.3, p. 48, exhibits two different policies that are both optimal. If two actions tie in q^* , any mixture of them is optimal too.
- **Optimal policies may be stochastic, but a deterministic one always exists** (Theorem 3.5). This is why every algorithm in the book can safely search among deterministic greedy policies.

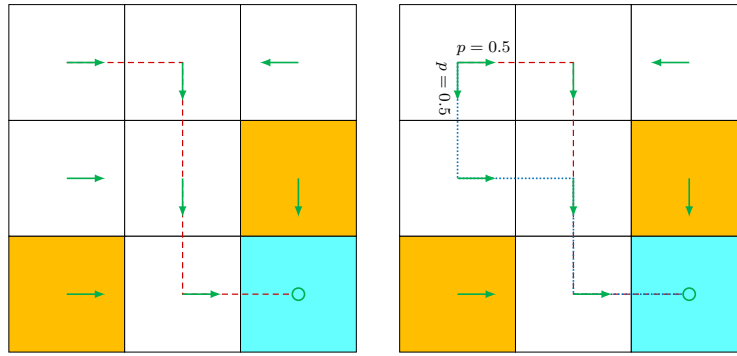


Figure 3.3. Examples for demonstrating that optimal policies may not be unique. The two policies are different but are both optimal.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§3.6 What actually determines the optimal policy

► Book: §3.5, pp. 49–53; Figure 3.4, p. 50 ► L3 P4

Only three things appear in the BOE, so only three things can matter: (i) the rewards r ; (ii) the discount rate γ ; (iii) the model $p(s' | s, a), p(r | s, a)$. The model is given, so the book varies the other two on a 5×5 grid [Book §3.5, pp. 49–53].

Worked example: the discount rate changes the personality of the agent [Book Figure 3.4, p. 50]

Baseline: $r_{\text{boundary}} = r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$, $r_{\text{other}} = 0$, $\gamma = 0.9$.

(a) $\gamma = 0.9$. Optimal values, row by row:

5.8	5.6	6.2	6.5	5.8
6.5	7.2	8.0	7.2	6.5
7.2	8.0	10.0	8.0	7.2
8.0	10.0	10.0	10.0	8.0
7.2	9.0	10.0	9.0	8.1

The striking behaviour: starting from row 4, column 1, the optimal policy is willing to *cut straight through forbidden cells*, eating -1 penalties, because the shortcut to the target is worth more in discounted terms than the long safe detour. The agent is far-sighted.

(b) $\gamma = 0.5$. Same rewards, different attitude: the agent no longer dares take the risk and travels the long way round, avoiding every forbidden cell. Values collapse toward zero except near the target (2.0 at the target, 0.1 or 0.0 far away).

(c) $\gamma = 0$. The agent maximises only the immediate reward. It becomes completely myopic and *never reaches the target at all* from most cells. All values are 0 except 1.0 in the cells adjacent to/at the target.

Remark (The spatial pattern). Across all these grids, states closer to the target have larger values. The reason is purely the discount rate: a distant state needs a longer trajectory before it collects the $+1$, and γ^k shrinks it [Book §3.5, p. 51].

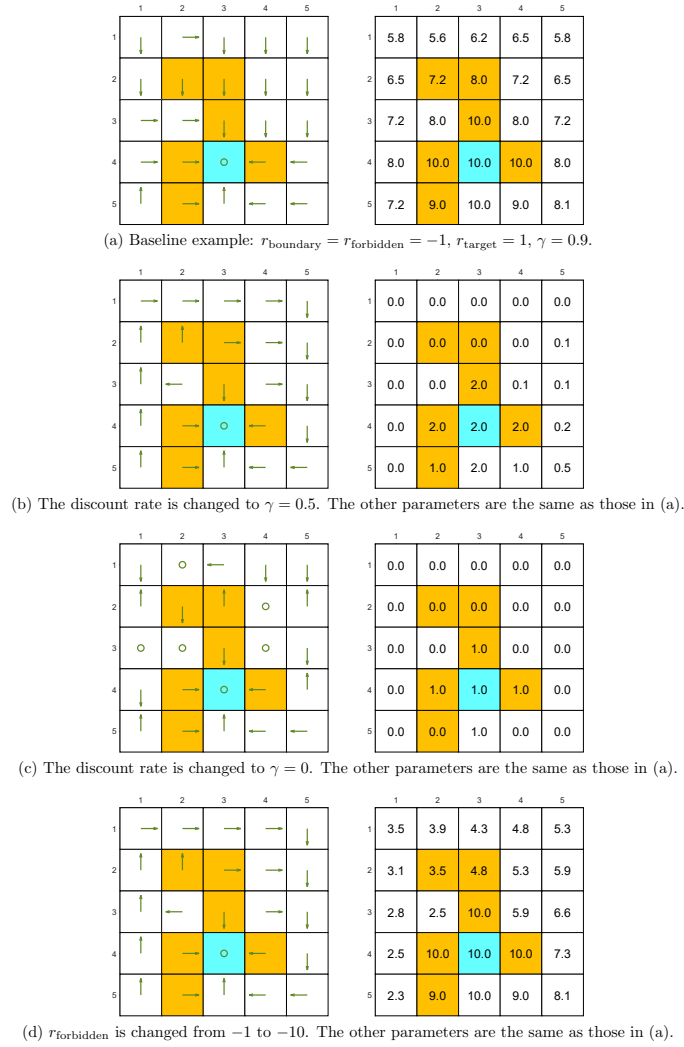


Figure 3.4. The optimal policies and optimal state values given different parameter values.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Worked example: changing a reward can change the optimal policy [Book Figure 3.4(d), p. 50]

Keep $\gamma = 0.9$ but change $r_{\text{forbidden}}$ from -1 to -10 . Now the optimal policy *never* enters a forbidden cell. The value grid becomes

3.5	3.9	4.3	4.8	5.3
3.1	3.5	4.8	5.3	5.9
2.8	2.5	10.0	5.9	6.6
2.5	10.0	10.0	10.0	7.3
2.3	9.0	10.0	9.0	8.1

So reward design is a real lever. But — and this is the subtle part — not every change to the rewards is a real change.

Theorem 3.6 (Optimal policy invariance — Theorem 3.6, p. 51; equation (3.8)). *If every reward r is replaced by $\alpha r + \beta$ with $\alpha > 0$, then the optimal state value becomes*

$$v' = \alpha v^* + \frac{\beta}{1 - \gamma} \mathbf{1},$$

and the greedy policy derived from v' is the same as the one derived from v^ .*

Proof (Box 3.5, pp. 51–52). Under $r \mapsto \alpha r + \beta$ we get $r_\pi \mapsto \alpha r_\pi + \beta \mathbf{1}$, so the new BOE is $v' = \max_\pi (\alpha r_\pi + \beta \mathbf{1} + \gamma P_\pi v')$. Guess $v' = \alpha v^* + c \mathbf{1}$ and substitute, using $P_\pi \mathbf{1} = \mathbf{1}$:

$$\alpha v^* + c \mathbf{1} = \max_\pi (\alpha r_\pi + \alpha \gamma P_\pi v^*) + \beta \mathbf{1} + c \gamma \mathbf{1} \implies \beta + c \gamma - c = 0 \implies c = \frac{\beta}{1 - \gamma}.$$

Since the BOE has a unique solution, this guess *is* the solution. An affine transformation preserves the ordering of the q -values, so the argmax is unchanged. $\square$

Common pitfall (Do you need a step penalty to prevent dithering?). [Book §3.5, pp. 52–53; Figure 3.5, p. 52] A near-universal beginner instinct: since $r_{\text{other}} = 0$, wandering costs nothing, so surely we must add -1 per step to make the agent hurry? **No, twice over.**

First, adding a constant to every reward is an affine transformation, so by Theorem 3.6 it changes nothing about the optimal policy.

Second, the discount rate already does the job. The book’s Figure 3.5 (p. 52) compares two policies in a 2×2 grid with the target at the bottom right. The direct policy $s_2 \rightarrow s_4$ earns

$$1 + \gamma + \gamma^2 + \dots = \frac{1}{1 - \gamma} = 10,$$

whereas the dithering policy $s_2 \rightarrow s_1 \rightarrow s_3 \rightarrow s_4$ earns

$$0 + \gamma \cdot 0 + \gamma^2(1 + \gamma + \dots) = \frac{\gamma^2}{1 - \gamma} = 8.1.$$

Ten beats 8.1: *the detour is punished automatically, because the reward it eventually collects arrives two periods later and is discounted twice.* An economist will recognise this instantly — it is just the time value of money.

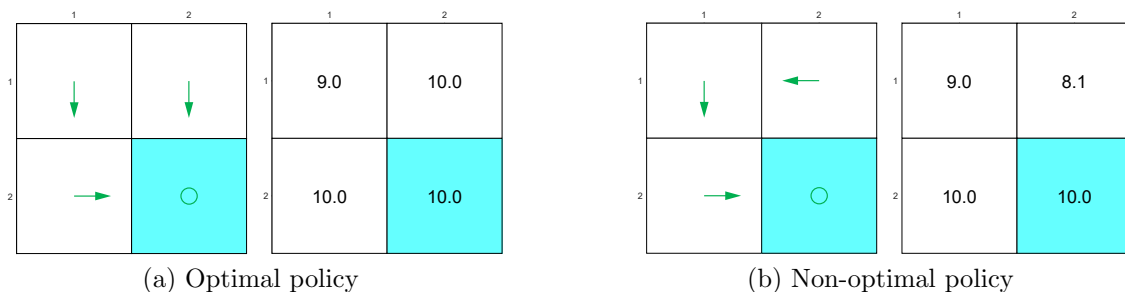


Figure 3.5. Examples illustrating that optimal policies do not take meaningless detours due to the discount rate.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§3.7 Chapter 3 self-check

1. Why does the BOE have two unknowns, and what is the two-step trick that resolves this? (*Example 3.1, p. 39.*)
2. State the three conclusions of the contraction mapping theorem and match each to a question about optimal policies (existence / uniqueness / computation).
3. Prove in one line that if $q^*(s, a_1) = q^*(s, a_2) > q^*(s, a)$ for all other a , then the policy putting probability 0.3 on a_1 and 0.7 on a_2 is also optimal.
4. Your agent takes a long safe route instead of a fast risky one, and you want it to be bolder. Name two parameters you could change and say what each would do. (*Raise γ ; or reduce $|r_{\text{forbidden}}|$; [Book §3.5, pp. 49–51].*)
5. True or false: adding +100 to every reward makes the agent reach the target faster. (*False — Theorem 3.6.*)

§4 Value Iteration and Policy Iteration

READING MAP — Chapter 4 of the textbook

Textbook: Chapter 4, pp. 57–76. Sections: §4.1 value iteration (p. 58; elementwise form p. 58, examples p. 59), §4.2 policy iteration (p. 62; analysis p. 62, elementwise form p. 65, examples p. 67), §4.3 truncated policy iteration (p. 70; comparison p. 70, algorithm p. 72), §4.4 summary (p. 74), §4.5 Q&A (p. 74). Algorithms 4.1 (p. 60), 4.2 (p. 66), 4.3 (p. 72).

Videos: ► L4 P1 — Value iteration; ► L4 P2 — Policy iteration; ► L4 P3 — Truncated policy iteration.

Status of these algorithms: they are **dynamic programming**, not really reinforcement learning, because they require the model $\{p(r | s, a), p(s' | s, a)\}$ [Book §4.1, p. 58]. They matter because every model-free algorithm in Chapters 5–10 is obtained by replacing one step of one of them with a sample estimate.

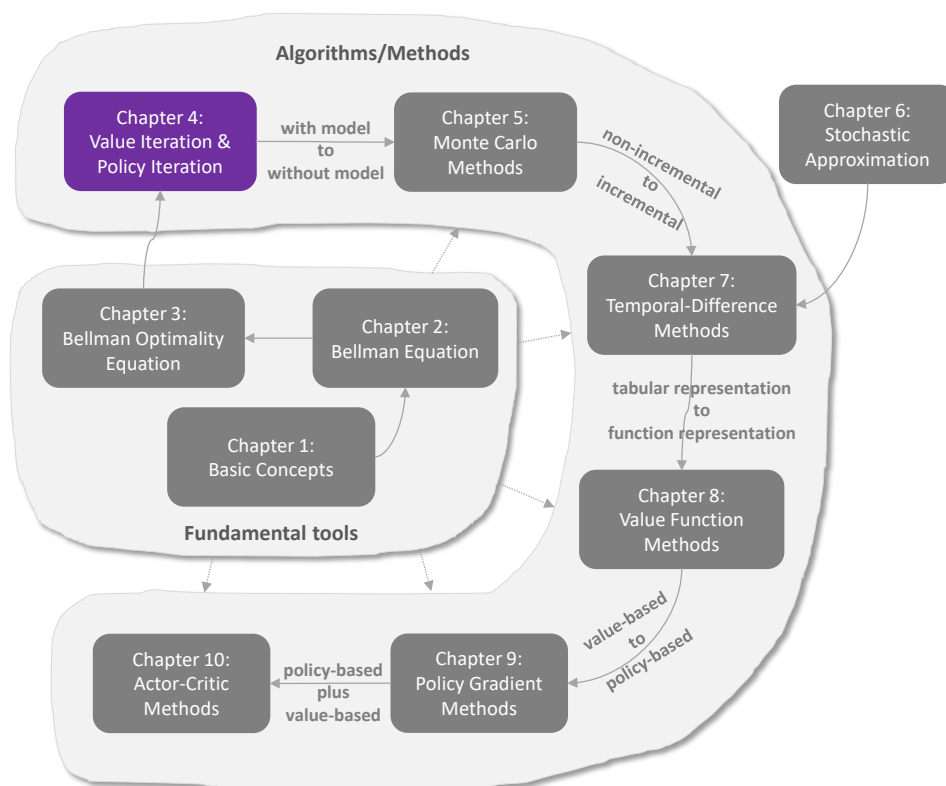


Figure 4.1. Where we are in this book.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§4.1 Value iteration

► Book: §4.1, pp. 58–59; Algorithm 4.1, p. 60

Value iteration is the contraction iteration of Theorem 3.3:

$$v_{k+1} = \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} v_k), \quad k = 0, 1, 2, \dots$$

Split each iteration into two named steps [Book §4.1, p. 58]:

1. **Policy update (PU):** $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k)$.
2. **Value update (VU):** $v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$ [Book equation (4.1), p. 58].

§4.1.1 Elementwise form — what you actually code

► Book: §4.1.1, pp. 58–59

For each state s , first compute the q -values from the current v_k :

$$q_k(s, a) = \sum_r p(r \mid s, a) r + \gamma \sum_{s'} p(s' \mid s, a) v_k(s').$$

Then

$$a_k^*(s) = \arg \max_a q_k(s, a), \quad \pi_{k+1}(a \mid s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & \text{otherwise,} \end{cases} \quad v_{k+1}(s) = \max_a q_k(s, a).$$

The book summarises the flow as [Book §4.1.1, p. 59]

$$v_k(s) \rightarrow q_k(s, a) \rightarrow \text{greedy } \pi_{k+1}(s) \rightarrow v_{k+1}(s) = \max_a q_k(s, a).$$

ALGORITHM — Value iteration (Algorithm 4.1, p. 60)

Input: the model $p(r \mid s, a)$, $p(s' \mid s, a)$ for all (s, a) ; an initial guess v_0 .

While $\|v_k - v_{k-1}\|$ exceeds a small threshold, **do**

- For every $s \in \mathcal{S}$:
 - For every $a \in \mathcal{A}(s)$: $q_k(s, a) = \sum_r p(r \mid s, a) r + \gamma \sum_{s'} p(s' \mid s, a) v_k(s')$
 - $a_k^*(s) = \arg \max_a q_k(s, a)$
 - Policy update: $\pi_{k+1}(a \mid s) = 1$ if $a = a_k^*(s)$, else 0
 - Value update: $v_{k+1}(s) = \max_a q_k(s, a)$

Common pitfall. v_k is **not a state value**. It does not satisfy the Bellman equation of π_k or of π_{k+1} or of any policy. It is a numerical iterate that happens to converge to v^* . Consequently q_k is not an action value either [Book §4.1.1, p. 59 and §4.5 Q&A, p. 74]. Contrast this with policy iteration below, where the intermediate values *are* genuine state values.

§4.1.2 A complete numerical run

► Book: §4.1.2, pp. 59–62; Figure 4.2, p. 60; Tables 4.1–4.3, pp. 60–61

The book's example is a 2×2 grid: s_1, s_2 on top, s_3, s_4 beneath; s_2 is forbidden, s_4 is the target; $r_{\text{boundary}} = r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$, $\gamma = 0.9$.

First write out the q -value expressions once (the book's Table 4.1, p. 60):

$q(s, a)$	a_1 (up)	a_2 (right)	a_3 (down)	a_4 (left)	a_5 (stay)
s_1	$-1 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$	$0 + \gamma v(s_3)$	$-1 + \gamma v(s_1)$	$0 + \gamma v(s_1)$
s_2	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_2)$	$1 + \gamma v(s_4)$	$0 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$
s_3	$0 + \gamma v(s_1)$	$1 + \gamma v(s_4)$	$-1 + \gamma v(s_3)$	$-1 + \gamma v(s_3)$	$0 + \gamma v(s_3)$
s_4	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_4)$	$-1 + \gamma v(s_4)$	$0 + \gamma v(s_3)$	$1 + \gamma v(s_4)$

Worked example: value iteration, iteration by iteration [Book §4.1.2, pp. 60–61]

$k = 0$: **start from** $v_0 = (0, 0, 0, 0)$.

Substituting into the table gives the book's Table 4.2 (p. 61):

q_0	a_1	a_2	a_3	a_4	a_5
s_1	-1	-1	0	-1	0
s_2	-1	-1	1	0	-1
s_3	0	1	-1	-1	0
s_4	-1	-1	-1	0	1

Greedy policy: $\pi_1(a_5 | s_1) = 1$, $\pi_1(a_3 | s_2) = 1$, $\pi_1(a_2 | s_3) = 1$, $\pi_1(a_5 | s_4) = 1$. (At s_1 the actions a_3 and a_5 tie at 0; either may be taken without affecting convergence [Book §4.1.1, p. 59].) This policy is visibly not optimal — it stays put at s_1 . Value update: $v_1 = (0, 1, 1, 1)$.

$k = 1$: **substitute** $v_1 = (0, 1, 1, 1)$, $\gamma = 0.9$ (the book's Table 4.3, p. 61):

q_1	a_1	a_2	a_3	a_4	a_5
s_1	$-1 + 0 = -1$	$-1 + 0.9 = -0.1$	$0 + 0.9 = \mathbf{0.9}$	-1	0
s_2	-0.1	-0.1	$1 + 0.9 = \mathbf{1.9}$	0	-0.1
s_3	0	$1 + 0.9 = \mathbf{1.9}$	-0.1	-0.1	0.9
s_4	-0.1	-0.1	-0.1	0.9	$1 + 0.9 = \mathbf{1.9}$

Greedy policy: $\pi_2(a_3 | s_1) = 1$, $\pi_2(a_3 | s_2) = 1$, $\pi_2(a_2 | s_3) = 1$, $\pi_2(a_5 | s_4) = 1$. Value update: $v_2 = (0.9, 1.9, 1.9, 1.9)$.

Result. π_2 is already optimal — two iterations sufficed for this tiny example [Book §4.1.2, p. 62]. The *values* keep improving for many more iterations (they converge to $(9, 10, 10, 10)$ in the limit), but the *policy* has already stopped changing. This gap between "the policy has converged" and "the values have converged" recurs throughout the book.

§4.2 Policy iteration

► Book: §4.2, pp. 62–69; Algorithm 4.2, p. 66

Policy iteration does not attack the BOE directly. It alternates:

- **Policy evaluation (PE):** given π_k , solve the *linear* Bellman equation $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$ [Book equation (4.3), p. 62].
- **Policy improvement (PI):** given v_{π_k} , set $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$.

Three questions must be answered [Book §4.2.1, p. 62]: how do we solve the evaluation step, why is π_{k+1} better than π_k , and why does the whole thing converge?

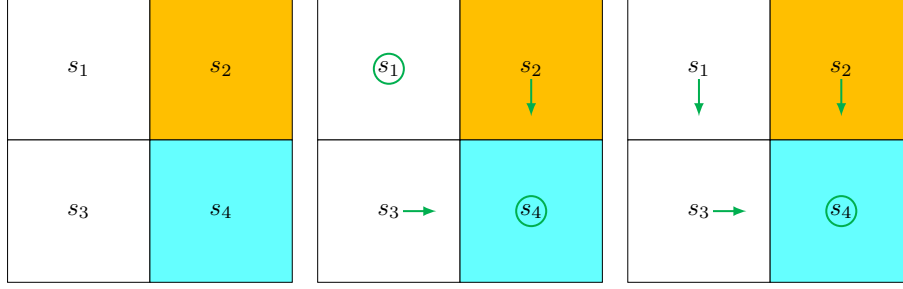


Figure 4.2. An example for demonstrating the implementation of the value iteration algorithm.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§4.2.1 Answer 1: how to evaluate

Chapter 2 gave two ways. The closed form $v_{\pi_k} = (I - \gamma P_{\pi_k})^{-1} r_{\pi_k}$ is impractical; instead use the inner iteration [Book equation (4.4), p. 63]

$$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \dots$$

which converges as $j \rightarrow \infty$ from any $v_{\pi_k}^{(0)}$.

Remark. So policy iteration is an iterative algorithm with a *second* iterative algorithm nested inside it. In theory the inner loop needs infinitely many steps; in practice you stop when $\|v^{(j+1)} - v^{(j)}\|$ is small or when j hits a cap. Does stopping early break things? No — and understanding why is exactly the point of truncated policy iteration in §4.3 [Book §4.2.1, p. 63].

§4.2.2 Answer 2: improvement really improves

Lemma 4.1 (Policy improvement — Lemma 4.1, p. 63). *If $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$, then $v_{\pi_{k+1}} \geq v_{\pi_k}$ elementwise.*

Proof (Box 4.1, pp. 63–64). Both are state values, so $v_{\pi_{k+1}} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_{k+1}}$ and $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$. By construction of π_{k+1} , $r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_k} \geq r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$. Therefore

$$v_{\pi_k} - v_{\pi_{k+1}} \leq (r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_k}) - (r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_{k+1}}) = \gamma P_{\pi_{k+1}} (v_{\pi_k} - v_{\pi_{k+1}}).$$

Iterating, $v_{\pi_k} - v_{\pi_{k+1}} \leq \gamma^n P_{\pi_{k+1}}^n (v_{\pi_k} - v_{\pi_{k+1}}) \rightarrow 0$. □

§4.2.3 Answer 3: convergence

Theorem 4.2 (Convergence of policy iteration — Theorem 4.1, p. 64). *The state-value sequence $\{v_{\pi_k}\}_{k=0}^{\infty}$ converges to v^* , and hence $\{\pi_k\}$ converges to an optimal policy.*

Proof idea (Box 4.2, p. 65). By Lemma 4.1 the sequence is nondecreasing, and it is bounded above by v^* :

$$v_{\pi_0} \leq v_{\pi_1} \leq v_{\pi_2} \leq \dots \leq v^*.$$

By the monotone convergence theorem it converges to some v_{∞} . To show $v_{\infty} = v^*$, run value iteration $v_{k+1} = f(v_k)$ alongside, started at $v_0 \leq v_{\pi_0}$, and prove by induction that $v_k \leq v_{\pi_k} \leq v^*$ for all k . Since $v_k \rightarrow v^*$, the sandwich forces $v_{\pi_k} \rightarrow v^*$. □

Intuition. The proof does more than establish convergence — it shows **policy iteration converges at least as fast as value iteration**, iteration for iteration, because it squeezes in extra evaluation sweeps. It is not free: each of those sweeps costs computation [Book §4.2.1, p. 64].

ALGORITHM — Policy iteration (Algorithm 4.2, p. 66)

Input: the model; an initial policy π_0 .

While v_{π_k} has not converged, **do**

- **Policy evaluation.** Initialise $v_{\pi_k}^{(0)}$ arbitrarily. While not converged, for every s :

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a | s) \left[\sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}^{(j)}(s') \right]$$

- **Policy improvement.** For every s and every a : $q_{\pi_k}(s, a) = \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}(s')$; then $a_k^*(s) = \arg \max_a q_{\pi_k}(s, a)$ and $\pi_{k+1}(a | s) = 1$ if $a = a_k^*(s)$, else 0.
-

§4.2.4 A two-state example done by hand

► Book: §4.2.3, pp. 67–68; Figure 4.3, p. 67; Tables 4.4–4.5, p. 68

Two states s_1, s_2 ; three actions $\mathcal{A} = \{a_\ell, a_0, a_r\}$ (left, stay, right); s_2 is the target; $r_{\text{boundary}} = -1$, $r_{\text{target}} = 1$, $\gamma = 0.9$. The initial policy π_0 moves *away* from the target at s_1 and drifts at s_2 .

Worked example: one full iteration of policy iteration [Book §4.2.3, pp. 67–68]

Policy evaluation. The Bellman equations of π_0 are

$$v_{\pi_0}(s_1) = -1 + \gamma v_{\pi_0}(s_1), \quad v_{\pi_0}(s_2) = 0 + \gamma v_{\pi_0}(s_1).$$

By hand: $v_{\pi_0}(s_1) = -1/(1 - 0.9) = -10$ and $v_{\pi_0}(s_2) = 0.9(-10) = -9$.

If instead you run the iterative solver from $v^{(0)} = (0, 0)$ you get

$$v^{(1)} = (-1, 0), \quad v^{(2)} = (-1.9, -0.9), \quad v^{(3)} = (-2.71, -1.71), \dots$$

crawling toward $(-10, -9)$ — exactly the geometric convergence promised in §2.7.2.

Policy improvement. The q -table (the book's Table 4.4, p. 68) is

$q_{\pi_k}(s, a)$	a_ℓ	a_0	a_r
s_1	$-1 + \gamma v_{\pi_k}(s_1)$	$0 + \gamma v_{\pi_k}(s_1)$	$1 + \gamma v_{\pi_k}(s_2)$
s_2	$0 + \gamma v_{\pi_k}(s_1)$	$1 + \gamma v_{\pi_k}(s_2)$	$-1 + \gamma v_{\pi_k}(s_2)$

Substituting $v_{\pi_0} = (-10, -9)$ gives Table 4.5, p. 68:

$q_{\pi_0}(s, a)$	a_ℓ	a_0	a_r
s_1	-10	-9	-7.1
s_2	-9	-7.1	-9.1

Greedy: $\pi_1(a_r | s_1) = 1$ and $\pi_1(a_0 | s_2) = 1$. **That is the optimal policy**, found in a single iteration.

Note that all the q -values are negative and yet the improvement step works perfectly. Only the *ordering* matters, never the sign — another face of Theorem 3.6.

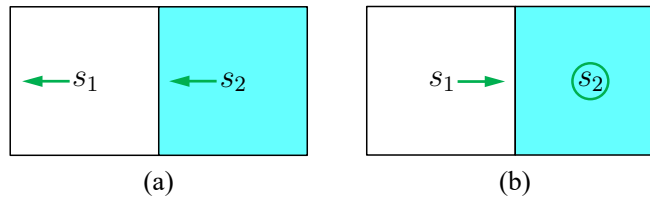


Figure 4.3. An example for illustrating the implementation of the policy iteration algorithm.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§4.2.5 A 5×5 example: how learning spreads

► Book: §4.2.3, pp. 68–69; Figure 4.4, p. 69

The book runs policy iteration on a 5×5 grid with $r_{\text{boundary}} = -1$, $r_{\text{forbidden}} = -10$, $r_{\text{target}} = 1$, $\gamma = 0.9$, starting from a random policy π_0 . It converges to the optimal policy π_{10} after ten iterations. Two patterns are worth memorising [Book §4.2.3, pp. 68–69]:

- **Optimality propagates outward from the target.** States adjacent to the target settle on their optimal actions first; distant states can only find good routes once the nearby states have found theirs. At π_1 only the cells around the target have nonzero values (10.0, 9.0); by π_5 the right-hand column has filled in (4.3, 4.8, 5.3, ...); only at π_{10} does the far-left column acquire its values (3.5, 3.1, 2.8, 2.5, 2.3).
- **Values decline with distance from the target**, for the discounting reason already met in §3.5.

§4.3 Truncated policy iteration

► Book: §4.3, pp. 70–74; Table 4.6, p. 71; Algorithm 4.3, p. 72

§4.3.1 The two algorithms are the same algorithm

► Book: §4.3.1, pp. 70–72

Lay them side by side [Book §4.3.1, p. 70]:

$$\begin{aligned} \text{Policy iteration: } & \pi_0 \xrightarrow{\text{PE}} v_{\pi_0} \xrightarrow{\text{PI}} \pi_1 \xrightarrow{\text{PE}} v_{\pi_1} \xrightarrow{\text{PI}} \pi_2 \rightarrow \dots \\ \text{Value iteration: } & v_0 \xrightarrow{\text{PU}} \pi'_1 \xrightarrow{\text{VU}} v_1 \xrightarrow{\text{PU}} \pi'_2 \xrightarrow{\text{VU}} v_2 \rightarrow \dots \end{aligned}$$

Start both from the same place, $v_0 = v_{\pi_0}$. Then steps 1–3 produce identical results. They diverge at step 4 [Book Table 4.6, p. 71]: value iteration computes the single line $v_1 = r_{\pi_1} + \gamma P_{\pi_1} v_0$, whereas

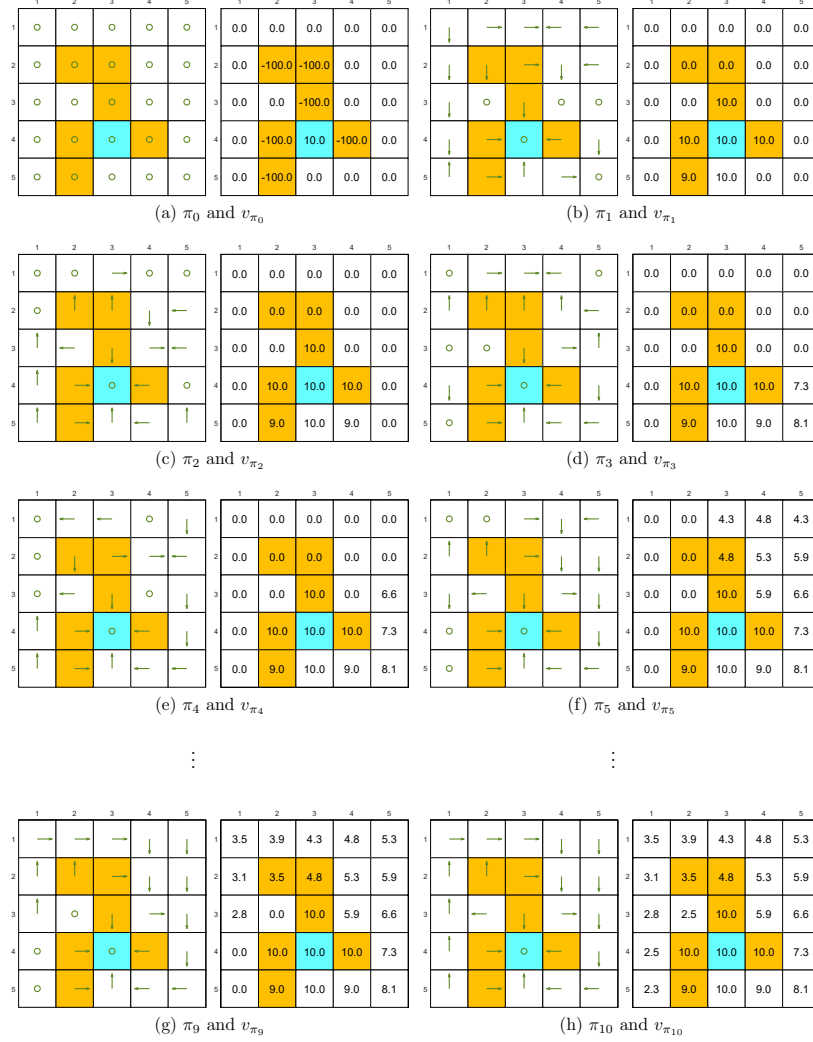


Figure 4.4. The evolution processes of the policies generated by the policy iteration algorithm.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

policy iteration solves $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$ — an infinite loop. Write that loop out with $v_{\pi_1}^{(0)} = v_0$:

$$\begin{aligned}
 v_{\pi_1}^{(0)} &= v_0 \\
 v_{\pi_1}^{(1)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} && \leftarrow \text{value iteration stops here} \\
 v_{\pi_1}^{(2)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \\
 &\vdots \\
 v_{\pi_1}^{(j_{\text{truncate}})} &= \dots && \leftarrow \text{truncated policy iteration stops here} \\
 &\vdots \\
 v_{\pi_1}^{(\infty)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)} && \leftarrow \text{policy iteration stops here}
 \end{aligned}$$

Remark. So there is one algorithm with one dial, j_{truncate} :

$$j_{\text{truncate}} = 1 \Rightarrow \text{value iteration}; \quad j_{\text{truncate}} = \infty \Rightarrow \text{policy iteration}; \quad 1 < j_{\text{truncate}} < \infty \Rightarrow \text{truncated policy}$$

This is one of the tidiest unifications in the book [Book §4.3.1, pp. 71–72].

ALGORITHM — Truncated policy iteration (Algorithm 4.3, p. 72)

Input: the model; an initial policy π_0 ; a cap j_{truncate} .
While v_k has not converged, **do**

- **Policy evaluation (truncated).** Set $v_k^{(0)} = v_{k-1}$. For $j = 0, \dots, j_{\text{truncate}} - 1$ and every s :

$$v_k^{(j+1)}(s) = \sum_a \pi_k(a | s) \left[\sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_k^{(j)}(s') \right].$$

Set $v_k = v_k^{(j_{\text{truncate}})}$.

- **Policy improvement.** For every s , compute $q_k(s, a) = \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_k(s')$ and set $\pi_{k+1}(a | s) = 1$ if $a = \arg \max_{a'} q_k(s, a')$, else 0.
-

§4.3.2 Does truncation break convergence?

► Book: §4.3.2, pp. 72–74; Proposition 4.1, p. 72; Figure 4.5, p. 73

No. The intermediate v_k are only *approximations* of v_{π_k} , but the algorithm still finds optimal policies, and it sits neatly between the two extremes — faster than value iteration (more evaluation per improvement), slower than full policy iteration (fewer evaluations) [Book §4.3.2, p. 72; Figure 4.5, p. 73].

Proposition 4.3 (Value improvement — Proposition 4.1, p. 72). *In the inner loop $v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}$, if the initial guess is $v_{\pi_k}^{(0)} = v_{\pi_{k-1}}$, then $v_{\pi_k}^{(j+1)} \geq v_{\pi_k}^{(j)}$ for all j : the inner iterates climb monotonically.*

Proof (Box 4.3, p. 73). Subtracting consecutive inner iterations gives $v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)} = \gamma^j P_{\pi_k}^j (v_{\pi_k}^{(1)} - v_{\pi_k}^{(0)})$, the book's equation (4.5). And with $v_{\pi_k}^{(0)} = v_{\pi_{k-1}}$,

$$v_{\pi_k}^{(1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_{k-1}} \geq r_{\pi_{k-1}} + \gamma P_{\pi_{k-1}} v_{\pi_{k-1}} = v_{\pi_{k-1}} = v_{\pi_k}^{(0)},$$

using $\pi_k = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_{k-1}})$. Since $P_{\pi_k}^j \geq 0$ the difference stays nonnegative for every j . ◻

Remark (Practical guidance). Run a few evaluation sweeps, not one and not a thousand [Book §4.5 Q&A, p. 75]. Going from $j_{\text{truncate}} = 1$ to $j = 3$ or 5 buys a noticeable speed-up; going from 5 to 500 buys almost nothing.

§4.4 Generalised policy iteration

► Book: §4.4, p. 74

All three algorithms share a shape: alternate a *value* update with a *policy* update, each making the other better. The book calls this **generalised policy iteration (GPI)**, and notes that most RL algorithms in the book fall under it [Book §4.4, p. 74 and §4.5 Q&A, p. 75]. When you meet Sarsa, Q-learning, DQN and actor-critic later, ask each time: which part is the value update, and which part is the policy update? You will always find both.

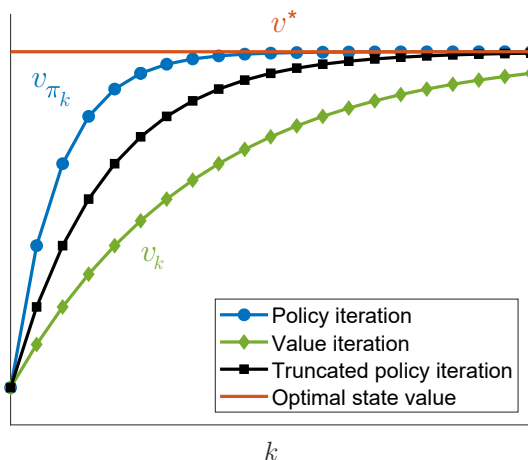


Figure 4.5. An illustration of the relationships between the value iteration, policy iteration, and truncated policy iteration algorithms.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Remark (“Model-based” does not mean what you think). The book is careful here [Book §4.5 Q&A, pp. 75–76]. The algorithms of this chapter require the model and are called *dynamic programming*, not reinforcement learning. The RL terminology is different: **model-based RL** means the agent *estimates* a model from data and then plans with it; **model-free RL** means no model is ever estimated. Chapters 5–10 are all model-free.

§4.5 Chapter 4 self-check

1. In value iteration, are the intermediate v_k state values? In policy iteration, are they? Justify both answers. (*No / yes*; [Book §4.5, p. 74].)
2. Redo the $k = 1$ value-iteration table with $\gamma = 0.5$ instead of 0.9. Does the greedy policy change?
3. Explain in one paragraph why policy iteration converges at least as fast as value iteration, and why that does not make it strictly better.
4. Set $j_{\text{truncate}} = 1$ in Algorithm 4.3 and show algebraically that you recover Algorithm 4.1.
5. In the 5×5 run, why do the far cells acquire correct values last?

§5 Monte Carlo Methods

READING MAP — Chapter 5 of the textbook

Textbook: Chapter 5, pp. 77–99. Sections: §5.1 motivating example — mean estimation (p. 78; law of large numbers, Box 5.1, p. 80), §5.2 MC Basic (p. 80; converting policy iteration to model-free p. 80, the algorithm p. 81, examples p. 83), §5.3 MC Exploring Starts (p. 86; using samples efficiently p. 86, updating policies efficiently p. 87, algorithm p. 88), §5.4 MC ϵ -Greedy (p. 89; ϵ -greedy policies p. 89, algorithm p. 90, examples p. 91), §5.5 exploration versus exploitation (p. 92), §5.6 summary (p. 97), §5.7 Q&A (p. 97). Algorithms 5.1 (p. 82), 5.2 (p. 88), 5.3 (p. 91).

Videos: ► L5 P1 — Motivating examples; ► L5 P2 — MC Basic intro; ► L5 P3 — MC Basic examples; ► L5 P4 — MC Exploring Starts; ► L5 P5 — MC ϵ -greedy intro; ► L5 P6 — MC ϵ -greedy examples.

The turning point of the book. These are the first *model-free* algorithms. The author’s one-line philosophy [Book §5, p. 77]: “If we do not have a model, we must have data. If we do not have data, we must have a model. If we have neither, we cannot find optimal policies.”

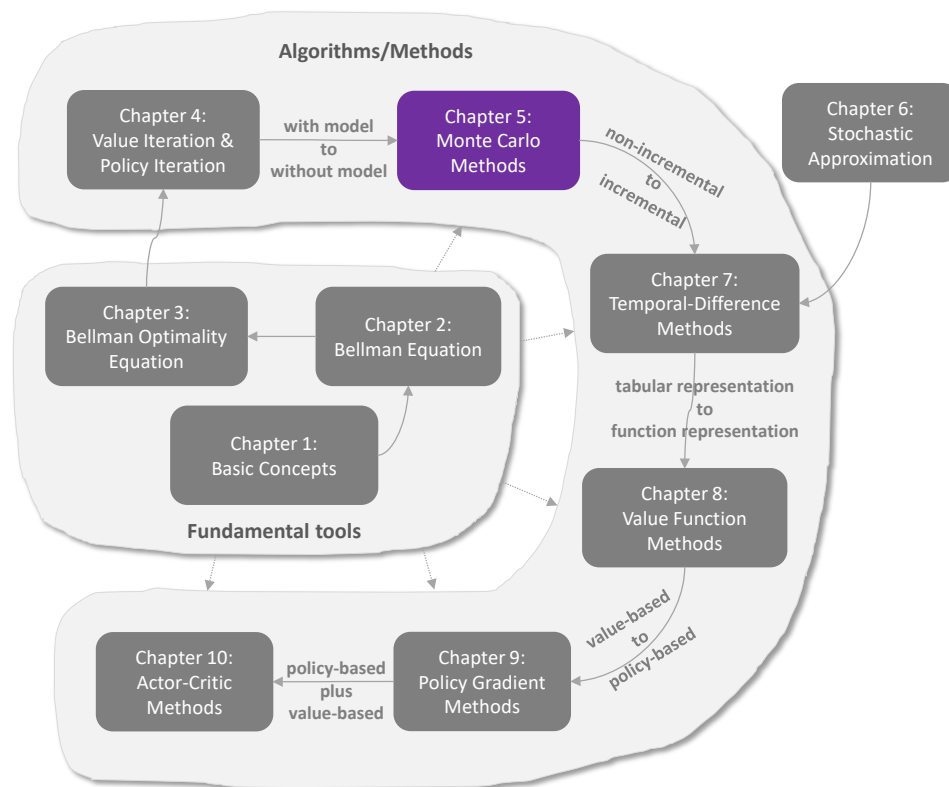


Figure 5.1. Where we are in this book.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§5.1 Mean estimation: the whole idea in miniature

► Book: §5.1, pp. 78–80

Everything in this chapter rests on one observation: *state and action values are means*, so estimating them is a mean-estimation problem.

Let X be a random variable taking values in a finite set $\mathcal{X}$. There are two ways to get $\mathbb{E}[X]$ [Book §5.1, pp. 78–79]:

- **Model-based:** if you know the distribution, $\mathbb{E}[X] = \sum_{x \in \mathcal{X}} p(x) x$.
- **Model-free:** if you do not, draw samples $\{x_1, \dots, x_n\}$ and use the sample average $\bar{x} = \frac{1}{n} \sum_{j=1}^n x_j$.

Worked example: coin flipping [Book §5.1, p. 79; Figure 5.2, p. 79]

Let $X = +1$ if heads, $X = -1$ if tails, with $p(X = 1) = p(X = -1) = 0.5$.

Model-based: $\mathbb{E}[X] = 0.5(1) + 0.5(-1) = 0$, computed instantly.

Model-free: flip the coin and average. With $n = 10$ the average bounces around; with $n = 200$ (the book's Figure 5.2, p. 79) it has visibly settled near zero. Nobody told the estimator that $p = 0.5$; the data did.

Fact (Law of large numbers — Box 5.1, p. 80). *If $\{x_i\}_{i=1}^n$ are i.i.d. samples of X and $\bar{x} = \frac{1}{n} \sum_i x_i$, then*

$$\mathbb{E}[\bar{x}] = \mathbb{E}[X], \quad \text{var}[\bar{x}] = \frac{\text{var}[X]}{n}.$$

So $\bar{x}$ is **unbiased**, and its variance vanishes as $n \rightarrow \infty$; hence $\bar{x} \rightarrow \mathbb{E}[X]$.

Proof (Box 5.1, p. 80). $\mathbb{E}[\bar{x}] = \frac{1}{n} \sum_i \mathbb{E}[x_i] = \mathbb{E}[X]$ (identical distribution). And $\text{var}(\bar{x}) = \frac{1}{n^2} \sum_i \text{var}[x_i] = \frac{n \text{var}[X]}{n^2} = \frac{\text{var}[X]}{n}$, where splitting the variance of the sum requires *independence*. $\square$

Common pitfall (i.i.d. is not decoration). [Book §5.1, p. 79] The book gives the sharp counterexample: if every sample is forced to equal the first one, the average always equals the first sample no matter how many you take. Correlated samples can make the estimate useless. Keep this in mind at §5.3.1, where the every-visit strategy deliberately introduces some correlation.

§5.2 Making policy iteration model-free

► **Book:** §5.2.1, pp. 80–81; equations (5.1)–(5.2)

Look at where policy iteration uses the model. In the improvement step,

$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a | s) q_{\pi_k}(s, a),$$

so **action values are the only thing that matters**. Policy evaluation exists solely to produce them. There are two routes to q_{π_k} [Book §5.2.1, p. 81]:

- **Model-based** (Chapter 4): solve for v_{π_k} , then

$$q_{\pi_k}(s, a) = \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}(s') \quad (5.1)$$

— which needs both halves of the model.

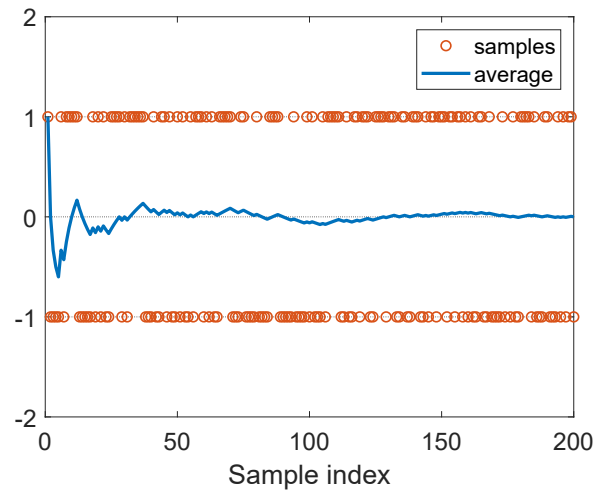


Figure 5.2. An example for demonstrating the law of large numbers. Here, the samples are drawn from $\{+1, -1\}$ following a uniform distribution. The average of the samples gradually converges to zero, which is the true expected value, as the number of samples increases.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

- **Model-free:** recall $q_{\pi_k}(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]$. It is a mean. So sample it: run n episodes starting from (s, a) under π_k , record their returns $g_{\pi_k}^{(i)}(s, a)$, and set

$$q_{\pi_k}(s, a) \approx \frac{1}{n} \sum_{i=1}^n g_{\pi_k}^{(i)}(s, a). \quad (5.2)$$

Common pitfall (Why estimate q and not v ?). [Book §5.2.1, p. 82; §5.7 Q&A] Because converting v into q requires equation (5.1), which requires the model. If you are model-free you must estimate action values *directly*. This is the reason q takes over from v as the central object for the rest of the book.

§5.3 MC Basic

► Book: §5.2.2, pp. 81–82; Algorithm 5.1, p. 82

ALGORITHM — MC Basic (Algorithm 5.1, p. 82)

Input: an initial policy π_0 . **No model required.**

For $k = 0, 1, 2, \dots$:

- For every $s \in \mathcal{S}$ and every $a \in \mathcal{A}(s)$:
 - Collect sufficiently many episodes starting from (s, a) , following π_k .
 - **Policy evaluation:** $q_k(s, a) \leftarrow$ the average return of those episodes.
 - **Policy improvement:** $a_k^*(s) = \arg \max_a q_k(s, a)$; $\pi_{k+1}(a \mid s) = 1$ if $a = a_k^*(s)$, else 0.
-

Remark. Compare Algorithm 5.1 with Algorithm 4.2 (policy iteration, p. 66). **The only difference is one line:** where policy iteration solves the Bellman equation, MC Basic averages sampled returns. Everything else — the greedy improvement, the convergence argument — is inherited [Book §5.2.2, p. 82].

§5.3.1 A hand computation

► Book: §5.2.3, pp. 83–84; Figure 5.3, p. 83

The setting is a 3×3 grid with $r_{\text{boundary}} = r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$, $\gamma = 0.9$. The initial policy π_0 is already optimal everywhere except s_1 and s_3 .

Worked example: estimating all five action values at s_1 [Book §5.2.3, pp. 83–84]

Because this example is deterministic in both policy and model, running an episode twice gives the same trajectory — so *one* episode per (s, a) suffices.

(s_1, a_1) — **up, into the wall.** Episode: $s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} s_1 \rightarrow \dots$, rewards $-1, -1, -1, \dots$

$$q_{\pi_0}(s_1, a_1) = -1 + \gamma(-1) + \gamma^2(-1) + \dots = \frac{-1}{1-\gamma} = -10.$$

(s_1, a_2) — **right.** Episode $s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_3} \dots$, rewards $0, 0, 0, 1, 1, \dots$

$$q_{\pi_0}(s_1, a_2) = 0 + \gamma 0 + \gamma^2 0 + \gamma^3(1) + \gamma^4(1) + \dots = \frac{\gamma^3}{1-\gamma} = \frac{0.729}{0.1} = 7.29.$$

(s_1, a_3) — **down.** Episode $s_1 \xrightarrow{a_3} s_4 \xrightarrow{a_2} s_5 \xrightarrow{a_3} \dots$, same reward pattern:

$$q_{\pi_0}(s_1, a_3) = \frac{\gamma^3}{1-\gamma} = 7.29.$$

(s_1, a_4) — **left, into the wall.** As with a_1 : $q_{\pi_0}(s_1, a_4) = \frac{-1}{1-\gamma} = -10$.

(s_1, a_5) — **stay.** Reward 0 now, then the policy at s_1 bumps the wall forever:

$$q_{\pi_0}(s_1, a_5) = 0 + \gamma(-1) + \gamma^2(-1) + \dots = \frac{-\gamma}{1-\gamma} = -9.$$

Improvement. The maximum is 7.29, achieved by both a_2 and a_3 , so $\pi_1(a_2 | s_1) = 1$ or $\pi_1(a_3 | s_1) = 1$ — either is optimal. One iteration was enough here because the policy was already good elsewhere [Book §5.2.3, p. 84].

§5.3.2 Episode length and the sparse-reward problem

► Book: §5.2.3, pp. 84–86; Figure 5.4, p. 85

On a 5×5 grid ($r_{\text{boundary}} = -1$, $r_{\text{forbidden}} = -10$, $r_{\text{target}} = 1$, $\gamma = 0.9$), the book varies the episode length and shows the result:

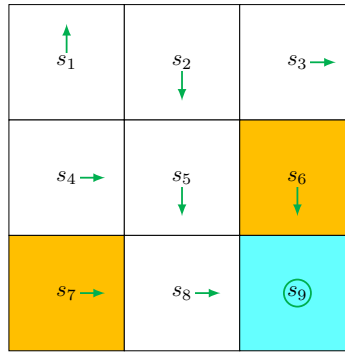


Figure 5.3. An example for illustrating the MC Basic algorithm.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Episode length	Outcome [Book Figure 5.4, p. 85]
1	Only cells adjacent to the target have nonzero value (1.0); everything else is 0.
2	Values 1.9 near the target; the frontier of nonzero values expands by one ring.
3, 4	2.7, then 3.4 near the target; still nothing in the far corners.
14	Values everywhere but wrong: row 1 reads 1.2, 1.6, 2.0, 2.5, 3.0.
15	The first length at which the bottom-left state can reach the target at all.
30	The <i>policy</i> is already optimal, though values are still short (9.6 at the target instead of 10.0).
100	Both policy and values essentially exact: 3.5, 3.9, 4.3, 4.8, 5.3 in row 1, 10.0 at the target.

Intuition (The sparse-reward problem). [Book §5.2.3, p. 86] If a state needs at least 15 steps to reach the only source of positive reward, then any episode shorter than 15 steps returns exactly zero from that state — the algorithm receives *no signal at all*. This is the **sparse reward** problem, and it is the main practical obstacle in large state spaces. The standard remedy is reward shaping: hand out small positive rewards near the target so that an “attractive field” guides the agent in [Book §5.2.3, p. 86].

Notice, though, the tension with Theorem 3.6: an *affine* change to rewards changes nothing. Shaping must be *state-dependent* to have any effect.

Common pitfall (Why MC Basic is unusable in practice). [Book §5.2.2, p. 82] It needs a fresh batch of episodes for *every* state–action pair in *every* iteration. For a 5×5 grid with five actions that is 125 batches per iteration. The next two sections are entirely about fixing this.

§5.4 MC Exploring Starts

► Book: §5.3, pp. 86–89; Algorithm 5.2, p. 88

Two independent efficiency ideas.

§5.4.1 Idea 1: reuse every visit in an episode

► Book: §5.3.1, pp. 86–87; equation (5.3)

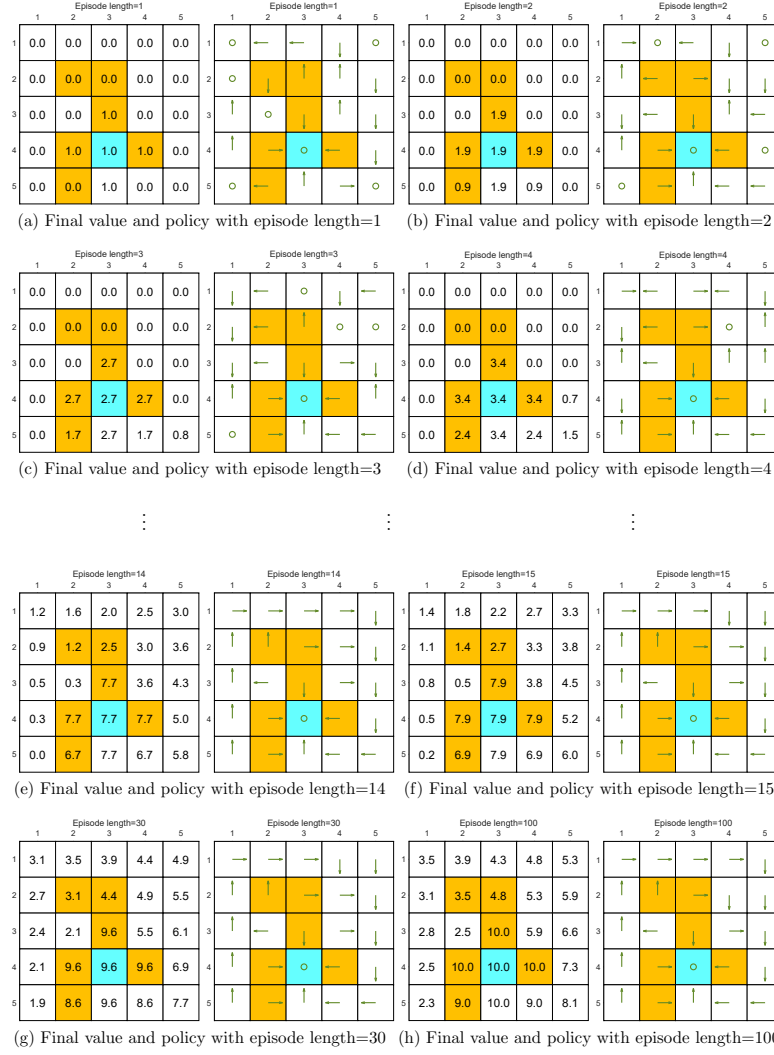


Figure 5.4. The policies and state values obtained by the MC Basic algorithm when given different episode lengths. Only if the length of each episode is sufficiently long, can the state values be accurately estimated.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Suppose an episode under π is

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \quad (5.3)$$

MC Basic uses this episode to estimate only $q(s_1, a_2)$ — the **initial-visit** strategy. But the tail of the episode after any visit is itself a valid episode:

$$\begin{aligned} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots & \text{ [original: estimates } q(s_1, a_2)] \\ s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots & \text{ [estimates } q(s_2, a_4)] \\ s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots & \text{ [estimates } q(s_1, a_2) \text{ again]} \\ s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots & \text{ [estimates } q(s_2, a_3)] \\ s_5 \xrightarrow{a_1} \dots & \text{ [estimates } q(s_5, a_1)] \end{aligned}$$

Three strategies [Book §5.3.1, p. 87; §5.7 Q&A, p. 98]:

- **initial-visit**: use the episode only for its starting pair (MC Basic);
- **first-visit**: use each pair the first time it appears;
- **every-visit**: use each pair every time it appears.

Every-visit squeezes the most out of the data. The price: the sub-episode starting at the second visit to (s_1, a_2) is a *subset* of the one starting at the first, so the returns are correlated — the i.i.d. assumption is bent. The book’s verdict: the correlation is weak if the two visits are far apart in the trajectory [Book §5.3.1, p. 87].

§5.4.2 Idea 2: update the policy after every episode

► Book: §5.3.2, pp. 87–88

MC Basic waits until all episodes for a pair have been collected before touching the policy. Instead, use the return from a *single* episode as a rough estimate and improve immediately. The estimate is noisy, but this is just generalised policy iteration: you may update the policy before the value estimate has converged [Book §5.3.2, p. 88; §4.4, p. 74].

ALGORITHM — MC Exploring Starts (Algorithm 5.2, p. 88)

Initialise: $\pi_0(a \mid s)$ and $q(s, a)$ for all (s, a) ; $\text{Returns}(s, a) = 0$ and $\text{Num}(s, a) = 0$.
For each episode:

- **Episode generation.** Choose a starting pair (s_0, a_0) , ensuring every pair can be chosen (*this is the exploring-starts condition*). Follow π to generate $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$.
 - Set $g \leftarrow 0$. For $t = T - 1, T - 2, \dots, 0$:
 - $g \leftarrow \gamma g + r_{t+1}$
 - $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$; $\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$
 - **Evaluation:** $q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$
 - **Improvement:** $\pi(a \mid s_t) = 1$ if $a = \arg \max_{a'} q(s_t, a')$, else 0
-

Remark (Why the loop runs backwards). [Book §5.3.3, p. 88] The recursion $g \leftarrow \gamma g + r_{t+1}$ computes every suffix return in a single backward pass, in $O(T)$ total time instead of $O(T^2)$. It is exactly the bootstrapping identity $G_t = R_{t+1} + \gamma G_{t+1}$ used as a computational device.

Common pitfall (The exploring-starts condition). [Book §5.3.3, p. 89; §5.7 Q&A, p. 98] The algorithm requires enough episodes starting from *every* state–action pair. In theory this is necessary: if some action is never tried, its value is never estimated, and the greedy policy may never select it even though it is the best. In practice it is often impossible — you cannot reset a physical robot into an arbitrary state and force an arbitrary first action. The next section removes the condition.

§5.5 MC ϵ -Greedy

► Book: §5.4, pp. 89–92; Algorithm 5.3, p. 91

§5.5.1 ε -greedy policies

► Book: §5.4.1, pp. 89–90

A policy is **soft** if every action has positive probability at every state. A single sufficiently long episode generated by a soft policy will visit every state–action pair many times — which is precisely what exploring starts was trying to buy.

Definition (ε -greedy policy). For $\varepsilon \in [0, 1]$,

$$\pi(a \mid s) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1), & \text{for the greedy action,} \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & \text{for each of the other } |\mathcal{A}(s)| - 1 \text{ actions.} \end{cases}$$

Worked example: concrete probabilities with $|\mathcal{A}| = 5$

$$\begin{array}{lll} \varepsilon = 0 : & \text{greedy action } 1.00, & \text{others } 0.00 \quad (\text{pure greedy}) \\ \varepsilon = 0.1 : & 1 - \frac{0.1}{5}(4) = 0.92, & \frac{0.1}{5} = 0.02 \\ \varepsilon = 0.5 : & 1 - \frac{0.5}{5}(4) = 0.60, & \frac{0.5}{5} = 0.10 \\ \varepsilon = 1 : & 0.20, & 0.20 \quad (\text{uniform}) \end{array}$$

Check: $0.6 + 4(0.1) = 1$. ✓

The greedy action always keeps the largest probability, since

$$1 - \frac{\varepsilon}{|\mathcal{A}|} (|\mathcal{A}| - 1) = 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}|} \geq \frac{\varepsilon}{|\mathcal{A}|} \quad \text{for all } \varepsilon \in [0, 1]$$

[Book §5.4.1, p. 90]. Implementation: draw $x \sim U[0, 1]$; if $x \geq \varepsilon$ take the greedy action, otherwise pick uniformly at random from $\mathcal{A}(s)$ — which may re-select the greedy action, which is why the total greedy probability is $1 - \varepsilon + \varepsilon/|\mathcal{A}|$ [Book §5.4.1, p. 90].

§5.5.2 The algorithm

► Book: §5.4.2, pp. 90–91; equations (5.4)–(5.5)

The only change is in the improvement step. Instead of

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi} \sum_a \pi(a \mid s) q_{\pi_k}(s, a) \quad (5.4)$$

over *all* policies Π , we optimise over Π_ε , the set of ε -greedy policies with that fixed ε :

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi_\varepsilon} \sum_a \pi(a \mid s) q_{\pi_k}(s, a), \quad (5.5)$$

whose solution is the ε -greedy policy centred on $a_k^* = \arg \max_a q_{\pi_k}(s, a)$.

ALGORITHM — MC ε -Greedy (Algorithm 5.3, p. 91)

Initialise: π_0 , $q(s, a)$, $\text{Returns}(s, a) = 0$, $\text{Num}(s, a) = 0$, and $\varepsilon \in (0, 1]$.
For each episode:

- Choose *any* starting pair (s_0, a_0) — **exploring starts is no longer required** — and follow π for T steps.
- $g \leftarrow 0$. For $t = T - 1, \dots, 0$:
 - $g \leftarrow \gamma g + r_{t+1}$; $\text{Returns}(s_t, a_t) += g$; $\text{Num}(s_t, a_t) += 1$
 - $q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$
 - With $a^* = \arg \max_a q(s_t, a)$, set $\pi(a | s_t) = 1 - \frac{\varepsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1)$ if $a = a^*$, and $\frac{\varepsilon}{|\mathcal{A}(s_t)|}$ otherwise.

Common pitfall (In what sense is the result optimal?). [Book §5.4.2, p. 91; §5.7 Q&A, p. 98] Yes and no. *Yes*: with enough samples the algorithm converges to the best policy **within** Π_ε . *No*: that is generally not the best policy within Π . If ε is small the two are close; if ε is large they are not, as the numbers below show.

§5.5.3 Learning from a single very long episode

► Book: §5.4.3, pp. 91–92; Figure 5.5, p. 92

The book runs MC ε -Greedy with $\varepsilon = 0.5$ using *one* episode of one million steps per iteration, starting from the uniform policy (0.2 on every action), with $r_{\text{boundary}} = r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$, $\gamma = 0.9$. The optimal ε -greedy policy is reached after **two iterations** [Book §5.4.3, p. 91]. One episode suffices because the soft policy visits every state–action pair many times.

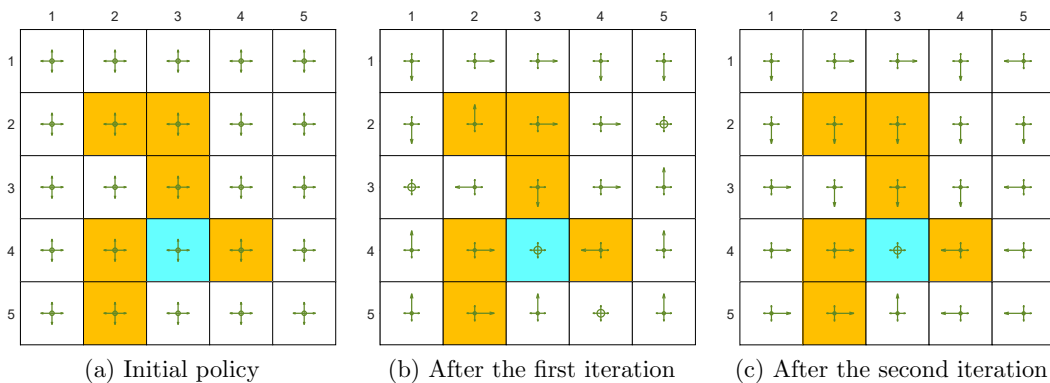


Figure 5.5. The evolution process of the MC ε -Greedy algorithm based on single episodes.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§5.6 Exploration versus exploitation, quantified

► Book: §5.5, pp. 92–97; Figures 5.6–5.8, pp. 93–96

Definition. *Exploration* means trying as many actions as possible so that every action value is well estimated. *Exploitation* means taking the action that currently looks best. They pull in opposite directions, because the current estimates may be wrong precisely where you have not explored.

The book measures the trade-off on a 5×5 grid ($r_{\text{boundary}} = -1$, $r_{\text{forbidden}} = -10$, $r_{\text{target}} = 1$, $\gamma = 0.9$).

Worked example: cost of exploration: state values collapse as ε grows [Book Figure 5.6, p. 93]

Take a fixed set of “consistent” ε -greedy policies (same action has the largest probability in each) and evaluate them:

$\varepsilon = 0$: row 1 = 3.5, 3.9, 4.3, 4.8, 5.3; target cell = 10.0
 $\varepsilon = 0.1$: row 1 = 0.4, 0.5, 0.9, 1.3, 1.4; target cell = 3.4
 $\varepsilon = 0.2$: row 1 = -2.2, -2.4, -2.1, -1.7, -1.8
 $\varepsilon = 0.5$: row 1 = -8.0, -9.0, -8.4, -7.2, -7.8; worst cell = -17.0

The values do not merely decline — they go sharply negative. The book explains why the *target* cell suffers most [Book §5.5, p. 95]: with ε large, an agent sitting on the target keeps randomly wandering into the surrounding forbidden cells and eating -10.

Worked example: when does exploration change the answer? [Book Figure 5.7, p. 94]

Now compute the policy that is *optimal within* Π_ε :

- $\varepsilon = 0$: the greedy optimal policy; values 3.5, ..., 10.0.
- $\varepsilon = 0.1$: the optimal ε -greedy policy is *consistent* with the greedy optimal one — same arrows, lower values (0.4, 0.5, 0.9, 1.3, 1.4 in row 1).
- $\varepsilon = 0.2$: consistency **breaks**. The optimal ε -greedy policy now points differently from the greedy optimum.
- $\varepsilon = 0.5$: badly inconsistent; values around -4 to -10.

Rule of thumb: keep ε small enough that the optimal ε -greedy policy still agrees with the greedy optimum [Book §5.5, p. 95].

Worked example: benefit of exploration: coverage [Book Figure 5.8, p. 96]

Count how often each of the 125 state-action pairs is visited in a single episode of one million steps:

- $\varepsilon = 1$ (uniform): every pair is visited roughly 7600–8300 times — almost perfectly even coverage.
- $\varepsilon = 0.5$: wildly uneven. Some pairs are visited more than 250,000 times while most receive only hundreds or even tens of visits.

So the exploration benefit is real and it degrades fast as ε falls.

Intuition (The standard resolution). [Book §5.5, p. 97] Start with ε large to explore, then *decay* it toward zero so that the final policy is (nearly) greedy and therefore (nearly) optimal. You get coverage early and optimality late. Every practical implementation — including DQN in Chapter 8 — does some version of this.

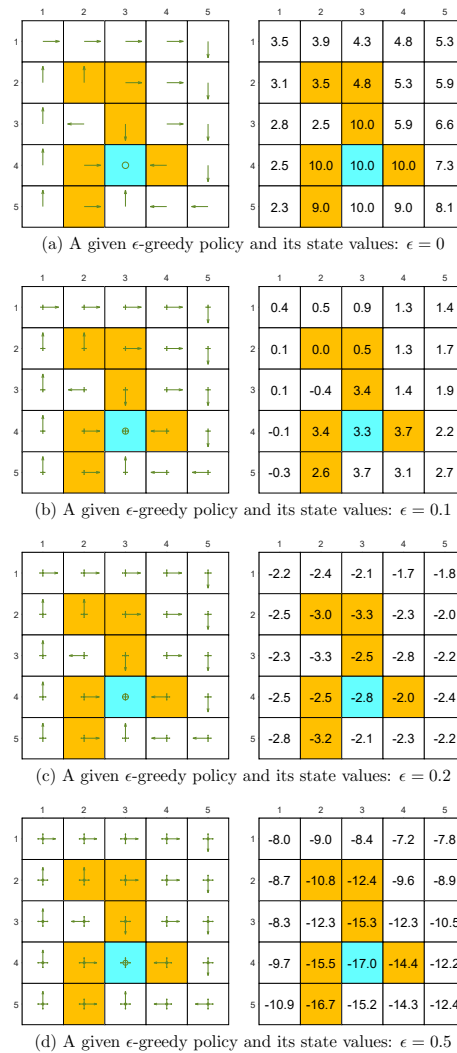


Figure 5.6. The state values of some ϵ -greedy policies. These ϵ -greedy policies are consistent with each other in the sense that the actions with the greatest probabilities are the same. It can be seen that, when the value of ϵ increases, the state values of the ϵ -greedy policies decrease and hence their optimality becomes worse.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§5.7 Summary and self-check

► Book: §5.6, p. 97

Algorithm	What it changes
MC Basic	Policy iteration with sampled returns replacing the model in the evaluation step. Correct but hopelessly sample-inefficient.
MC Exploring Starts	Adds first-visit / every-visit sample reuse and episode-by-episode policy updates. Still needs exploring starts.
MC ϵ -Greedy	Optimises over Π_ϵ instead of Π in the improvement step. Soft policies supply exploration, so exploring starts is dropped; the price is optimality within Π_ϵ only.

1. Why is a model-free algorithm forced to estimate q_π rather than v_π ? (*Equation (5.1) needs the*

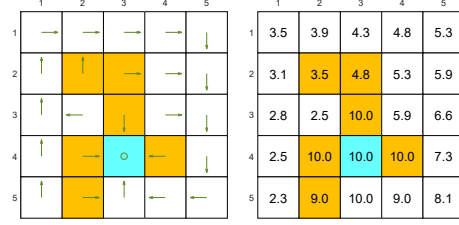
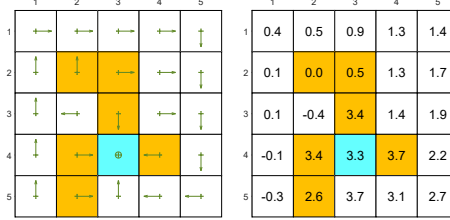
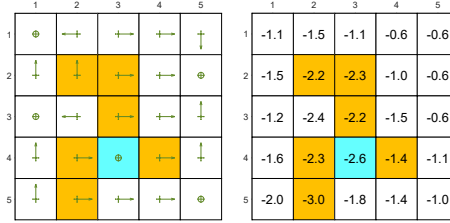
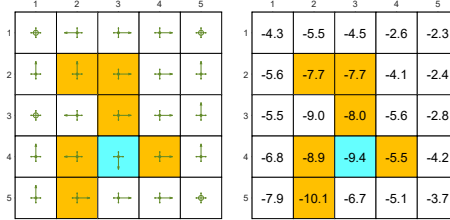
(a) The optimal ϵ -greedy policy and its state values: $\epsilon = 0$ (b) The optimal ϵ -greedy policy and its state values: $\epsilon = 0.1$ (c) The optimal ϵ -greedy policy and its state values: $\epsilon = 0.2$ (d) The optimal ϵ -greedy policy and its state values: $\epsilon = 0.5$

Figure 5.7. The optimal ϵ -greedy policies and their corresponding state values under different values of ϵ . Here, these ϵ -greedy policies are optimal among all ϵ -greedy ones (with the same value of ϵ). It can be seen that, when the value of ϵ increases, the optimal ϵ -greedy policies are no longer consistent with the optimal one as in (a).

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

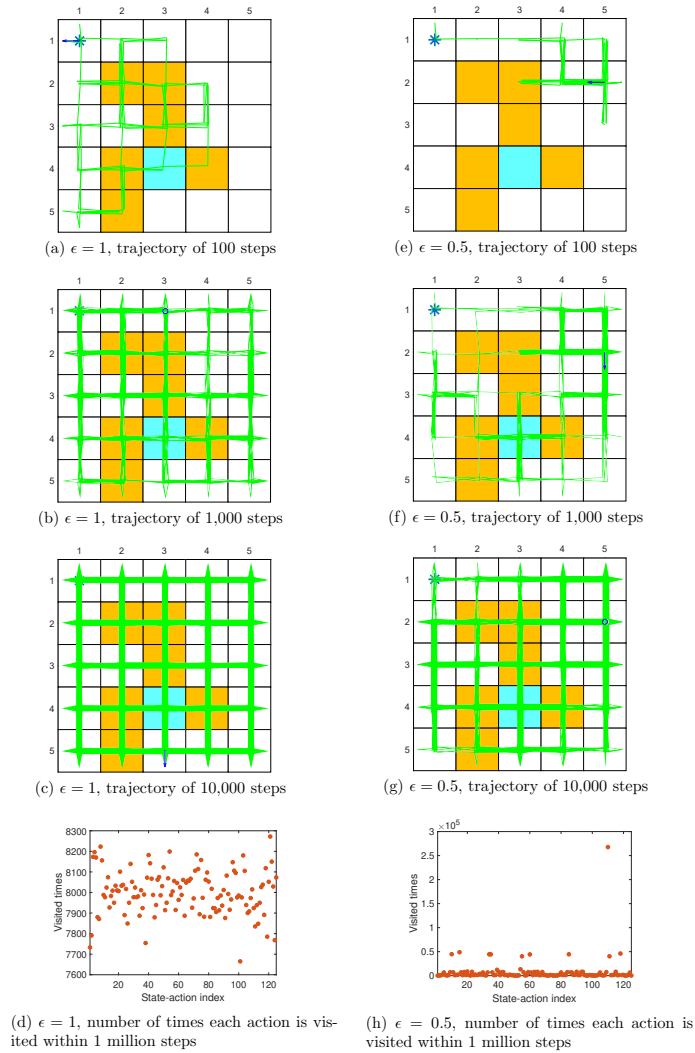


Figure 5.8. Exploration abilities of ϵ -greedy policies with different values of ϵ .

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

model.)

2. Recompute $q_{\pi_0}(s_1, a_5) = -\gamma/(1 - \gamma)$ from scratch with $\gamma = 0.5$ and explain the sign.
3. Distinguish initial-visit, first-visit and every-visit, and say which one risks violating the i.i.d. assumption.
4. Give one theoretical and one practical reason the exploring-starts condition is unattractive.
5. With $|\mathcal{A}| = 5$ and $\epsilon = 0.4$, write out all five probabilities and check that they sum to one.
6. Explain why increasing ϵ lowers the value of the *target* state more than that of a far-away state. ([Book §5.5, p. 95].)

§6 Stochastic Approximation

READING MAP — Chapter 6 of the textbook

Textbook: Chapter 6, pp. 101–124. Sections: §6.1 motivating example — mean estimation (p. 102), §6.2 Robbins–Monro algorithm (p. 103; convergence properties p. 105, application to mean estimation p. 108), §6.3 Dvoretzky’s convergence theorem (p. 109; proof p. 110, applications pp. 112–113, extension p. 113), §6.4 stochastic gradient descent (p. 114; mean estimation p. 116, convergence pattern p. 116, deterministic formulation p. 118, BGD/SGD/MBGD p. 119, convergence of SGD p. 121), §6.5 summary (p. 123), §6.6 Q&A (p. 123).

Videos: ► L6 P1 — Motivating example; ► L6 P2 — RM: introduction; ► L6 P3 — RM: convergence; ► L6 P4 — SGD: introduction; ► L6 P5 — SGD: examples; ► L6 P6 — SGD: properties; ► L6 P7 — SGD: comparison.

Why this chapter exists. It introduces no RL algorithm at all. It exists to close a gap: everything so far has been *non-incremental* (collect all the data, then compute), and everything from Chapter 7 onward is *incremental* (update after every single observation). Without this chapter the temporal-difference update in Chapter 7 looks like it fell out of the sky [Book §6, pp. 101–102].

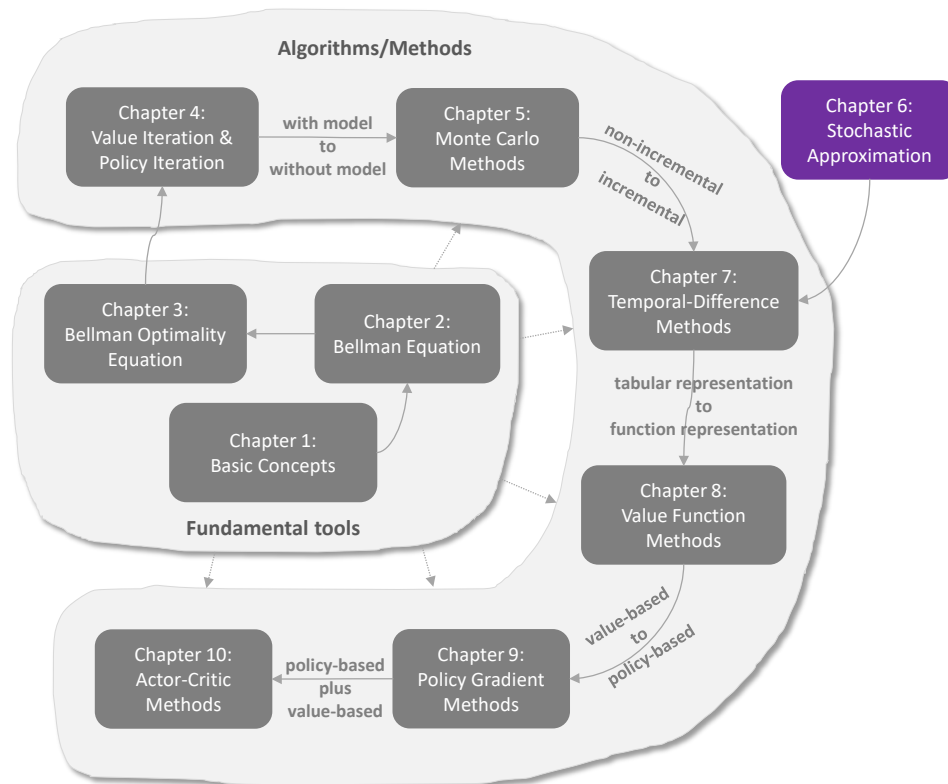


Figure 6.1. Where we are in this book.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§6.1 From batch to incremental: mean estimation again

► Book: §6.1, pp. 102–103; equations (6.1)–(6.4)

We want $\mathbb{E}[X]$ from i.i.d. samples $\{x_i\}$. The batch estimator is $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ [Book equation (6.1), p. 102]. Its drawback: you must wait for all n samples before you have any answer at all.

Define $w_{k+1} = \frac{1}{k} \sum_{i=1}^k x_i$, so that $w_k = \frac{1}{k-1} \sum_{i=1}^{k-1} x_i$. Then [Book §6.1, p. 102]

$$w_{k+1} = \frac{1}{k} \sum_{i=1}^k x_i = \frac{1}{k} \left(\sum_{i=1}^{k-1} x_i + x_k \right) = \frac{1}{k} ((k-1)w_k + x_k) = w_k - \frac{1}{k}(w_k - x_k).$$

Definition (Incremental mean estimation, equation (6.2), p. 103).

$$w_{k+1} = w_k - \frac{1}{k}(w_k - x_k).$$

Worked example: unrolling the recursion [Book equation (6.3), p. 103]

$$\begin{aligned} w_1 &= x_1, \\ w_2 &= w_1 - \frac{1}{1}(w_1 - x_1) = x_1, \\ w_3 &= w_2 - \frac{1}{2}(w_2 - x_2) = x_1 - \frac{1}{2}(x_1 - x_2) = \frac{1}{2}(x_1 + x_2), \\ w_4 &= w_3 - \frac{1}{3}(w_3 - x_3) = \frac{1}{3}(x_1 + x_2 + x_3), \\ &\vdots \\ w_{k+1} &= \frac{1}{k} \sum_{i=1}^k x_i. \end{aligned}$$

So the incremental form is *exactly* the running average — no approximation anywhere. It merely lets you read off an answer at every step instead of only at the end [Book §6.1, p. 103].

Intuition (Read the update as an error correction). Write $w_{k+1} = w_k - \alpha_k(w_k - x_k)$. The bracket $(w_k - x_k)$ is “what I currently believe minus what I just saw” — a prediction error. The update nudges the belief *against* the error, by a fraction α_k . Every learning rule for the rest of this book has this shape:

$$\text{new estimate} = \text{old estimate} - \text{step size} \times \text{error}.$$

Chapter 7’s TD update, Chapter 8’s gradient update and Chapter 10’s actor update are all instances.

Now generalise the coefficient [Book equation (6.4), p. 103]:

$$w_{k+1} = w_k - \alpha_k(w_k - x_k).$$

With $\alpha_k = 1/k$ we recover the exact average. With a general α_k there is no closed form — so we need a theory that tells us when this still converges. That theory is stochastic approximation.

§6.2 The Robbins–Monro algorithm

► Book: §6.2, pp. 103–105; Figures 6.2–6.3, pp. 104–105

The problem. Find the root of $g(w) = 0$, where $g : \mathbb{R} \rightarrow \mathbb{R}$, when

- you do *not* know the expression of g ,
- you do *not* know its derivative,
- and you can only obtain *noisy* evaluations $\tilde{g}(w, \eta) = g(w) + \eta$ at inputs w of your choosing.

It is a black box [Book Figure 6.2, p. 104]. Note that this covers optimisation too: minimising $J(w)$ is finding the root of $g(w) \doteq \nabla_w J(w) = 0$, and solving $g(w) = c$ is finding the root of $g(w) - c$ [Book §6.2, p. 104].

Definition (Robbins–Monro algorithm, equation (6.5), p. 104).

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k), \quad k = 1, 2, 3, \dots$$

That is the entire algorithm. No gradients, no function evaluations other than the noisy output.

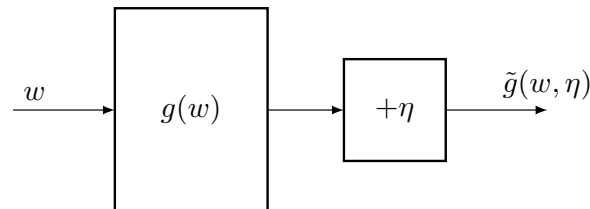


Figure 6.2. An illustration of the problem of solving $g(w) = 0$ from w and $\tilde{g}$.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Worked example: RM in action [Book Figure 6.3, p. 105]

Take $g(w) = w^3 - 5$, whose true root is $5^{1/3} \approx 1.71$. We may observe only $\tilde{g}(w) = g(w) + \eta$ with $\eta \sim N(0, 1)$ i.i.d. Start at $w_1 = 0$ with $a_k = 1/k$. Despite every observation being corrupted by standard normal noise, w_k converges to 1.71 within about 50 iterations.

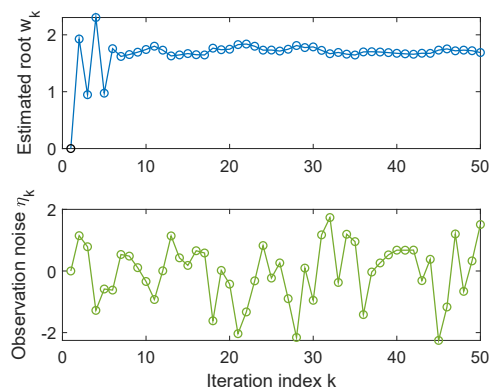


Figure 6.3. An illustrative example of the RM algorithm.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§6.2.1 Why does it work?

► Book: §6.2.1, pp. 105–106; Figure 6.4, p. 106

Strip out the noise and take $g(w) = \tanh(w - 1)$, root $w^* = 1$, $w_1 = 3$, $a_k = 1/k$. The update is $w_{k+1} = w_k - a_k g(w_k)$, and the logic is two lines [Book §6.2.1, p. 105]:

- If $w_k > w^*$ then $g(w_k) > 0$, so $w_{k+1} = w_k - a_k g(w_k) < w_k$: the estimate moves *down*, toward w^* .
- If $w_k < w^*$ then $g(w_k) < 0$, so $w_{k+1} > w_k$: the estimate moves *up*, toward w^* .

Provided the step $a_k g(w_k)$ is not so large that it overshoots past w^* , every iteration strictly reduces the distance to the root.

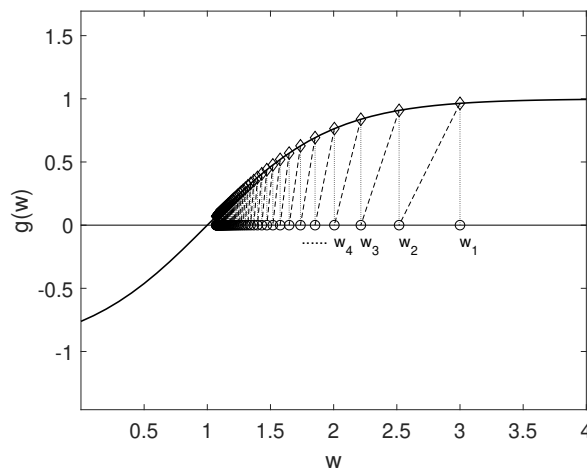


Figure 6.4. An example for illustrating the convergence of the RM algorithm.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§6.2.2 The convergence theorem

► Book: Theorem 6.1, p. 106

Theorem 6.1 (Robbins–Monro, Theorem 6.1, p. 106). *In the RM algorithm (6.5), suppose*

- $0 < c_1 \leq \nabla_w g(w) \leq c_2$ for all w ;
- $\sum_{k=1}^{\infty} a_k = \infty$ and $\sum_{k=1}^{\infty} a_k^2 < \infty$;
- $\mathbb{E}[\eta_k \mid H_k] = 0$ and $\mathbb{E}[\eta_k^2 \mid H_k] < \infty$, where $H_k = \{w_k, w_{k-1}, \dots\}$.

Then w_k converges almost surely to the root w^ with $g(w^*) = 0$.*

Each condition earns its keep [Book §6.2.1, pp. 106–107].

(a) Monotonicity. $\nabla_w g > 0$ means g is strictly increasing, which guarantees the root exists and is *unique*. If g is decreasing, apply the theorem to $-g$. In the optimisation reading $g = \nabla_w J$, so this condition says J is convex — the standard assumption. The upper bound $\nabla_w g \leq c_2$ says the slope cannot blow up: $\tanh(w - 1)$ qualifies; $w^3 - 5$ does *not*, which is why the earlier example needed a sensible starting point.

(b) The step-size conditions. These two conditions appear over and over in RL, so understand them properly.

- $\sum_k a_k^2 < \infty$ forces $a_k \rightarrow 0$. Why do we need that? Because $w_{k+1} - w_k = -a_k \tilde{g}(w_k, \eta_k)$, so if a_k does not vanish the iterates keep jittering forever and never settle [Book §6.2.1, p. 107].
- $\sum_k a_k = \infty$ forces a_k to shrink *slowly*. Why? Sum the increments:

$$w_1 - w_\infty = \sum_{k=1}^{\infty} a_k \tilde{g}(w_k, \eta_k).$$

If $\sum_k a_k < \infty$ then the total distance travelled is bounded by some finite b : $|w_1 - w_\infty| \leq b$ [Book equation (6.6), p. 107]. Choose a starting point with $|w_1 - w^*| > b$ and the algorithm *cannot possibly* reach the root — it runs out of steam. The condition $\sum_k a_k = \infty$ guarantees the algorithm has infinite “fuel” and can therefore start anywhere.

Worked example: $a_k = 1/k$ satisfies both conditions [Book §6.2.1, pp. 107–108]

Divergence of the sum. The harmonic numbers satisfy $\lim_{n \rightarrow \infty} (\sum_{k=1}^n \frac{1}{k} - \ln n) = \kappa \approx 0.577$, the Euler–Mascheroni constant. Since $\ln n \rightarrow \infty$, $\sum_{k=1}^{\infty} \frac{1}{k} = \infty$. ✓

Convergence of the squared sum. The Basel problem gives $\sum_{k=1}^{\infty} \frac{1}{k^2} = \frac{\pi^2}{6} < \infty$. ✓

Small modifications such as $a_k = 1/(k+1)$, or $a_k = c_k/k$ with c_k bounded, also work.

Remark (What practitioners actually do). [Book §6.2.1, p. 108] In real implementations a_k is usually a small *constant*. That violates condition (b), since $\sum a_k^2 = \infty$. The algorithm still converges in a weaker sense — it enters and stays in a small neighbourhood of w^* rather than converging to the point. For nonstationary problems this is a feature, not a bug: a constant step size keeps the estimate responsive to change.

(c) Noise. Mild. It does *not* require Gaussian noise. If $\{\eta_k\}$ is i.i.d. with mean zero and finite variance, then η_k is independent of H_k , so $\mathbb{E}[\eta_k | H_k] = \mathbb{E}[\eta_k] = 0$ automatically.

§6.2.3 Mean estimation is an RM algorithm

► Book: §6.2.2, pp. 108–109

Worked example: casting mean estimation as root-finding

Define $g(w) \doteq w - \mathbb{E}[X]$. Solving $g(w) = 0$ recovers $\mathbb{E}[X]$. Given a sample x of X , the observable quantity is $\tilde{g} = w - x$, and

$$\tilde{g}(w, \eta) = w - x = \underbrace{(w - \mathbb{E}[X])}_{g(w)} + \underbrace{(\mathbb{E}[X] - x)}_{\eta}.$$

The RM algorithm for this g reads

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k (w_k - x_k),$$

which is precisely equation (6.4). Therefore Theorem 6.1 gives: $w_k \rightarrow \mathbb{E}[X]$ almost surely whenever $\sum \alpha_k = \infty$, $\sum \alpha_k^2 < \infty$ and $\{x_k\}$ is i.i.d. **No assumption at all is made about the distribution of X** [Book §6.2.2, p. 109].

§6.3 Dvoretzky's theorem (the general convergence machinery)

► Book: §6.3, pp. 109–114; Theorem 6.2, p. 109; Theorem 6.3, p. 113

This section is more mathematical and can be skimmed on a first pass; but Theorem 6.3 is the tool used to prove Q-learning converges, so it is worth knowing that it exists.

Theorem 6.2 (Dvoretzky, Theorem 6.2, p. 109). *Consider the stochastic process*

$$\Delta_{k+1} = (1 - \alpha_k)\Delta_k + \beta_k \eta_k, \quad \alpha_k \geq 0, \beta_k \geq 0.$$

Then $\Delta_k \rightarrow 0$ almost surely provided

- (a) $\sum_k \alpha_k = \infty$, $\sum_k \alpha_k^2 < \infty$ and $\sum_k \beta_k^2 < \infty$ *uniformly almost surely*;
- (b) $\mathbb{E}[\eta_k \mid H_k] = 0$ and $\mathbb{E}[\eta_k^2 \mid H_k] \leq C$ *almost surely*.

Intuition (How to read the recursion). Δ_k is an *error*. The factor $(1 - \alpha_k)$ contracts it; the term $\beta_k \eta_k$ injects zero-mean noise. Condition (a) says: contract infinitely often ($\sum \alpha_k = \infty$) but inject only finite total noise energy ($\sum \beta_k^2 < \infty$). Signal wins, noise dies.

Unlike the RM theorem, Dvoretzky's theorem allows α_k, β_k to be *random* and to depend on the history H_k [Book §6.3, p. 110]. That extra generality is exactly what is needed to prove the RM theorem itself.

Proof sketch (§6.3.1, pp. 110–111). Set $h_k = \Delta_k^2$. Expanding,

$$h_{k+1} - h_k = -\alpha_k(2 - \alpha_k)\Delta_k^2 + \beta_k^2 \eta_k^2 + 2(1 - \alpha_k)\beta_k \eta_k \Delta_k.$$

Take $\mathbb{E}[\cdot \mid H_k]$. Because Δ_k is H_k -measurable it comes out of the expectation; the cross term vanishes since $\mathbb{E}[\eta_k \mid H_k] = 0$; and for k large enough $\alpha_k \leq 1$ so $-\alpha_k(2 - \alpha_k)\Delta_k^2 \leq 0$. Hence

$$\mathbb{E}[h_{k+1} - h_k \mid H_k] \leq \beta_k^2 C, \quad \text{so} \quad \sum_k \mathbb{E}[h_{k+1} - h_k \mid H_k] \leq C \sum_k \beta_k^2 < \infty.$$

By the quasimartingale convergence theorem, h_k converges. Rearranging the same identity shows $\sum_k \alpha_k \Delta_k^2 < \infty$; combined with $\sum_k \alpha_k = \infty$ this forces $\Delta_k \rightarrow 0$. $\square$

Worked example: mean estimation via Dvoretzky [Book §6.3.2, p. 112]

Let $w^* = \mathbb{E}[X]$ and $\Delta_k = w_k - w^*$. Then

$$\Delta_{k+1} = \Delta_k + \alpha_k(x_k - w^* - \Delta_k) = (1 - \alpha_k)\Delta_k + \alpha_k \underbrace{(x_k - w^*)}_{\eta_k}.$$

Because $\{x_k\}$ is i.i.d., $\mathbb{E}[\eta_k | H_k] = \mathbb{E}[x_k] - w^* = 0$ and $\mathbb{E}[\eta_k^2 | H_k] = \mathbb{E}[x_k^2] - (w^*)^2 = \text{var}[X] < \infty$. Dvoretzky applies.

Worked example: proving Robbins–Monro via Dvoretzky [Book §6.3.3, pp. 112–113]

With $\Delta_k = w_k - w^*$ and $g(w^*) = 0$, the mean value theorem gives $g(w_k) - g(w^*) = \nabla_w g(w'_k)(w_k - w^*)$ for some w'_k between w_k and w^* . Then

$$\Delta_{k+1} = \left[1 - \underbrace{a_k \nabla_w g(w'_k)}_{\alpha_k}\right] \Delta_k + a_k(-\eta_k).$$

Since $0 < c_1 \leq \nabla_w g \leq c_2$, the conditions $\sum a_k = \infty$, $\sum a_k^2 < \infty$ transfer to α_k . Note that this α_k is *random* (it depends on w_k) — which is exactly why we needed Dvoretzky rather than a deterministic argument.

Theorem 6.3 (Multi-variable extension, Theorem 6.3, p. 113). *For a finite index set S and the process $\Delta_{k+1}(s) = (1 - \alpha_k(s))\Delta_k(s) + \beta_k(s)\eta_k(s)$, we get $\Delta_k(s) \rightarrow 0$ almost surely for every $s \in S$ if, for every s ,*

- (a) $\sum_k \alpha_k(s) = \infty$, $\sum_k \alpha_k^2(s) < \infty$, $\sum_k \beta_k^2(s) < \infty$, and $\mathbb{E}[\beta_k(s) | H_k] \leq \mathbb{E}[\alpha_k(s) | H_k]$;
- (b) $\|\mathbb{E}[\eta_k(s) | H_k]\|_\infty \leq \gamma \|\Delta_k\|_\infty$ with $\gamma \in (0, 1)$;
- (c) $\text{var}[\eta_k(s) | H_k] \leq C(1 + \|\Delta_k(s)\|_\infty)^2$.

Remark. Read s as “state” or “state–action pair” [Book §6.3.4, p. 114]. Two upgrades over Theorem 6.2: it handles many variables at once via the max-norm, and it permits the noise to have *nonzero* conditional mean, provided that mean is bounded by γ times the current error. That γ is the discount rate in disguise — which is how the contraction property of Chapter 3 re-enters the convergence proofs of Chapter 7. **Crucially, the conditions must hold at every s** , which is the formal reason RL algorithms need every state–action pair visited infinitely often.

§6.4 Stochastic gradient descent

► Book: §6.4, pp. 114–123

§6.4.1 Three algorithms, one problem

► Book: §6.4, pp. 114–115; equations (6.10)–(6.13)

The problem [Book equation (6.10), p. 114]:

$$\min_w J(w) = \mathbb{E}[f(w, X)].$$

Gradient descent (GD) [Book equation (6.11), p. 114]:

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k) = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)].$$

Unusable when the distribution of X is unknown — and it usually is.

Batch approximation [Book equation (6.12), p. 115]:

$$w_{k+1} = w_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla_w f(w_k, x_i).$$

Better, but it needs all n samples at every single iteration.

Stochastic gradient descent (SGD) [Book equation (6.13), p. 115]:

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k).$$

One sample per update.

Intuition (Why replacing the gradient by one noisy sample is legitimate). [Book §6.4, p. 115] Decompose

$$\nabla_w f(w_k, x_k) = \mathbb{E}[\nabla_w f(w_k, X)] + \underbrace{(\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)])}_{\eta_k},$$

so SGD is exactly GD plus a perturbation: $w_{k+1} = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)] - \alpha_k \eta_k$. Because $\{x_k\}$ is i.i.d., $\mathbb{E}[\eta_k] = 0$. A zero-mean perturbation with a decaying step size averages itself away.

§6.4.2 Mean estimation is an SGD algorithm

► Book: §6.4.1, p. 116; equation (6.14)

Worked example: closing the circle

Pose mean estimation as an optimisation [Book equation (6.14), p. 116]:

$$\min_w J(w) = \mathbb{E}\left[\frac{1}{2} \|w - X\|^2\right], \quad f(w, X) = \frac{1}{2} \|w - X\|^2, \quad \nabla_w f(w, X) = w - X.$$

Setting $\nabla_w J(w) = \mathbb{E}[w - X] = 0$ gives $w^* = \mathbb{E}[X]$. ✓

GD: $w_{k+1} = w_k - \alpha_k \mathbb{E}[w_k - X]$ — unusable, since $\mathbb{E}[X]$ is the very thing we are trying to find.

SGD: $w_{k+1} = w_k - \alpha_k (w_k - x_k)$ — which is equation (6.4) again.

So we have the chain

$$\text{mean estimation} \subset \text{SGD} \subset \text{RM algorithm},$$

each a special case of the next [Book §6.5, p. 123]. That is the architecture of the whole chapter.

§6.4.3 The convergence pattern of SGD

► Book: §6.4.2, pp. 116–118; equation (6.15); Figure 6.5, p. 118

Does the randomness make SGD slow? The answer is a specific and useful one.

Define the relative error between the stochastic and true gradients:

$$\delta_k \doteq \frac{|\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]|}{|\mathbb{E}[\nabla_w f(w_k, X)]|}.$$

Since $\mathbb{E}[\nabla_w f(w^*, X)] = 0$, the denominator can be rewritten and expanded by the mean value theorem [Book equation (6.15), p. 117], giving, when f is strictly convex with $\nabla_w^2 f \geq c > 0$,

$$\delta_k \leq \frac{\overbrace{|\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]|}^{\text{noise in the gradient}}}{c \underbrace{|w_k - w^*|}_{\text{distance to the optimum}}}.$$

Intuition (The headline result). **The relative error is inversely proportional to how far you are from the optimum.** Far away, the true gradient is large and the sampling noise is proportionally negligible, so SGD behaves like plain gradient descent and converges fast. Close to the optimum, the true gradient is nearly zero and the noise dominates, so the iterates rattle around in a neighbourhood [Book §6.4.2, p. 117].

For mean estimation this is exact: $\delta_k = \frac{|\mathbb{E}[X] - x_k|}{|w_k - w^*|}$, so the typical size of δ_k is proportional to the standard deviation of X and inversely proportional to the distance from the answer [Book §6.4.2, p. 117].

The book illustrates this with X uniform on a square of side 20 centred at the origin, $\mathbb{E}[X] = 0$, and 100 i.i.d. samples: the estimate rockets in from a distant initial guess and then wobbles near the origin [Book Figure 6.5, p. 118].

§6.4.4 The deterministic formulation

► Book: §6.4.3, pp. 118–119; equation (6.16)

You will often see SGD written with no random variables anywhere:

$$\min_w J(w) = \frac{1}{n} \sum_{i=1}^n f(w, x_i), \quad \text{updated as} \quad w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k), \quad (6.16)$$

where x_k is “the item fetched at step k ”, not “the k th item of the list”. Is this really SGD? And should we cycle through the list in order or sample at random?

Remark (The answer, [Book §6.4.3, p. 119]). Introduce a random variable X uniform on $\{x_1, \dots, x_n\}$, so $p(X = x_i) = 1/n$. Then

$$\frac{1}{n} \sum_{i=1}^n f(w, x_i) = \mathbb{E}[f(w, X)]$$

exactly — not approximately. So (6.16) is SGD, and it converges *provided x_k is sampled uniformly and independently* from the set. Sampling with replacement means the same item may be drawn twice, which is fine. Marching through the list in a fixed order is *not* covered by the theory.

§6.4.5 BGD, MBGD and SGD

► Book: §6.4.4, pp. 119–121

$$\text{BGD: } w_{k+1} = w_k - \alpha_k \frac{1}{n} \sum_{i=1}^n \nabla_w f(w_k, x_i)$$

$$\text{MBGD: } w_{k+1} = w_k - \alpha_k \frac{1}{m} \sum_{j \in \mathcal{I}_k} \nabla_w f(w_k, x_j), \quad |\mathcal{I}_k| = m$$

$$\text{SGD: } w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k)$$

Common pitfall. Setting $m = 1$ turns MBGD into SGD. But setting $m = n$ does *not* turn MBGD into BGD [Book §6.4.4, p. 120]. MBGD draws n samples *randomly, with replacement*, so it may draw some items twice and miss others; BGD uses each of the n items exactly once. A subtle point, and a favourite exam question.

Worked example: all three on mean estimation [Book §6.4.4, pp. 120–121]

Minimise $J(w) = \frac{1}{2n} \sum_i \|w - x_i\|^2$, optimum $w^* = \bar{x}$. The three updates become

$$\text{BGD: } w_{k+1} = w_k - \alpha_k (w_k - \bar{x}), \quad \text{MBGD: } w_{k+1} = w_k - \alpha_k (w_k - \bar{x}_k^{(m)}), \quad \text{SGD: } w_{k+1} = w_k - \alpha_k (w_k - x_k),$$

with $\bar{x}_k^{(m)} = \frac{1}{m} \sum_{j \in \mathcal{I}_k} x_j$. With $\alpha_k = 1/k$ these unroll to

$$\text{BGD: } w_{k+1} = \frac{1}{k} \sum_{j=1}^k \bar{x} = \bar{x}, \quad \text{MBGD: } w_{k+1} = \frac{1}{k} \sum_{j=1}^k \bar{x}_j^{(m)}, \quad \text{SGD: } w_{k+1} = \frac{1}{k} \sum_{j=1}^k x_j.$$

BGD lands on the exact answer at every step (it had all the information from the start). MBGD converges faster than SGD because each of its increments is already an average and hence less noisy. In the book's simulation, $m = 50$ converges fastest, $m = 5$ next, and $m = 1$ (SGD) slowest — though SGD is still quick while it is far from the answer, exactly as §6.4.2 predicts [Book Figure 6.5, p. 118].

§6.4.6 Convergence of SGD

► Book: §6.4.5, pp. 121–123; Theorem 6.4, p. 121; Box 6.1, p. 122

Theorem 6.4 (Convergence of SGD, Theorem 6.4, p. 121). *For the SGD algorithm (6.13), w_k converges almost surely to the root of $\nabla_w \mathbb{E}[f(w, X)] = 0$ if*

- (a) $0 < c_1 \leq \nabla_w^2 f(w, X) \leq c_2$ (strong convexity with bounded curvature; in the vector case $\nabla_w^2 f$ is the Hessian);
- (b) $\sum_k a_k = \infty$ and $\sum_k a_k^2 < \infty$;
- (c) $\{x_k\}$ are i.i.d.

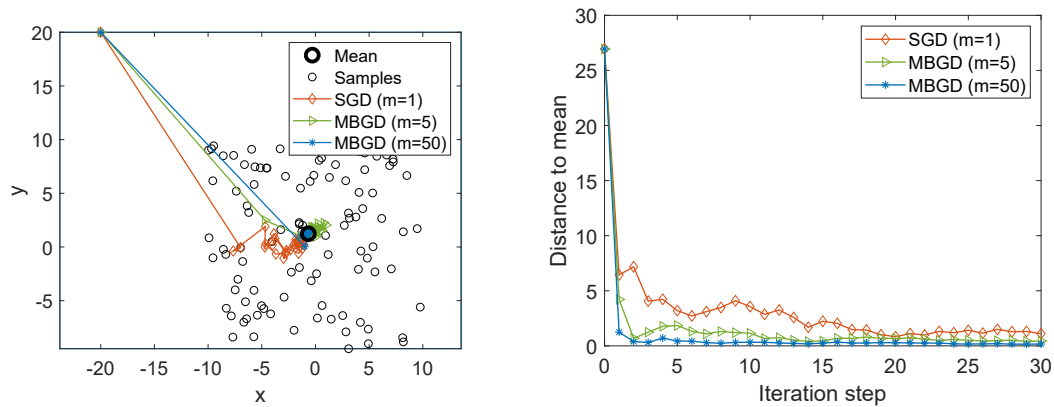


Figure 6.5. An example for demonstrating stochastic and mini-batch gradient descent algorithms. The distribution of $X \in \mathbb{R}^2$ is uniform in the square area centered at the origin with a side length as 20. The mean is $E[X] = 0$. The mean estimation is based on 100 i.i.d. samples.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Proof (Box 6.1, pp. 122–123): SGD is a special RM algorithm. Set $g(w) = \nabla_w J(w) = \mathbb{E}[\nabla_w f(w, X)]$; SGD looks for the root of $g(w) = 0$. The measurable quantity is

$$\tilde{g}(w, \eta) = \nabla_w f(w, x) = \underbrace{\mathbb{E}[\nabla_w f(w, X)]}_{g(w)} + \underbrace{(\nabla_w f(w, x) - \mathbb{E}[\nabla_w f(w, X)])}_{\eta},$$

so the RM algorithm $w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k)$ is SGD. Now check Theorem 6.1's conditions: (a) $\nabla_w g(w) = \mathbb{E}[\nabla_w^2 f(w, X)] \in [c_1, c_2]$; (b) identical; (c) since x_k is independent of $H_k = \{w_k, w_{k-1}, \dots\}$, $\mathbb{E}[\eta_k | H_k] = \mathbb{E}_{x_k}[\nabla_w f(w_k, x_k)] - \mathbb{E}[\nabla_w f(w_k, X)] = 0$, and $\mathbb{E}[\eta_k^2 | H_k] < \infty$ whenever the gradient is bounded. $\square$

§6.5 Chapter 6 summary and self-check

► Book: §6.5, p. 123

Result	Why you need it later
Incremental mean update (6.4)	The template for the TD update in Chapter 7.
RM algorithm (6.5)	TD learning <i>is</i> an RM algorithm applied to the Bellman equation [Book §7.1.2, p. 128].
Step-size conditions $\sum a_k = \infty$, $\sum a_k^2 < \infty$	Reappear verbatim in Theorems 7.1 and 7.2.
Dvoretzky, Theorem 6.3	The machinery behind the convergence of Sarsa and Q-learning.
SGD	The optimiser for every function-approximation method in Chapters 8–10.

- Derive $w_{k+1} = w_k - \frac{1}{k}(w_k - x_k)$ from the definition of the running average, then verify $w_3 = \frac{1}{2}(x_1 + x_2)$.
- Explain, using the “fuel” argument, why $\sum_k a_k = \infty$ is necessary, and construct a step-size sequence that fails it.

3. Show $a_k = 1/\sqrt{k}$ satisfies $\sum a_k = \infty$ but violates $\sum a_k^2 < \infty$. What behaviour would you predict?
4. Why is SGD fast far from the optimum and noisy near it? Give the inequality.
5. Explain why MBGD with $m = n$ is not BGD.
6. Show that mean estimation is simultaneously an instance of MC estimation (Chapter 5), of SGD, and of the RM algorithm.

§7 Temporal-Difference Methods

READING MAP — Chapter 7 of the textbook

Textbook: Chapter 7, pp. 125–150. Sections: §7.1 TD learning of state values (p. 126; algorithm p. 126, properties p. 128, convergence p. 130), §7.2 Sarsa (p. 133; algorithm p. 133, optimal policy learning p. 134), §7.3 n -step Sarsa (p. 138), §7.4 Q-learning (p. 140; algorithm p. 140, off-policy vs on-policy p. 141, implementation p. 144, examples p. 144), §7.5 a unified viewpoint (p. 145), §7.6 summary (p. 148), §7.7 Q&A (p. 149). Algorithms 7.1 (p. 135), 7.2 (p. 143), 7.3 (p. 143). Table 7.2 (p. 148).

Videos: ► L7 P1 — Motivating example; ► L7 P2 — TD: introduction; ► L7 P3 — TD: convergence; ► L7 P4 — Sarsa; ► L7 P5 — Expected & n -step Sarsa; ► L7 P6 — Q-learning: introduction; ► L7 P7 — Q-learning: pseudocode; ► L7 P8 — Unified viewpoint.

One sentence: every algorithm in this chapter is a stochastic approximation algorithm (Chapter 6) applied to a Bellman equation (Chapter 2) or to the Bellman optimality equation (Chapter 3).

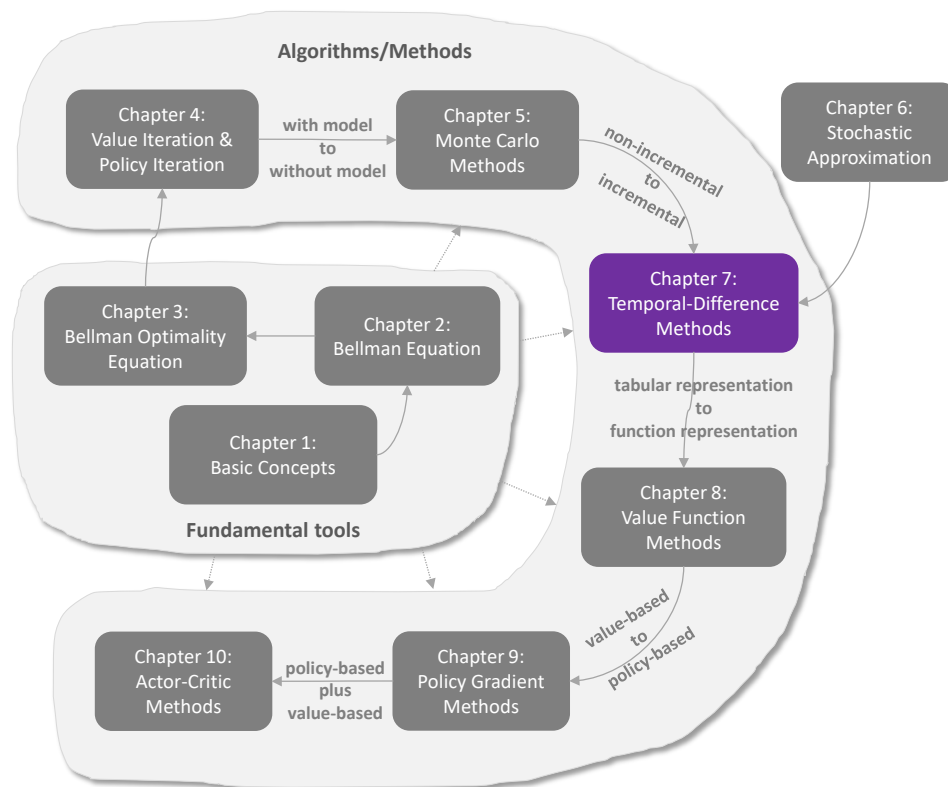


Figure 7.1. Where we are in this book.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§7.1 TD learning of state values

► Book: §7.1.1, pp. 126–128; equations (7.1)–(7.5)

Given a policy π and a stream of experience $(s_0, r_1, s_1, \dots, s_t, r_{t+1}, s_{t+1}, \dots)$ generated by π , we want $v_\pi(s)$ for all s .

Definition (TD learning of state values, equations (7.1)–(7.2), p. 126).

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) [v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))],$$

$$v_{t+1}(s) = v_t(s) \quad \text{for all } s \neq s_t.$$

Common pitfall. The second line is not decorative. **At each time step only the state you actually visited is updated**; every other state keeps its old estimate. The book notes that this line is often omitted for brevity but that the algorithm is mathematically incomplete without it [Book §7.1.1, p. 126].

§7.1.1 Where does this update come from?

► Book: Box 7.1, pp. 127–128

Start from the definition of the state value and use bootstrapping:

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \quad (7.3)$$

and, because $\mathbb{E}[G_{t+1} \mid S_t = s] = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a) v_\pi(s') = \mathbb{E}[v_\pi(S_{t+1}) \mid S_t = s]$,

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \quad (7.4)$$

which the book calls the **Bellman expectation equation** [Book §7.1.1, p. 127]. It is exactly the Bellman equation of Chapter 2, rewritten as an expectation.

Worked example: deriving the TD update from Robbins–Monro [Book Box 7.1, pp. 127–128]

Define

$$g(v_\pi(s_t)) \doteq v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s_t],$$

so that (7.4) is the statement $g(v_\pi(s_t)) = 0$: a **root-finding problem**. We cannot evaluate g (it contains an unknown expectation), but from the sample (s_t, r_{t+1}, s_{t+1}) we can form the noisy observation

$$\tilde{g}(v_\pi(s_t)) = v_\pi(s_t) - (r_{t+1} + \gamma v_\pi(s_{t+1})) = \underbrace{g(v_\pi(s_t))}_{\text{signal}} + \underbrace{\mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid s_t] - (r_{t+1} + \gamma v_\pi(s_{t+1}))}_{\eta, \text{ zero mean}}.$$

Feed this into the RM algorithm $w_{k+1} = w_k - a_k \tilde{g}$ (equation (6.5)) and out comes

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) [v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))]. \quad (7.5)$$

One gap remains: (7.5) contains the *true* $v_\pi(s_{t+1})$, which we do not know. Replace it by the current estimate $v_t(s_{t+1})$ and you get (7.1). Whether that substitution preserves convergence is exactly the content of Theorem 7.1 [Book Box 7.1, p. 128].

§7.1.2 TD target and TD error

► Book: §7.1.2, pp. 128–129; equation (7.6)

Rewrite the update with names attached [Book equation (7.6), p. 128]:

$$\underbrace{v_{t+1}(s_t)}_{\text{new estimate}} = \underbrace{v_t(s_t)}_{\text{current estimate}} - \alpha_t(s_t) \left[\overbrace{v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))}^{\text{TD error } \delta_t} \right].$$

TD target $\bar{v}_t$

Why “target”? Subtract $\bar{v}_t$ from both sides [Book §7.1.2, p. 128]:

$$v_{t+1}(s_t) - \bar{v}_t = [1 - \alpha_t(s_t)](v_t(s_t) - \bar{v}_t) \implies |v_{t+1}(s_t) - \bar{v}_t| < |v_t(s_t) - \bar{v}_t|$$

whenever $0 < \alpha_t(s_t) < 1$. The update provably moves the estimate *toward* $\bar{v}_t$. Hence the name.

Why “temporal difference”? Because $\delta_t = v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))$ compares quantities at two consecutive times t and $t + 1$.

What does the error measure? If the estimates were correct, $v_t = v_\pi$, then

$$\mathbb{E}[\delta_t \mid S_t = s_t] = v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid s_t] = 0$$

by (7.4). **So the TD error is zero in expectation exactly when the estimate is right** [Book §7.1.2, p. 129]. A nonzero TD error is therefore evidence that the current value function is wrong, and its size says by how much.

Intuition (Innovation). [Book §7.1.2, p. 129] The book flags the deeper reading: δ_t is the **innovation** — the genuinely new information contained in the sample (s_t, r_{t+1}, s_{t+1}) that was not already in v_t . This is the same notion as the innovation in Kalman filtering. TD learning is: form a prediction, observe, compute the surprise, and correct the prediction in proportion to the surprise.

§7.1.3 TD versus Monte Carlo

► Book: Table 7.1, p. 130

TD learning	MC learning
Incremental. Updates immediately after each single transition.	Non-incremental. Must wait for a complete episode before it can compute a return.
Continuing tasks. Works with or without terminal states.	Episodic tasks only. Needs episodes that terminate in finite time.
Bootstraps. The update of $v(s_t)$ uses the current estimate $v(s_{t+1})$, so an initial guess is required.	Does not bootstrap. Estimates values directly from returns; no initial guess needed.
Low variance. Sarsa needs samples of only three random variables: $R_{t+1}, S_{t+1}, A_{t+1}$.	High variance. $q_\pi(s_t, a_t)$ requires a sample of $R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$. With episode length L and $ \mathcal{A} $ actions per state there are $ \mathcal{A} ^L$ possible episodes.

Intuition (The bias–variance trade-off in one line). MC is unbiased but noisy: it uses the real return, but the real return depends on every random choice for the rest of the episode. TD is biased but calm: it replaces the far future by a current guess $v_t(s_{t+1})$ — wrong at first, but it collapses the randomness of the entire remaining episode into a single number. Chapter 7’s n -step Sarsa is the dial between the two.

§7.1.4 Convergence

► Book: §7.1.3, pp. 130–132; Theorem 7.1, p. 130; proof in Box 7.2, pp. 131–132

Theorem 7.1 (Convergence of TD learning, Theorem 7.1, p. 130). *Given a policy π , the TD algorithm (7.1) satisfies $v_t(s) \rightarrow v_\pi(s)$ almost surely for every $s \in \mathcal{S}$, provided $\sum_t \alpha_t(s) = \infty$ and $\sum_t \alpha_t^2(s) < \infty$ **for every** $s \in \mathcal{S}$.*

Proof sketch (Box 7.2, pp. 131–132). Let $\Delta_t(s) = v_t(s) - v_\pi(s)$. Subtracting $v_\pi(s)$ from the update gives, for $s = s_t$,

$$\Delta_{t+1}(s) = (1 - \alpha_t(s))\Delta_t(s) + \alpha_t(s) \underbrace{(r_{t+1} + \gamma v_t(s_{t+1}) - v_\pi(s))}_{\eta_t(s)},$$

and for $s \neq s_t$ the same expression holds with $\alpha_t(s) = \eta_t(s) = 0$. This is precisely the process of Theorem 6.3. Condition (a) is the hypothesis. Condition (b) is the interesting one: for $s = s_t$, using $v_\pi(s_t) = \mathbb{E}[r_{t+1} + \gamma v_\pi(s_{t+1}) \mid s_t]$,

$$|\mathbb{E}[\eta_t(s)]| = \gamma \left| \sum_{s'} p(s' \mid s_t) [v_t(s') - v_\pi(s')] \right| \leq \gamma \max_{s'} |v_t(s') - v_\pi(s')| = \gamma \|\Delta_t\|_\infty,$$

which is exactly condition (b) of Theorem 6.3 [Book equation (7.11), p. 132]. Condition (c) follows since r_{t+1} is bounded. $\square$

Remark (Two practical readings of the step-size condition). [Book §7.1.3, pp. 130–131]

- $\alpha_t(s) > 0$ only when s is the state visited at time t ; otherwise it is 0. So $\sum_t \alpha_t(s) = \infty$ demands that *every state be visited infinitely often*. This is exploration, entering through the back door of a convergence proof.
- In practice α is a small constant, which violates $\sum \alpha_t^2 < \infty$. The algorithm then converges only in a weaker sense. The book gives the real reason a constant is used [Book §7.7 Q&A, p. 149]: during optimal-policy learning the policy being evaluated *keeps changing*, so the target is nonstationary; a decaying step size would eventually be too small to track it.

§7.2 Sarsa

► Book: §7.2, pp. 133–137; equation (7.12), p. 133; Algorithm 7.1, p. 135

Estimating v_π is not enough — policy improvement needs q_π . Take the TD update and replace state values with action values.

Definition (Sarsa, equation (7.12), p. 133). *Given samples $(s_0, a_0, r_1, s_1, a_1, \dots)$ generated by π ,*

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})) \right],$$

with $q_{t+1}(s, a) = q_t(s, a)$ for all $(s, a) \neq (s_t, a_t)$.

Remark (The name). Each update consumes the quintuple $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ — **state**, **action**, **reward**, **state**, **action** [Book §7.2.1, p. 133].

What is it solving? The Bellman equation in action-value form [Book equation (7.13), p. 133; proof in Box 7.3, p. 134]:

$$q_\pi(s, a) = \mathbb{E}[R + \gamma q_\pi(S', A') \mid s, a] \quad \text{for all } (s, a).$$

Theorem 7.2 (Convergence of Sarsa, Theorem 7.2, p. 134). $q_t(s, a) \rightarrow q_\pi(s, a)$ *almost surely for all (s, a) provided $\sum_t \alpha_t(s, a) = \infty$ and $\sum_t \alpha_t^2(s, a) < \infty$ for every (s, a) — which requires every state–action pair to be visited infinitely often.*

§7.2.1 Finding optimal policies with Sarsa

► Book: §7.2.2, pp. 134–136; Algorithm 7.1, p. 135

ALGORITHM — Optimal policy learning by Sarsa (Algorithm 7.1, p. 135)

Initialise: $\alpha_t(s, a) = \alpha > 0$; $\varepsilon \in (0, 1)$; $q_0(s, a)$; an ε -greedy π_0 derived from q_0 .

Goal: learn a policy that leads the agent from s_0 to the target state.

For each episode:

- Generate a_0 at s_0 following $\pi_0(s_0)$.
 - While s_t is not the target state:
 - Interact with the environment to obtain r_{t+1}, s_{t+1} ; generate a_{t+1} following $\pi_t(s_{t+1})$.
 - **Value update:** $q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}))]$.
 - **Policy update (ε -greedy at s_t only):** $\pi_{t+1}(a \mid s_t) = 1 - \frac{\varepsilon}{|\mathcal{A}(s_t)|}(|\mathcal{A}(s_t)| - 1)$ if $a = \arg \max_a q_{t+1}(s_t, a)$, else $\frac{\varepsilon}{|\mathcal{A}(s_t)|}$.
 - $s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$.
-

Remark. Notice how aggressive this is: *one* sample updates *one* q -value, and the policy at that state is immediately revised and immediately used to generate the next sample. There is no attempt to evaluate the policy accurately before improving it. This is generalised policy iteration taken to its limit [Book §7.2.2, p. 135].

Worked example: Sarsa finding a path [Book Figure 7.2, p. 136]

Setup. 5×5 grid. Every episode starts at the top-left cell and ends at the target. Rewards: $r_{\text{target}} = 0$, $r_{\text{forbidden}} = r_{\text{boundary}} = -10$, $r_{\text{other}} = -1$. Constant $\alpha = 0.1$, $\varepsilon = 0.1$, $q_0(s, a) = 0$, and π_0 uniform (0.2 each).

Note the task has changed: we want an optimal *path from one starting state*, not optimal policies for all states [Book §7.2.2, p. 135]. That is much cheaper — only states near the path need exploring — but the answer may be locally rather than globally optimal.

Results over 500 episodes.

- *Total reward per episode* rises steadily from about -400 toward 0: early policies keep hitting walls and forbidden cells.

- *Episode length* falls from over 200 steps toward a short path.
- *Spikes*. Around episode 460 the length jumps and the reward collapses. Why? The policy is ε -greedy, so with probability ε it takes a bad action, and a single bad action early in an episode can cost a lot. **The remedy is a decaying ε** [Book §7.2.2, p. 136].
- The final policy reaches the target reliably, but the policies at states far from the path are *not* optimal, because they were never explored.

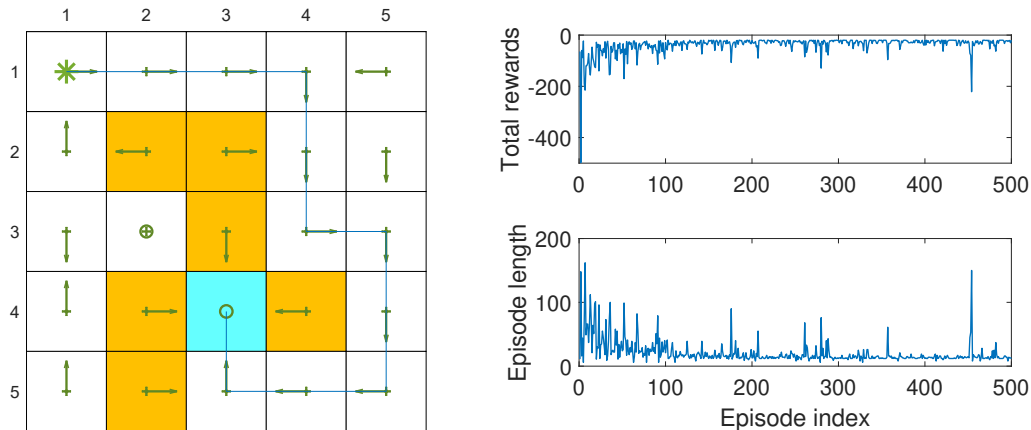


Figure 7.2. An example for demonstrating Sarsa. All the episodes start from the top-left state and terminate when reaching the target state (the blue cell). The goal is to find an optimal path from the starting state to the target state. The reward settings are $r_{\text{target}} = 0$, $r_{\text{forbidden}} = r_{\text{boundary}} = -10$, and $r_{\text{other}} = -1$. The learning rate is $\alpha = 0.1$ and the value of ε is 0.1. The left figure shows the final policy obtained by the algorithm. The right figures show the total reward and length of every episode.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§7.2.2 Expected Sarsa

► Book: Box 7.4, p. 137; equation (7.15)

Definition (Expected Sarsa).

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1}, A)]) \right],$$

where $\mathbb{E}[q_t(s_{t+1}, A)] = \sum_a \pi_t(a | s_{t+1}) q_t(s_{t+1}, a) \doteq v_t(s_{t+1})$.

The only change is the TD target: $r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1}, A)]$ instead of $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$.

Intuition. Why bother computing an expectation? **Variance reduction.** Sarsa's target depends on the random draw a_{t+1} ; Expected Sarsa averages that draw out analytically. The random variables shrink from $\{s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}\}$ to $\{s_t, a_t, r_{t+1}, s_{t+1}\}$ [Book Box 7.4, p. 137]. You pay a little computation and buy a lot of stability — the same trade you make when you use a conditional expectation instead of a single draw anywhere in statistics.

Substituting $\mathbb{E}[q_\pi(S_{t+1}, A_{t+1}) | S_{t+1}] = v_\pi(S_{t+1})$ into (7.15) shows Expected Sarsa solves $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | s, a]$ — the Bellman equation once again [Book Box 7.4, p. 137].

§7.3 n -step Sarsa

► Book: §7.3, pp. 138–139; equation (7.17)

The return admits a whole family of decompositions [Book §7.3, pp. 138–139]:

$$\begin{aligned}
 G_t^{(1)} &= R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) && \longleftarrow \text{Sarsa} \\
 G_t^{(2)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}) \\
 &\vdots \\
 G_t^{(n)} &= R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n}) && \longleftarrow n\text{-step Sarsa} \\
 &\vdots \\
 G_t^{(\infty)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots && \longleftarrow \text{Monte Carlo}
 \end{aligned}$$

Common pitfall. $G_t = G_t^{(1)} = G_t^{(2)} = \cdots = G_t^{(\infty)}$ — these are all *the same random variable*. The superscripts label different ways of writing it, not different quantities [Book §7.3, p. 138]. What differs is which pieces we replace by estimates.

Definition (n -step Sarsa, equation (7.17), p. 139).

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})) \right].$$

Remark (Implementation detail). At time t you do not yet have $(r_{t+n}, s_{t+n}, a_{t+n})$, so the update for (s_t, a_t) must wait until time $t + n$ [Book §7.3, p. 139]:

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t) - \alpha_{t+n-1}(s_t, a_t) \left[q_{t+n-1}(s_t, a_t) - (r_{t+1} + \cdots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n})) \right].$$

Intuition (The dial). $n = 1$ gives Sarsa; $n = \infty$ with $\alpha_t = 1$ gives MC learning. In between:

- **large** $n \Rightarrow$ close to MC: *small bias, large variance* (more real rewards, more randomness);
- **small** $n \Rightarrow$ close to Sarsa: *large bias, small variance* (more reliance on a possibly-wrong bootstrap).

[Book §7.3, p. 139]

§7.4 Q-learning

► Book: §7.4, pp. 140–145; equation (7.18), p. 140; Box 7.5, pp. 140–141

Definition (Q-learning, equation (7.18), p. 140).

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} q_t(s_{t+1}, a)) \right],$$

with all other q -values unchanged.

Two differences from Sarsa, both consequential [Book §7.4.1, p. 140]:

	Sarsa	Q-learning
TD target	$r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$	$r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
Samples needed	$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$	$(s_t, a_t, r_{t+1}, s_{t+1})$
Equation solved	Bellman equation of π	Bellman optimality equation

What it solves [Book equation (7.19), p. 140]:

$$q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) \mid S_t = s, A_t = a \right].$$

This is the BOE (Box 7.5, pp. 140–141). Write the expectation out: $q(s, a) = \sum_r p(r \mid s, a)r + \gamma \sum_{s'} p(s' \mid s, a) \max_a q(s', a)$. Take $\max_{a \in \mathcal{A}(s)}$ of both sides and set $v(s) \doteq \max_a q(s, a)$:

$$v(s) = \max_{a \in \mathcal{A}(s)} \left[\sum_r p(r \mid s, a)r + \gamma \sum_{s'} p(s' \mid s, a)v(s') \right],$$

which is the Bellman optimality equation of Chapter 3 [Book equation (3.1), p. 38]. $\square$

Intuition (Why the max changes everything). The max says: “when I evaluate the consequence of (s_t, a_t) , assume I will behave optimally from s_{t+1} onward — regardless of what I *actually* do next.” That single word decouples what the agent learns about from what the agent does, which is exactly the definition of off-policy learning.

§7.4.1 On-policy versus off-policy

► Book: §7.4.2, pp. 141–143

Definition. Every RL task involves two policies. The **behaviour policy** generates the experience samples. The **target policy** is the one being improved toward optimality. If they coincide, the learning is **on-policy**; if they may differ, it is **off-policy**.

Sarsa is on-policy. Its samples are generated as

$$s_t \xrightarrow{\pi_b} a_t \xrightarrow{\text{model}} r_{t+1}, s_{t+1} \xrightarrow{\pi_b} a_{t+1}.$$

The target $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$ contains a_{t+1} , which was drawn from π_b . So the policy being evaluated is the policy generating the data [Book §7.4.2, p. 142].

Q-learning is off-policy. Its samples are generated as

$$s_t \xrightarrow{\pi_b} a_t \xrightarrow{\text{model}} r_{t+1}, s_{t+1},$$

and the target $r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$ contains *no* action drawn from π_b — the max is computed over the table. The generation of (r_{t+1}, s_{t+1}) is governed by the environment, not by π_b . Hence π_b may be anything [Book §7.4.2, p. 142].

MC learning is on-policy , for the same reason as Sarsa.

Intuition (Why off-policy is valuable). [Book §7.4.2, p. 141] You can learn an optimal policy from data generated by someone else — a human operator, an old policy, a logged dataset. In particular you can use a *strongly exploratory* behaviour policy (say uniform random) to guarantee coverage, while the target policy converges to something purely greedy. Sarsa cannot do this: its exploration is capped by ε , which must stay small to keep the learned policy good, so it is squeezed from both sides.

Common pitfall (On/off-policy is not online/offline). [Book §7.4.2, p. 143] *Online* means updating while interacting with the environment; *offline* means updating from pre-collected data. An on-policy algorithm can be run online but cannot consume other policies' logged data. An off-policy algorithm can be run either way.

§7.4.2 Two implementations

► Book: §7.4.3, p. 144; Algorithms 7.2 and 7.3, p. 143

ALGORITHM — Q-learning, on-policy version (Algorithm 7.2, p. 143)

Initialise: $\alpha > 0$, $\varepsilon \in (0, 1)$, $q_0(s, a)$, and an ε -greedy π_0 .
For each episode, while s_t is not the target state:

- Generate a_t following $\pi_t(s_t)$; obtain r_{t+1}, s_{t+1} from the environment.
- $q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a))]$.
- Update π at s_t to be ε -greedy w.r.t. q_{t+1} .

ALGORITHM — Q-learning, off-policy version (Algorithm 7.3, p. 143)

Initialise: $q_0(s, a)$; a behaviour policy $\pi_b(a | s)$; $\alpha > 0$.
For each episode $\{s_0, a_0, r_1, s_1, a_1, r_2, \dots\}$ **generated by** π_b :

- For each step t :
 - $q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a))]$.
 - Update the target policy at s_t to be **greedy**: $\pi_{T,t+1}(a | s_t) = 1$ if $a = \arg \max_a q_{t+1}(s_t, a)$, else 0.

Remark. Why greedy in Algorithm 7.3 but ε -greedy in Algorithm 7.2? Because in the off-policy version the target policy is never used to collect data, so it has no obligation to explore [Book §7.7 Q&A, p. 150].

§7.4.3 What the experiments show

► Book: §7.4.4, pp. 144–145; Figures 7.3–7.5, pp. 144–147

Worked example: on-policy Q-learning finding a path [Book Figure 7.3, p. 144]

Same setup as the Sarsa run ($r_{\text{target}} = 0$, $r_{\text{forbidden}} = r_{\text{boundary}} = -10$, $r_{\text{other}} = -1$, $\alpha = 0.1$, $\varepsilon = 0.1$). Over 500 episodes the episode length falls and the total reward rises, and an optimal path is found — qualitatively the same picture as Sarsa.

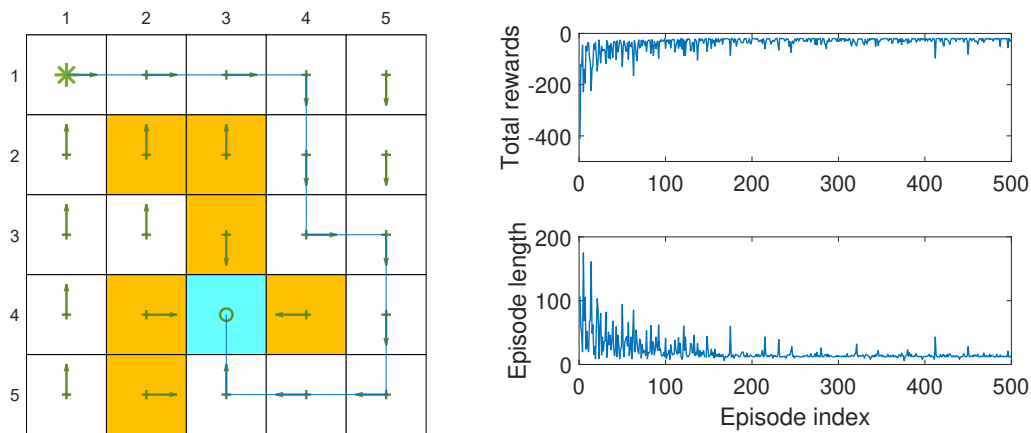


Figure 7.3. An example for demonstrating Q-learning. All the episodes start from the top-left state and terminate after reaching the target state. The aim is to find an optimal path from the starting state to the target state. The reward settings are $r_{\text{target}} = 0$, $r_{\text{forbidden}} = r_{\text{boundary}} = -10$, and $r_{\text{other}} = -1$. The learning rate is $\alpha = 0.1$ and the value of ε is 0.1. The left figure shows the final policy obtained by the algorithm. The right figure shows the total reward and length of every episode.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Worked example: off-policy Q-learning learning *every* state’s policy [Book Figure 7.4, pp. 146]

Setup: $r_{\text{boundary}} = r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$, $\gamma = 0.9$, $\alpha = 0.1$.

Ground truth (from model-based policy iteration): optimal values

5.8	5.6	6.2	6.5	5.8
6.5	7.2	8.0	7.2	6.5
7.2	8.0	10.0	8.0	7.2
8.0	10.0	10.0	10.0	8.0
7.2	9.0	10.0	9.0	8.1

Data: a behaviour policy that is uniform (0.2 on every action), used to generate *one* episode of 100,000 steps. Because the behaviour policy explores well, that single episode visits every state–action pair many times.

Result: the learned target policy is optimal, and the RMS error of the estimated state values converges to zero. Note it is not the *same* arrow pattern as the ground-truth figure — and that is fine, because optimal policies are not unique (§3.4) [Book §7.4.4, p. 145].

Sensitivity to the initial guess: Q-learning bootstraps, so q_0 matters. A good q_0 converges in roughly 10,000 steps; a poor one needs more — but still converges quickly [Book Figure 7.4(g)–(h)].

Sensitivity to the behaviour policy [Book Figure 7.5, p. 147]: with ε -greedy behaviour policies at $\varepsilon = 1$ (uniform), then 0.5, then 0.1, the learning speed degrades *sharply*. Weak exploration means many state-action pairs are barely visited, so their values are never learned. **Off-policy learning does not free you from needing exploration; it frees you from needing the *target* policy to provide it.**

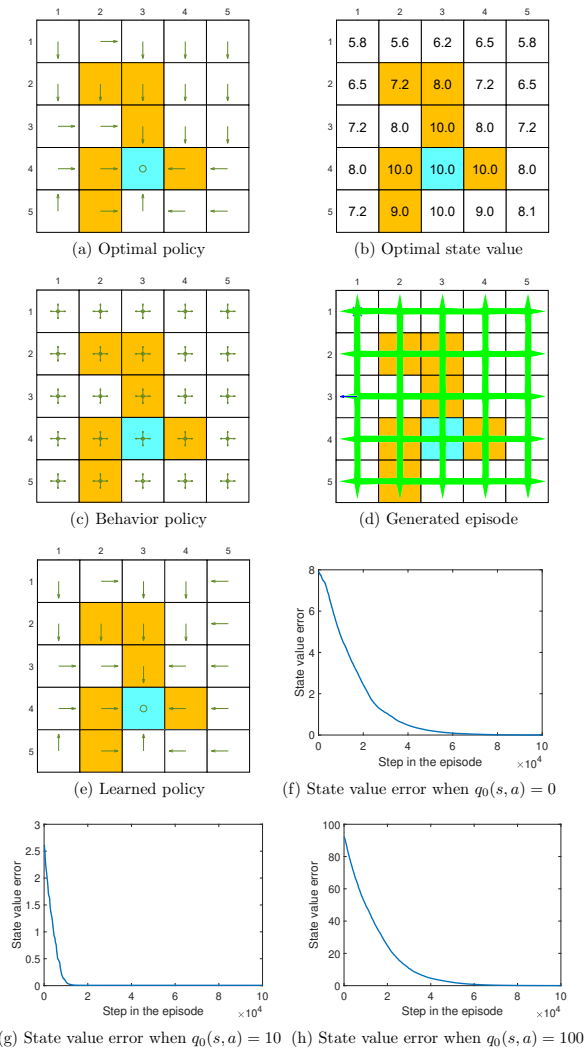


Figure 7.4. Examples for demonstrating off-policy learning via Q-learning. The optimal policy and optimal state values are shown in (a) and (b), respectively. The behavior policy and the generated episode are shown in (c) and (d), respectively. The estimated policy and the estimation error evolution are shown in (e) and (f), respectively. The cases with different initial values are shown in (g) and (h).

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§7.5 A unified viewpoint

► Book: §7.5, pp. 145–148; Table 7.2, p. 148

Every action-value TD algorithm has the same skeleton [Book equation (7.20), p. 145]:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - \bar{q}_t]$$

and differs only in the TD target $\bar{q}_t$.

Algorithm	TD target $\bar{q}_t$	Equation being solved
Sarsa	$r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$	BE: $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid s, a]$
n -step Sarsa	$r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})$	BE: $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n}) \mid s, a]$
Q-learning	$r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$	BOE: $q(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_a q(S_{t+1}, a) \mid s, a]$
Monte Carlo	$r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots$	BE: $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots \mid s, a]$

MC learning is the special case $\alpha_t(s_t, a_t) = 1$, which collapses the update to $q_{t+1}(s_t, a_t) = \bar{q}_t$ [Book §7.5, p. 148].

Remark (The one-line summary of the chapter). **Every algorithm here solves the Bellman equation of a given policy — except Q-learning, which solves the Bellman optimality equation.** That single distinction explains why Q-learning is off-policy and the rest are on-policy [Book §7.6, p. 148].

§7.6 Chapter 7 self-check

1. Write the TD update and identify the TD target and TD error. Prove that the update moves $v_t(s_t)$ strictly closer to the target.
2. Show that $\mathbb{E}[\delta_t \mid S_t = s_t] = 0$ when $v_t = v_\pi$, and explain why this makes δ_t a sensible learning signal.
3. Which of TD and MC has higher variance, and why? Which is biased?
4. Sarsa's target uses a_{t+1} ; Q-learning's uses $\max_a$. Explain in two sentences why this makes one on-policy and the other off-policy.
5. In Figure 7.2, why does the episode length spike near episode 460, and what would you change to prevent it?
6. Theorem 7.1 requires $\sum_t \alpha_t(s) = \infty$ for every s . Translate this condition into a statement about exploration.
7. Fill in the unified table from memory: four algorithms, four TD targets, four equations.

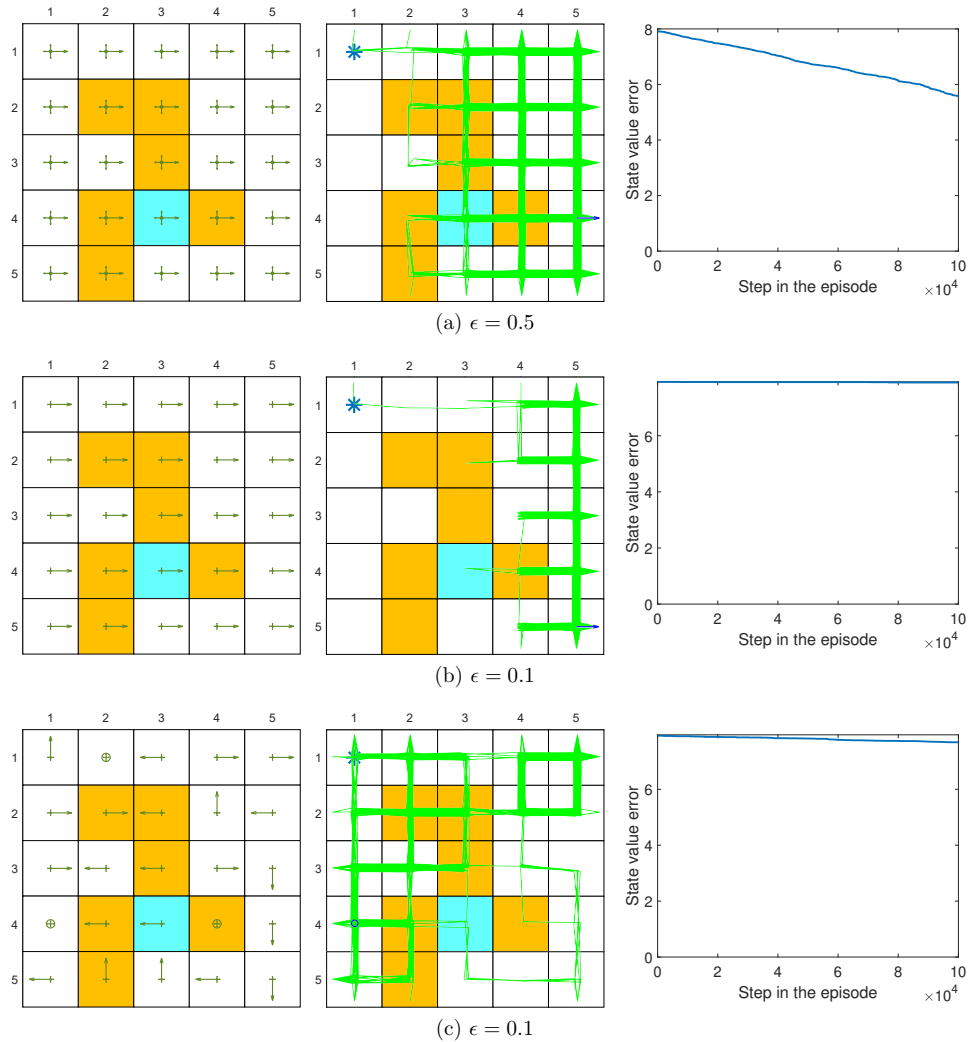


Figure 7.5. The performance of Q-learning drops when the behavior policy is not exploratory. The figures in the left column show the behavior policies. The figures in the middle column show the generated episodes following the corresponding behavior policies. The episode in each example has 100,000 steps. The figures in the right column show the evolution of the root-mean-square error of the estimated state values.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§8 Value Function Methods

READING MAP — Chapter 8 of the textbook

Textbook: Chapter 8, pp. 151–190. Sections: §8.1 from table to function (p. 152), §8.2 TD learning of state values with function approximation (p. 155; objective function p. 156, optimisation algorithms p. 161, choosing approximators p. 162, examples p. 164, theory p. 167), §8.3 TD learning of action values (p. 179; Sarsa p. 179, Q-learning p. 180), §8.4 deep Q-learning (p. 181; algorithm p. 182, examples p. 184), §8.5 summary (p. 186), §8.6 Q&A (p. 187). Algorithms 8.1 (p. 163), 8.2 (p. 180), 8.3 (p. 183). Boxes 8.1–8.6, pp. 157–176.

Videos: ► L8 P1 — Curve fitting; ► L8 P2 — Objective function; ► L8 P3 — Optimisation algorithm; ► L8 P4 — Examples and analysis; ► L8 P5 — Sarsa and Q-learning; ► L8 P6 — DQN basic idea; ► L8 P7 — DQN experience replay; ► L8 P8 — DQN implementation.

The one change: values stop being a lookup table and become a *parametric function*. Everything else in this chapter follows from that.

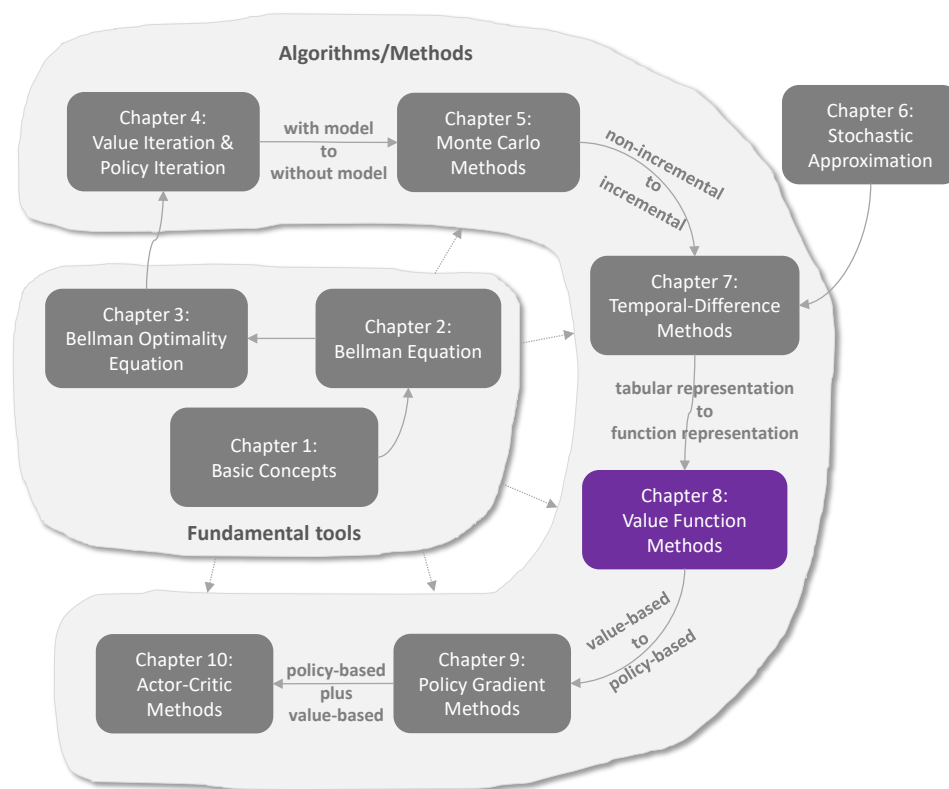


Figure 8.1. Where we are in this book.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§8.1 From table to function

► Book: §8.1, pp. 152–155; Figures 8.2–8.4, pp. 152–154; equations (8.1)–(8.2)

Until now, n states meant n stored numbers $\hat{v}(s_1), \dots, \hat{v}(s_n)$. Plot those n points $(s_i, \hat{v}(s_i))$ and fit a curve through them. The simplest curve is a line [Book equation (8.1), p. 152]:

$$\hat{v}(s, w) = as + b = \underbrace{\begin{bmatrix} s, & 1 \end{bmatrix}}_{\phi^\top(s)} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_w = \phi^\top(s)w.$$

Here $\phi(s)$ is the **feature vector** of s and w the parameter vector. A quadratic works too [Book equation (8.2), p. 154]: $\hat{v}(s, w) = as^2 + bs + c = [s^2, s, 1] [a, b, c]^\top$.

Intuition (An economist already knows this). This is a regression. $\hat{v}(s, w) = \phi^\top(s)w$ is a linear model with regressors $\phi(s)$ and coefficients w . If you knew the true values $\{v_\pi(s_i)\}$ you would just run OLS: minimise $J_1 = \sum_i (\phi^\top(s_i)w - v_\pi(s_i))^2 = \|\Phi w - v_\pi\|^2$ with solution $w^* = (\Phi^\top \Phi)^{-1} \Phi^\top v_\pi$ [Book §8.1, p. 155]. The entire difficulty of this chapter is that the “dependent variable” v_π is unobserved and must be bootstrapped.

Two things change once values live in a function.

Retrieving a value [Book §8.1, pp. 152–153]. With a table you read the entry. With a function you must *compute* $\phi(s)$ and then $\phi^\top(s)w$ (or run a forward pass through a network). The pay-off is storage: you keep m numbers instead of n , with $m \ll n$. The cost is **accuracy**: a straight line cannot pass through arbitrary points. Compressing high-dimensional data into a low-dimensional vector must lose information. Hence “approximation”.

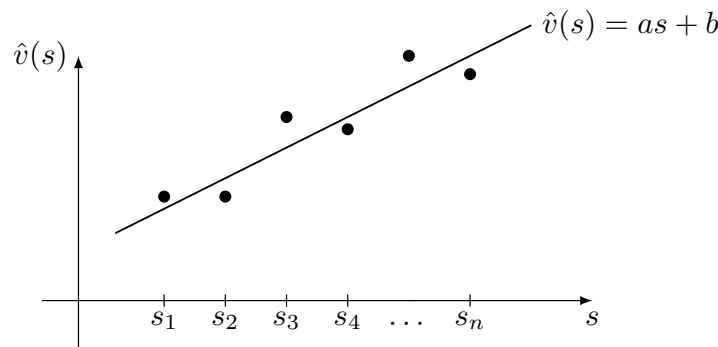


Figure 8.2. An illustration of the function approximation method. The x-axis and y-axis correspond to s and $\hat{v}(s)$, respectively.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Updating a value [Book §8.1, pp. 153–154; Figure 8.4]. With a table you rewrite one entry and nothing else moves. With a function you change w , and changing w moves the value of *every* state at once.

Intuition (Generalisation: the real prize). Suppose you observe a sample only at s_3 . The tabular method updates $\hat{v}(s_3)$ and leaves $\hat{v}(s_1), \hat{v}(s_2)$ untouched. The function method updates w , which shifts the whole curve, so $\hat{v}(s_1)$ and $\hat{v}(s_2)$ also improve — even though those states were never visited [Book Figure 8.4, p. 154]. **One experience teaches you about states you have never seen.** That is what makes large-scale RL possible, and it is why the DQN example below needs 1,000 steps where tabular Q-learning needed 100,000.

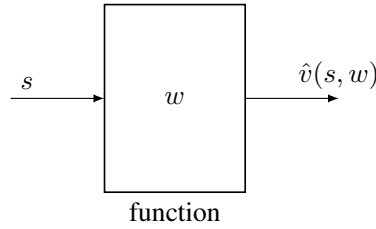
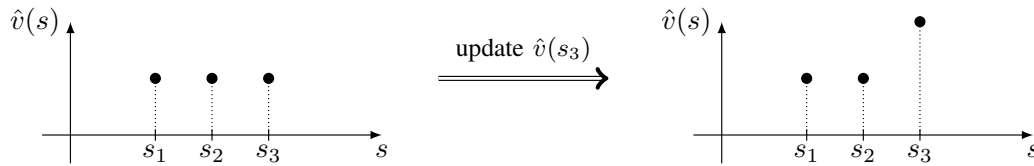
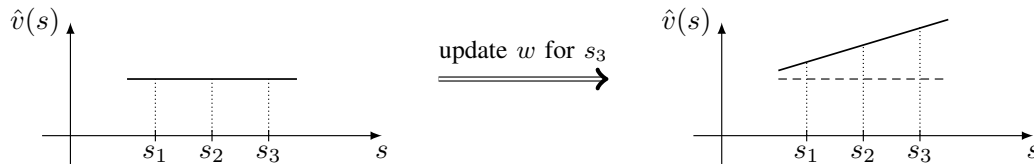


Figure 8.3. An illustration of the process for retrieving the value of s when using the function approximation method.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).



(a) Tabular method: when $\hat{v}(s_3)$ is updated, the other values remain the same.



(b) Function approximation method: when we update $\hat{v}(s_3)$ by changing w , the values of the neighboring states are also changed.

Figure 8.4. An illustration of how to update the value of a state.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Remark (Linear versus nonlinear). $\hat{v}(s, w) = \phi^\top(s)w$ is linear in w , though possibly nonlinear in s . This is **linear function approximation**. Its difficulty is choosing ϕ , which requires knowing something about the task in advance — and you usually do not. The alternative is a neural network as a black-box nonlinear approximator [Book §8.1, pp. 154–155].

§8.2 The objective function

► Book: §8.2.1, pp. 156–161; equations (8.3)–(8.5); Box 8.1, pp. 157–161

Definition (Value-error objective, equation (8.3), p. 156).

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2].$$

An expectation with respect to S — but distributed how? Two answers [Book §8.2.1, p. 156]:

- **Uniform:** $J(w) = \frac{1}{n} \sum_s (v_\pi(s) - \hat{v}(s, w))^2$ [Book equation (8.4)]. Simple, but it treats a state the agent almost never reaches as equal in importance to one it occupies constantly.

- **Stationary distribution:** $J(w) = \sum_s d_\pi(s)(v_\pi(s) - \hat{v}(s, w))^2$ [Book equation (8.5), p. 157]. Weight each state by how often the agent actually ends up there in the long run. This is the book's choice.

§8.2.1 The stationary distribution

► Book: Box 8.1, pp. 157–161; equations (8.6)–(8.10)

Let P_π be the transition matrix under π . Then $[P_\pi^k]_{ij}$ is the probability of going from s_i to s_j in exactly k steps. If d_0 is the initial state distribution, then $d_k^\top = d_0^\top P_\pi^k$ [Book equation (8.7), p. 158]. Under suitable conditions $\lim_{k \rightarrow \infty} P_\pi^k = \mathbf{1}_n d_\pi^\top$, and therefore

$$\lim_{k \rightarrow \infty} d_k^\top = d_0^\top \mathbf{1}_n d_\pi^\top = d_\pi^\top,$$

independently of d_0 [Book equation (8.9), p. 158]. Taking limits in $d_k^\top = d_{k-1}^\top P_\pi$ gives the defining equation

$$\boxed{d_\pi^\top = d_\pi^\top P_\pi} \quad (8.10)$$

so d_π is the left eigenvector of P_π for eigenvalue 1, normalised to sum to one [Book p. 159].

Remark (When does it exist and is it unique?). [Book Box 8.1, p. 159] A Markov process is **irreducible** if every state is reachable from every other in finitely many steps, and **regular** if there is a single k with $P_\pi^k > 0$ elementwise. Regular implies irreducible. Regular processes have a unique stationary distribution with $d_\pi(s) > 0$ for all s . **Which policies give regular chains? Exploratory ones** — an ε -greedy policy assigns positive probability to every action at every state, so states communicate whenever the environment allows [Book p. 160]. Once again, exploration turns out to be a hypothesis of a theorem.

Worked example: computing d_π two ways [Book Figure 8.5, p. 160]

A 2×2 grid under an ε -greedy policy with $\varepsilon = 0.5$, with s_1, s_2, s_3, s_4 the top-left, top-right, bottom-left, bottom-right cells.

Theory. The chain is irreducible (all states communicate) and regular (each state can transition to itself). Here

$$P_\pi^\top = \begin{pmatrix} 0.3 & 0.1 & 0.1 & 0 \\ 0.1 & 0.3 & 0 & 0.1 \\ 0.6 & 0 & 0.3 & 0.1 \\ 0 & 0.6 & 0.6 & 0.8 \end{pmatrix}.$$

Its eigenvalues are $\{-0.0449, 0.3, 0.4449, 1\}$. The unit eigenvector for eigenvalue 1 is $[0.0463, 0.1455, 0.1785, 0.9720]^\top$; rescaling to sum to one,

$$d_\pi = [0.0345, 0.1084, 0.1330, 0.7241]^\top.$$

So the agent spends about 72% of its time in the bottom-right cell.

Simulation. Start at s_1 , run 1,000 steps under the policy, and record the fraction of time spent in each state. The empirical frequencies converge to the theoretical values after a few hundred steps [Book Figure 8.5, p. 160].

Remark. We will never need to compute d_π — doing so would require P_π , i.e. the model. It appears only in the analysis [Book §8.2.1, p. 157]. This is a recurring pattern: the model shows up in the theory and vanishes from the algorithm.

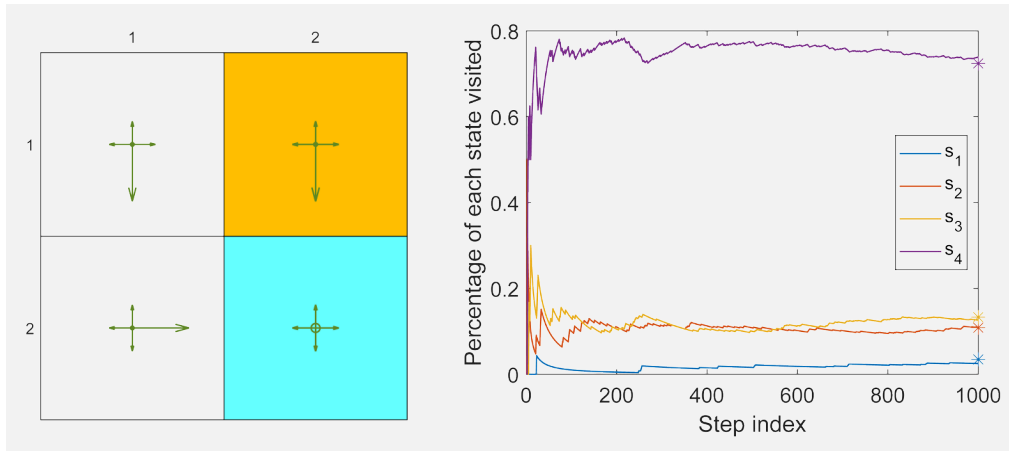


Figure 8.5. Long-term behavior of an ε -greedy policy with $\varepsilon = 0.5$. The asterisks in the right figure represent the theoretical values of the elements of $d\pi$.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§8.3 The algorithm

► Book: §8.2.2, pp. 161–162; equations (8.11)–(8.14)

Differentiate J :

$$\nabla_w J(w) = -2 \mathbb{E}[(v_\pi(S) - \hat{v}(S, w)) \nabla_w \hat{v}(S, w)],$$

so gradient *ascent* on $-J$ gives [Book equation (8.11), p. 162]

$$w_{k+1} = w_k + \alpha_k \mathbb{E}[(v_\pi(S) - \hat{v}(S, w_k)) \nabla_w \hat{v}(S, w_k)].$$

Replace the expectation by a sample (Chapter 6) [Book equation (8.12), p. 162]:

$$w_{t+1} = w_t + \alpha_t (v_\pi(s_t) - \hat{v}(s_t, w_t)) \nabla_w \hat{v}(s_t, w_t).$$

Common pitfall. This is still not implementable: it contains $v_\pi(s_t)$, which is exactly what we are trying to learn [Book §8.2.2, p. 162]. We must substitute an estimate.

Substitution 1 — Monte Carlo. Use the realised return g_t :

$$w_{t+1} = w_t + \alpha_t (g_t - \hat{v}(s_t, w_t)) \nabla_w \hat{v}(s_t, w_t).$$

Substitution 2 — temporal difference. Use the TD target $r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t)$:

Definition (TD learning with function approximation, equation (8.13), p. 162).

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)] \nabla_w \hat{v}(s_t, w_t).$$

In the linear case $\nabla_w \hat{v}(s, w) = \phi(s)$, giving **TD-Linear** [Book equation (8.14), p. 163]:

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \phi^\top(s_{t+1}) w_t - \phi^\top(s_t) w_t] \phi(s_t).$$

ALGORITHM — TD learning of state values with function approximation (Algorithm 8.1, p. 163)

Initialise: a function $\hat{v}(s, w)$ differentiable in w ; initial w_0 .

Goal: learn the state values of a given policy π .

For each episode $\{(s_t, r_{t+1}, s_{t+1})\}_t$ generated by π , for each sample:

- General: $w_{t+1} = w_t + \alpha_t[r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)]\nabla_w \hat{v}(s_t, w_t)$
 - Linear: $w_{t+1} = w_t + \alpha_t[r_{t+1} + \gamma\phi^\top(s_{t+1})w_t - \phi^\top(s_t)w_t]\phi(s_t)$
-

Worked example: tabular TD is a special case of TD-Linear [Book Box 8.2, pp. 163–164]

Choose the feature vector to be the indicator $\phi(s) = e_s \in \mathbb{R}^n$ — a one-hot vector with a 1 in the position of s . Then $\hat{v}(s, w) = e_s^\top w = w(s)$, so the parameter vector *is* the value table. Substituting into (8.14):

$$w_{t+1} = w_t + \alpha_t[r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t)]e_{s_t},$$

and because of e_{s_t} only the s_t entry moves. Left-multiplying by $e_{s_t}^\top$,

$$w_{t+1}(s_t) = w_t(s_t) + \alpha_t[r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t)],$$

which is exactly the tabular TD algorithm (7.1). **Tables are just a particularly stupid choice of features** — one that guarantees zero generalisation.

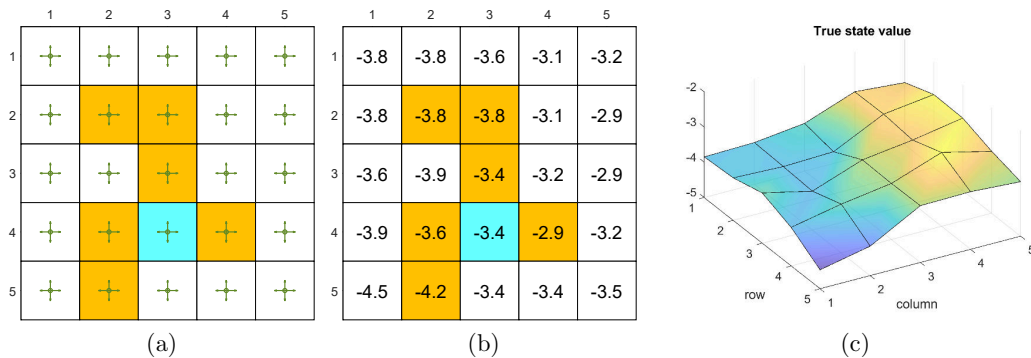


Figure 8.6. (a) The policy to be evaluated. (b) The true state values are represented as a table. (c) The true state values are represented as a 3D surface.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§8.4 Choosing features, with numbers

► **Book:** §8.2.3–8.2.4, pp. 162–167; Figures 8.6–8.8, pp. 164–167

Worked example: approximating 25 state values with fewer than 25 numbers [Book §8.2.4, pp. 164–167]

Setup. 5×5 grid; the policy takes each action with probability 0.2; $r_{\text{forbidden}} = r_{\text{boundary}} = -1$, $r_{\text{target}} = 1$, $\gamma = 0.9$; 500 episodes of 500 steps each; w initialised from a standard normal. The true state values form a bumpy surface with entries such as $-3.8, -3.8, -3.6, -3.1, -3.2$ in row 1 and $-4.5, -4.2, -3.4, -3.4, -3.5$ in row 5 [Book Figure 8.6(b), p. 164].

Polynomial features. Let x, y be the normalised column and row indices in $[-1, 1]$.

- $\phi(s) = [x, y]^\top \in \mathbb{R}^2$ gives $\hat{v} = w_1x + w_2y$: a plane *through the origin*. Too restrictive, since the true surface does not pass through the origin.
- $\phi(s) = [1, x, y]^\top \in \mathbb{R}^3$ [Book equation (8.15), p. 165] gives a general plane. Result: the RMSE falls and then *plateaus* — a plane cannot bend [Book Figure 8.7(a), p. 166].
- $\phi(s) = [1, x, y, x^2, y^2, xy]^\top \in \mathbb{R}^6$ [Book equation (8.16)]: a quadratic surface. Lower plateau.
- $\phi(s) = [1, x, y, x^2, y^2, xy, x^3, y^3, x^2y, xy^2]^\top \in \mathbb{R}^{10}$ [Book equation (8.17)]: lower still.

In all three cases the error plateaus above zero. More features means less bias, but a linear approximator of fixed dimension has a floor it cannot go below [Book §8.2.4, p. 166].

Fourier features. Normalise x, y into $[0, 1]$ and use [Book equation (8.18), p. 167]

$$\phi(s) = \left[\begin{array}{c} 1 \\ \cos(\pi(c_1x + c_2y)) \\ \vdots \end{array} \right]^\top \in \mathbb{R}^{(q+1)^2}, \quad c_1, c_2 \in \{0, 1, \dots, q\}.$$

For $q = 1$: $\phi(s) = [1, \cos(\pi y), \cos(\pi x), \cos(\pi(x + y))]^\top \in \mathbb{R}^4$. Taking $q = 1, 2, 3$ gives feature dimensions 4, 9, 16 and monotonically better accuracy [Book Figure 8.8, p. 167].

Note π here is the circle constant, not a policy — the book flags the clash [Book p. 167].

§8.5 What is TD with function approximation actually minimising?

► Book: §8.2.5, pp. 167–178

Here is an uncomfortable fact the book states plainly: **the algorithm (8.13) does not minimise the objective function (8.3) it was derived from** [Book §8.2.5, p. 168]. The derivation was heuristic (we substituted an estimate for v_π mid-derivation). So what *does* it do? The linear case can be answered completely.

§8.5.1 Convergence

► Book: §8.2.5, pp. 168–169; Lemma 8.1, p. 168; equations (8.20)–(8.22)

Consider the deterministic (expected) version [Book equation (8.19), p. 168]:

$$w_{t+1} = w_t + \alpha_t \mathbb{E}[(r_{t+1} + \gamma \phi^\top(s_{t+1})w_t - \phi^\top(s_t)w_t) \phi(s_t)],$$

where $s_t \sim d_\pi$. Define [Book equation (8.20), p. 168]

$$\Phi = \begin{bmatrix} \vdots \\ \phi^\top(s) \\ \vdots \end{bmatrix} \in \mathbb{R}^{n \times m}, \quad D = \text{diag}(d_\pi(s)) \in \mathbb{R}^{n \times n}.$$

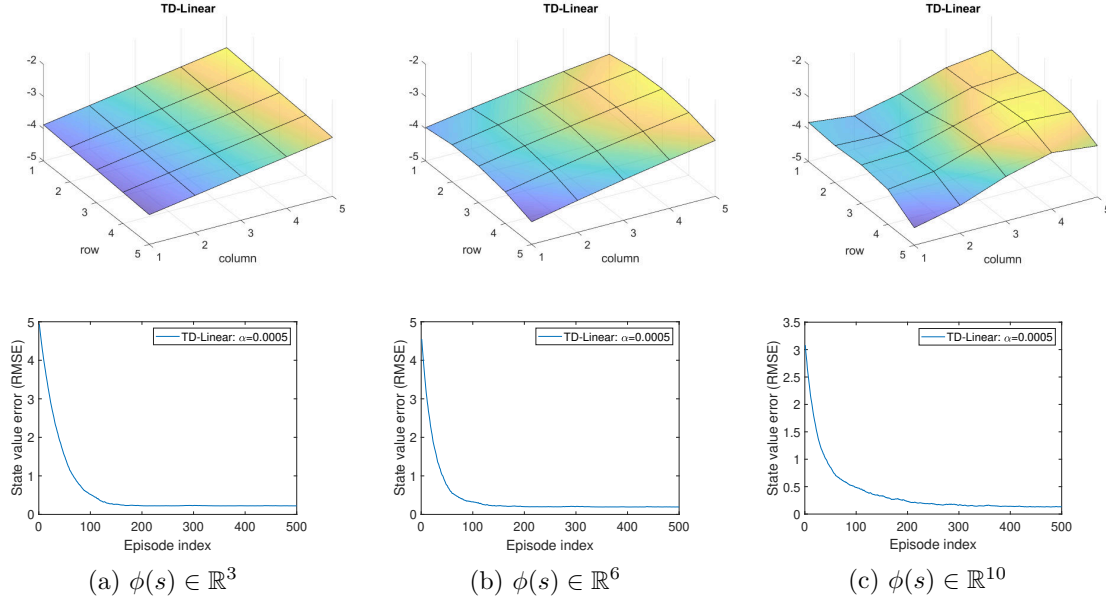


Figure 8.7. TD-Linear estimation results obtained with the polynomial features in (8.15), (8.16), and (8.17).

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

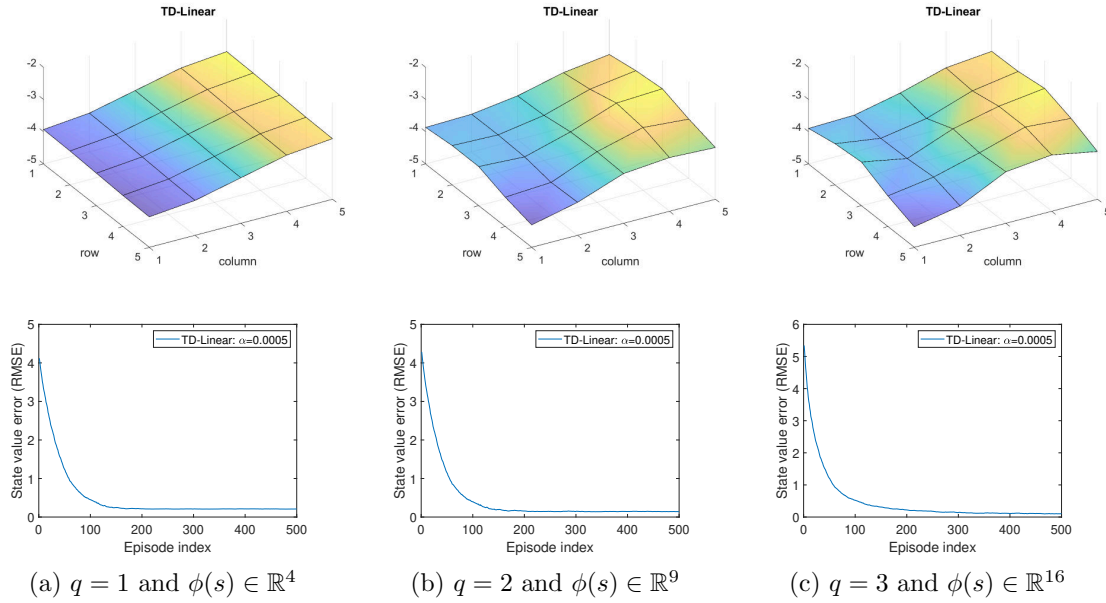


Figure 8.8. TD-Linear estimation results obtained with the Fourier features in (8.18).

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Lemma 8.1 (Lemma 8.1, p. 168; equation (8.21)).

$$\mathbb{E}[(r_{t+1} + \gamma \phi^\top(s_{t+1})w_t - \phi^\top(s_t)w_t)\phi(s_t)] = b - Aw_t,$$

where

$$A \doteq \Phi^\top D(I - \gamma P_\pi) \Phi \in \mathbb{R}^{m \times m}, \quad b \doteq \Phi^\top D r_\pi \in \mathbb{R}^m.$$

So the deterministic algorithm is simply $w_{t+1} = w_t + \alpha_t(b - Aw_t)$ [Book equation (8.22), p. 169] — a linear recursion whose only possible fixed point is

$$w^* = A^{-1}b.$$

Theorem 8.2 (Positive definiteness of A , Box 8.4, pp. 171–173). $A = \Phi^\top D(I - \gamma P_\pi) \Phi$ is positive definite, hence invertible.

Proof idea. Set $M = D(I - \gamma P_\pi)$; it suffices that $M \succ 0$, since Φ has full column rank when the features are linearly independent. Because $x^\top(M - M^\top)x = 0$, $M \succ 0$ iff $M + M^\top \succ 0$. Now use $P_\pi \mathbf{1}_n = \mathbf{1}_n$ and $P_\pi^\top d_\pi = d_\pi$ to get

$$(M + M^\top) \mathbf{1}_n = 2(1 - \gamma)d_\pi > 0,$$

using $d_\pi(s) > 0$ for all s (Box 8.1). Since M has positive diagonal and nonpositive off-diagonal entries, this says $M + M^\top$ is strictly diagonally dominant, and strictly diagonally dominant matrices are positive definite. $\square$

Remark (Two proofs of convergence [Book §8.2.5, pp. 169–170]). *Proof 1.* With $\delta_t = w_t - w^*$ and constant α , $\delta_{t+1} = (I - \alpha A)\delta_t$, so $\|\delta_{t+1}\|_2 \leq \|I - \alpha A\|_2^{t+1} \|\delta_0\|_2$, and $\|I - \alpha A\|_2 < 1$ for small $\alpha > 0$ because $A \succ 0$.

Proof 2. With $g(w) = b - Aw$, finding w^* is a root-finding problem and (8.22) is a Robbins–Monro algorithm — Chapter 6 again, now applied to a deterministic process.

Worked example: the tabular case recovered exactly [Book equation (8.23), p. 169]

Take $\phi(s) = e_s$, so $\Phi = I$. Then $A = D(I - \gamma P_\pi)$ and $b = D r_\pi$, hence

$$w^* = A^{-1}b = (I - \gamma P_\pi)^{-1} D^{-1} D r_\pi = (I - \gamma P_\pi)^{-1} r_\pi = v_\pi.$$

The learned parameter vector is the true state value vector — consistent with Box 8.2.

§8.5.2 Three objective functions

► Book: §8.2.5, pp. 173–175; equation (8.30); Box 8.5, pp. 174–175

- **Value error** $J_E(w) = \|\hat{v}(w) - v_\pi\|_D^2$, where $\|x\|_D^2 = x^\top D x$. The natural objective — but it needs the unknown v_π .
- **Bellman error** $J_{BE}(w) = \|\hat{v}(w) - T_\pi(\hat{v}(w))\|_D^2$ with the Bellman operator $T_\pi(x) \doteq r_\pi + \gamma P_\pi x$ [Book equation (8.30), p. 173]. Implementable, but generally *cannot be driven to zero*: the true v_π may lie outside the span of your approximator.

- **Projected Bellman error** $J_{PBE}(w) = \|\hat{v}(w) - MT_\pi(\hat{v}(w))\|_D^2$, where $M = \Phi(\Phi^\top D\Phi)^{-1}\Phi^\top D$ projects orthogonally onto the span of Φ . **This one can always be driven to zero**, because both sides then live in the same subspace.

Theorem 8.3 (Box 8.5, pp. 174–175). $w^* = A^{-1}b = \arg \min_w J_{PBE}(w)$.

Proof. $J_{PBE}(w) = 0 \iff \Phi w = MT_\pi(\Phi w)$. Substituting M and using that Φ has full column rank,

$$w = (\Phi^\top D\Phi)^{-1}\Phi^\top D(r_\pi + \gamma P_\pi \Phi w) \iff \Phi^\top D r_\pi = \Phi^\top D(I - \gamma P_\pi)\Phi w \iff w = A^{-1}b.$$

□

Intuition (What the projection means). Your approximator can only produce values in the range of Φ — a low-dimensional subspace. Applying the Bellman operator generally throws you out of that subspace. So you cannot ask for $\hat{v} = T_\pi(\hat{v})$; the best you can ask is that $\hat{v}$ equal the *projection back* of $T_\pi(\hat{v})$. TD learning finds that fixed point. It is the RL analogue of “the OLS residual is orthogonal to the regressor space”.

Theorem 8.4 (Error bound, equation (8.32), p. 176; proof in Box 8.6).

$$\|\hat{v}(w^*) - v_\pi\|_D \leq \frac{1}{1-\gamma} \min_w \|\hat{v}(w) - v_\pi\|_D = \frac{1}{1-\gamma} \min_w \sqrt{J_E(w)}.$$

Remark. The right-hand side is the best your approximator class could possibly do, inflated by $1/(1-\gamma)$. The book is candid: this bound is **loose**, especially when γ is near one (at $\gamma = 0.9$ the factor is 10), and is mainly of theoretical value [Book §8.2.5, p. 176].

§8.5.3 Least-squares TD

► [Book: §8.2.5, pp. 177–178; equation \(8.34\)](#)

A and b are themselves expectations [Book p. 177]:

$$A = \mathbb{E}[\phi(s_t)(\phi(s_t) - \gamma\phi(s_{t+1}))^\top], \quad b = \mathbb{E}[r_{t+1}\phi(s_t)].$$

So estimate them by sample averages and invert once [Book equation (8.34), p. 177]:

$$\hat{A}_t = \sum_{k=0}^{t-1} \phi(s_k)(\phi(s_k) - \gamma\phi(s_{k+1}))^\top, \quad \hat{b}_t = \sum_{k=0}^{t-1} r_{k+1}\phi(s_k), \quad w_t = \hat{A}_t^{-1}\hat{b}_t.$$

The $1/t$ factors cancel and are omitted. In practice $\hat{A}_t$ is regularised as $\hat{A}_t + \sigma I$ for small σ , since it may be singular for small t [Book p. 178]. LSTD uses data more efficiently than TD-Linear but requires a matrix inverse and works only in the linear case.

§8.6 Sarsa and Q-learning with function approximation

► [Book: §8.3, pp. 179–181; equations \(8.35\)–\(8.36\); Algorithms 8.2–8.3, pp. 180–181](#)

The extension is mechanical: replace $\hat{v}(s, w)$ with $\hat{q}(s, a, w)$.

Definition (Sarsa with function approximation, equation (8.35), p. 179).

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t)] \nabla_w \hat{q}(s_t, a_t, w_t).$$

In the linear case $\hat{q}(s, a, w) = \phi^\top(s, a)w$ and $\nabla_w \hat{q} = \phi(s, a)$.

Definition (Q-learning with function approximation, equation (8.36), p. 180).

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t).$$

ALGORITHM — Sarsa with function approximation (Algorithm 8.2, p. 180)

Initialise: $w_0; \pi_0; \alpha > 0; \varepsilon \in (0, 1)$.

For each episode: generate a_0 at s_0 from π_0 ; while s_t is not the target:

- Obtain r_{t+1}, s_{t+1} from the environment; draw $a_{t+1} \sim \pi_t(s_{t+1})$.
- $w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t)] \nabla_w \hat{q}(s_t, a_t, w_t)$.
- Set $\pi_{t+1}(\cdot | s_t)$ to be ε -greedy w.r.t. $\hat{q}(s_t, \cdot, w_{t+1})$.
- $s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$.

Worked example: results on the grid world [Book Figures 8.9–8.10, pp. 180–181]

Both use Fourier features of order 5, with $\gamma = 0.9$, $\varepsilon = 0.1$, $r_{\text{boundary}} = r_{\text{forbidden}} = -10$, $r_{\text{target}} = 1$, $\alpha = 0.001$. Over 500 episodes the total reward rises from around -4000 toward 0 and the episode length falls, in both Sarsa and Q-learning. Both find a good path from the top-left state.

Common pitfall. In Algorithms 8.2 and 8.3 the *values* are functions but the *policy* is still a table — $\pi(a | s)$ is stored per state and updated per state. So the state and action spaces must still be finite. Removing that last table is the entire subject of Chapter 9 [Book §8.3.2, p. 181].

§8.7 Deep Q-learning (DQN)

► Book: §8.4, pp. 181–186; equations (8.37)–(8.38); Algorithm 8.3, p. 183

Definition (DQN objective, equation (8.37), p. 182).

$$J = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w) \right)^2 \right],$$

the *squared Bellman optimality error*.

The justification: the Bellman optimality equation in action-value form (Box 7.5, p. 140) says the bracket is zero in expectation when $\hat{q}$ is exact.

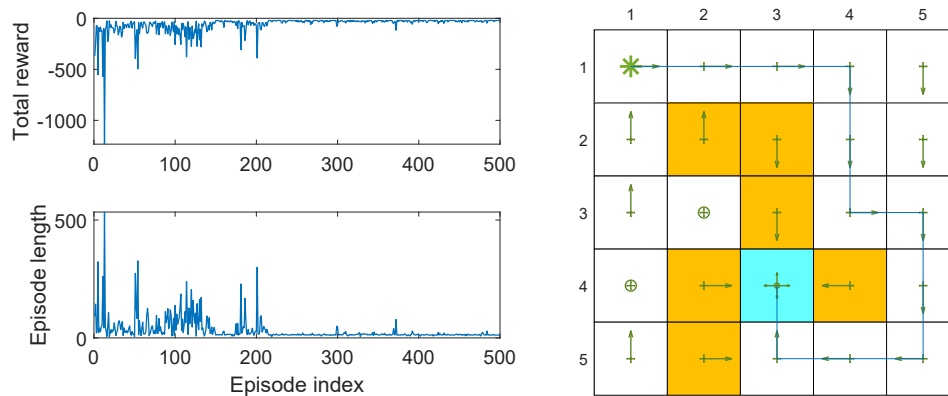


Figure 8.9. Sarsa with linear function approximation. Here, $\gamma = 0.9$, $\varepsilon = 0.1$, $r_{\text{boundary}} = r_{\text{forbidden}} = -10$, $r_{\text{target}} = 1$, and $\alpha = 0.001$.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

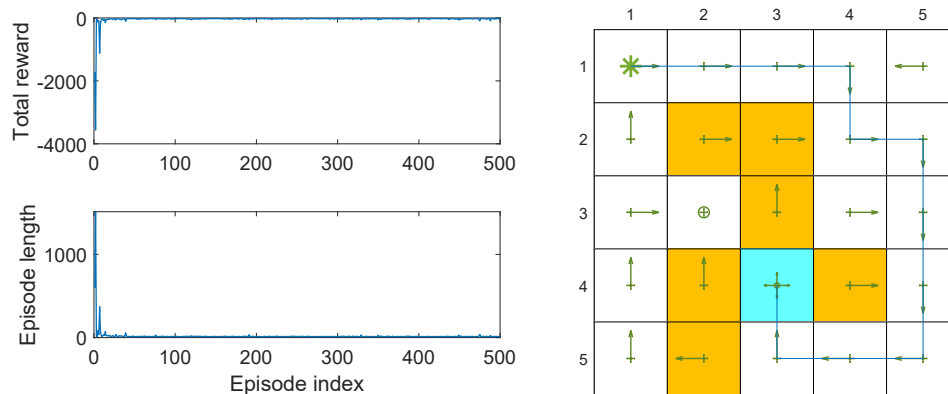


Figure 8.10. Q-learning with linear function approximation. Here, $\gamma = 0.9$, $\varepsilon = 0.1$, $r_{\text{boundary}} = r_{\text{forbidden}} = -10$, $r_{\text{target}} = 1$, and $\alpha = 0.001$.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§8.7.1 Trick 1: two networks

► [Book: §8.4.1, pp. 182–183](#)

The parameter w appears *twice* — inside $\hat{q}(S, A, w)$ and inside the target $y = R + \gamma \max_a \hat{q}(S', a, w)$ — which makes the gradient awkward. DQN freezes the copy inside the target:

- a **main network** $\hat{q}(s, a, w)$, updated every iteration;
- a **target network** $\hat{q}(s, a, w_T)$, held fixed and refreshed by $w_T \leftarrow w$ every C iterations.

With w_T fixed, [\[Book equation \(8.38\), p. 182\]](#)

$$\nabla_w J = -\mathbb{E} \left[\left(R + \gamma \max_a \hat{q}(S', a, w_T) - \hat{q}(S, A, w) \right) \nabla_w \hat{q}(S, A, w) \right].$$

Intuition. Without the frozen copy you are chasing a target that moves every time you step toward it — the classic recipe for instability. Freezing w_T for C iterations turns each phase into an ordinary supervised regression with fixed labels.

§8.7.2 Trick 2: experience replay

► Book: §8.4.1, pp. 183–184

Store transitions (s, a, r, s') in a **replay buffer** $\mathcal{B}$, and each update draw a *mini-batch uniformly at random* from $\mathcal{B}$ rather than using samples in the order they arrived.

Remark (The book’s reason, which is not the usual folklore). [Book §8.4.1, pp. 183–184; §8.6 Q&A, p. 187] Look back at the objective (8.37). To define that expectation we must specify the distribution of (S, A) . The distributions of R and S' follow from the model once (S, A) is given; for (S, A) the simplest assumption is *uniform*. But samples generated sequentially by a behaviour policy are neither uniform nor independent. **Uniform sampling from the replay buffer is what makes the assumption behind the objective function true.** A side benefit — often quoted as the main one — is that each sample can be reused many times, which matters when data is scarce.

Remark. Does tabular Q-learning need replay? *No*, but it can use it, and it benefits from the sample reuse [Book §8.6 Q&A, p. 187].

ALGORITHM — Deep Q-learning, off-policy (Algorithm 8.3, p. 183)

Initialise: a main network and a target network with identical parameters.

Goal: learn optimal action values from samples generated by a behaviour policy π_b .

Store samples generated by π_b in a replay buffer $\mathcal{B} = \{(s, a, r, s')\}$. Then for each iteration:

- Draw a mini-batch uniformly from $\mathcal{B}$.
- For each sample, compute the target $y_T = r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$.
- Update the main network to minimise $\sum (y_T - \hat{q}(s, a, w))^2$ over the mini-batch (using standard neural-network training tools, not equation (8.38) directly).
- Every C iterations, set $w_T \leftarrow w$.

§8.7.3 Two experiments, one encouraging and one sobering

► Book: §8.4.2, pp. 184–186; Figures 8.11–8.12, pp. 185–186

Worked example: DQN with 1,000 samples [Book Figure 8.11, p. 185]

Setup. 5×5 grid, $\gamma = 0.9$, $r_{\text{boundary}} = r_{\text{forbidden}} = -10$, $r_{\text{target}} = 1$. Behaviour policy: uniform (probability 0.2 on each action). **A single episode of 1,000 steps.** Replay buffer holds those 1,000 transitions; mini-batch size 100.

Network. One hidden layer of 100 neurons; three inputs (normalised row, normalised column, normalised action index) and one output. The book notes the design choice: feeding row and column rather than a flat state index tells the network that states have a 2-D geometry — *the more structure you encode, the better the network performs* [Book p. 185]. An alternative is two inputs and five outputs (one per action), as in the original DQN paper.

Result. The loss converges to zero and the state-value RMSE converges to zero; the greedy policy extracted from the learned $\hat{q}$ is optimal.

The headline comparison. Tabular Q-learning needed an episode of 100,000 steps for the same task [Book Figure 7.4]; DQN needed 1,000 — a hundredfold reduction. Two reasons [Book p. 185]: function approximation generalises across states, and replay lets each sample be used many times.

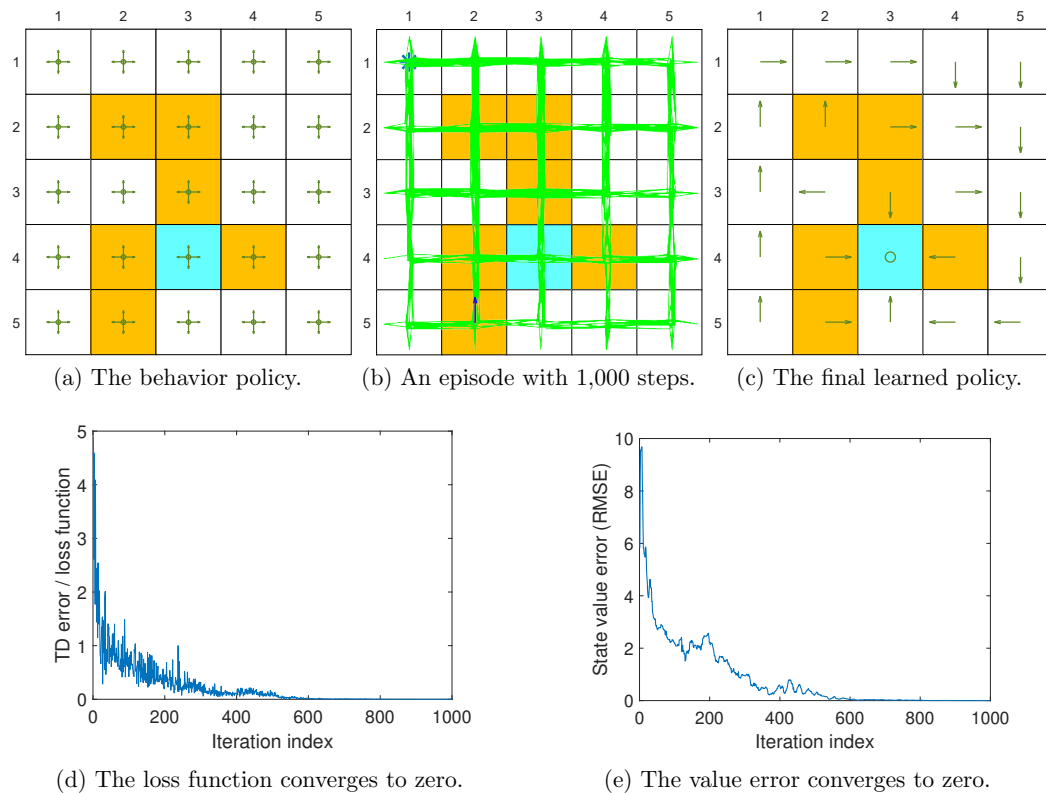


Figure 8.11. Optimal policy learning via deep Q-learning. Here, $\gamma = 0.9$, $r_{\text{boundary}} = r_{\text{forbidden}} = -10$, and $r_{\text{target}} = 1$. The batch size is 100.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Worked example: DQN with 100 samples [Book Figure 8.12, p. 186]

Same setup, but the episode is only 100 steps and the mini-batch size is 50.

Result. The loss function *still* converges to zero — but the state-value RMSE plateaus around 5 and never converges.

Diagnosis. The network fits the training samples perfectly and the optimal action values badly. Zero training loss is not evidence of a correct value function; it is evidence that you have enough parameters to memorise the data you have. Any econometrician will recognise the phenomenon: in-sample fit is not identification.

§8.8 Chapter 8 summary and self-check

► Book: §8.5, pp. 186–187

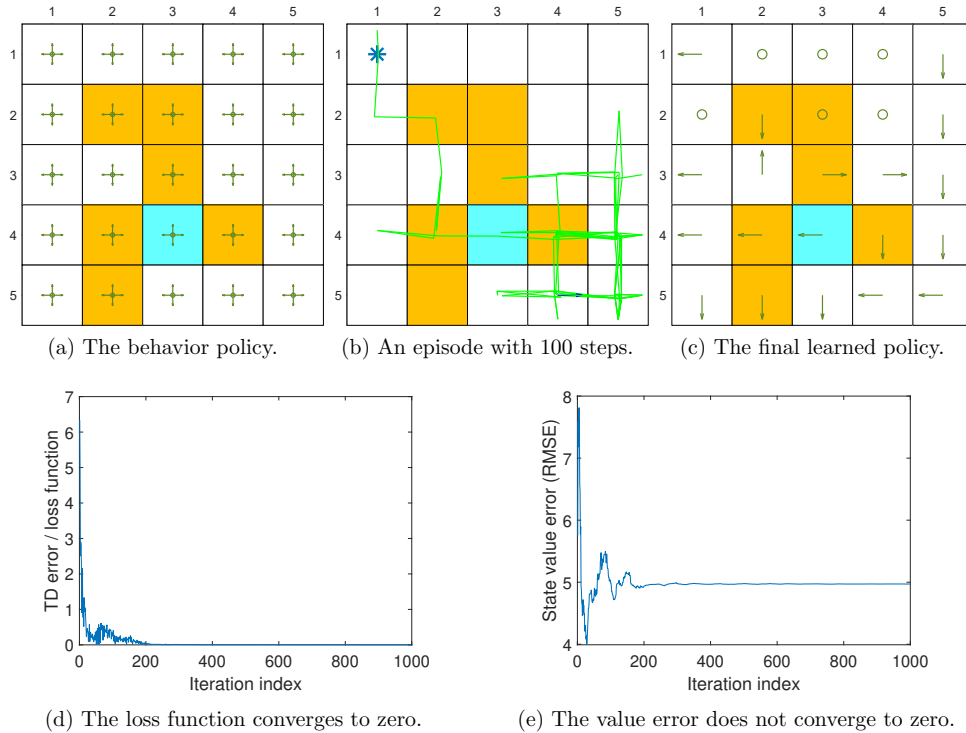


Figure 8.12. Optimal policy learning via deep Q-learning. Here, $\gamma = 0.9$, $r_{\text{boundary}} = r_{\text{forbidden}} = -10$, and $r_{\text{target}} = 1$. The batch size is 50.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

Concept	The point
Table $\rightarrow$ function	Saves storage and, more importantly, generalises across states. Costs accuracy.
Stationary distribution d_π	Weights the objective by how often each state is actually visited; exists and is positive for exploratory policies.
TD-Linear	$w_{t+1} = w_t + \alpha_t[r_{t+1} + \gamma\phi^\top(s_{t+1})w_t - \phi^\top(s_t)w_t]\phi(s_t)$; tabular TD is the case $\phi(s) = e_s$.
$w^* = A^{-1}b$	The unique fixed point; it minimises the <i>projected</i> Bellman error, not the value error.
DQN	Squared Bellman optimality error, minimised with a frozen target network and uniform experience replay.

1. Explain generalisation using Figure 8.4: why does a sample at s_3 change $\hat{v}(s_1)$ under function approximation but not under a table?
2. Show that $\phi(s) = e_s$ turns TD-Linear into tabular TD.
3. Why is the objective weighted by d_π rather than uniformly? What property of the policy guarantees d_π exists and is strictly positive?
4. Distinguish J_E , J_{BE} and J_{PBE} . Which one can always be driven to zero, and why?
5. In the $\varepsilon = 0.5$ example, verify that $d_\pi = [0.0345, 0.1084, 0.1330, 0.7241]^\top$ satisfies $d_\pi^\top = d_\pi^\top P_\pi$ numerically.

6. State the book's mathematical reason for uniform experience replay — not the folklore reason.
7. In Figure 8.12 the loss goes to zero but the value error does not. Explain the difference in one sentence.

§9 Policy Gradient Methods

READING MAP — Chapter 9 of the textbook

Textbook: Chapter 9, pp. 191–214. Sections: §9.1 policy representation — from table to function (p. 192), §9.2 metrics for defining optimal policies (p. 193), §9.3 gradients of the metrics (p. 198; discounted case p. 200, undiscounted case p. 205), §9.4 Monte Carlo policy gradient / REINFORCE (p. 210), §9.5 summary (p. 213), §9.6 Q&A (p. 213). Theorem 9.1 (p. 198), Lemmas 9.1–9.2 (p. 200), Theorems 9.2–9.3 (pp. 201–203), Theorems 9.4–9.5 (pp. 205–209), Algorithm 9.1 (p. 212). Table 9.2 (p. 197).

Videos: ► L9 P1 — Basic idea; ► L9 P2 — Metric 1: average value; ► L9 P3 — Metric 2: average reward; ► L9 P4 — Gradients of the metrics; ► L9 P5 — Gradient-based algorithms & REINFORCE.

The break with everything before. Chapters 1–8 are *value-based*: learn values, then read off a greedy policy. This chapter is *policy-based*: parameterise the policy directly and do gradient ascent on a scalar objective. Values become optional.

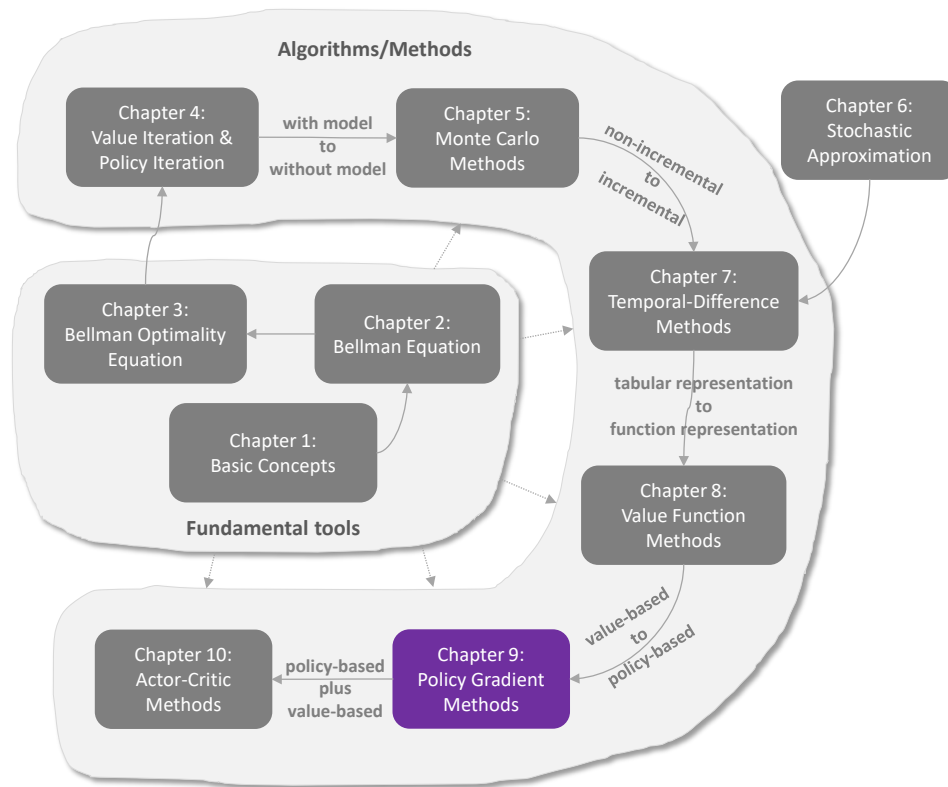


Figure 9.1. Where we are in this book.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§9.1 From a table of probabilities to a function

► Book: §9.1, pp. 192–193; Table 9.1 and Figure 9.2, p. 192

Until now a policy was a table: one row per state, one column per action, holding $\pi(a | s)$. Now write

$$\pi(a | s, \theta), \quad \theta \in \mathbb{R}^m,$$

also written $\pi_\theta(a | s)$. Three things change [Book §9.1, p. 192]:

	Tabular policy	Function policy
Optimality means	maximise <i>every</i> state value	maximise a chosen <i>scalar metric</i>
Update by	rewriting table entries	changing θ
Retrieve $\pi(a s)$ by	reading an entry	evaluating the function at (s, a)

Common pitfall. The redefinition of optimality is the substantive change, not a technicality. With a table you could demand dominance at every state simultaneously (Definition 3.1) and Chapter 3 proved such a policy exists. With a parametric function you cannot: the parameter budget forces trade-offs across states. So optimality must be defined by a single number $J(\theta)$ [Book §9.1, p. 192; §3.7 Q&A, p. 54].

The method in one line [Book §9.1, p. 193]: pick a scalar metric $J(\theta)$ and run gradient *ascent*,

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta_t).$$

Three questions follow, and the chapter answers them in order: which metric (§9.2), what is its gradient (§9.3), how do we estimate that gradient from samples (§9.4)?

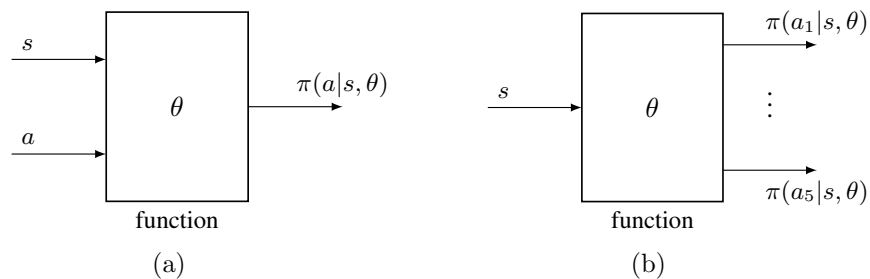


Figure 9.2. Function representations of policies. The functions may have different structures.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§9.2 Two metrics

► Book: §9.2, pp. 193–198; Table 9.2, p. 197

§9.2.1 Metric 1: average state value

► Book: §9.2, pp. 193–195; equation (9.1)

Definition.

$$\bar{v}_\pi = \sum_{s \in \mathcal{S}} d(s) v_\pi(s) = \mathbb{E}_{S \sim d}[v_\pi(S)] = d^\top v_\pi,$$

where $d(s) \geq 0$ and $\sum_s d(s) = 1$.

Two choices of the weights [Book §9.2, pp. 193–194]:

- **d independent of π** , written d_0 and the metric $\bar{v}_\pi^0$. Either $d_0(s) = 1/|\mathcal{S}|$ (all states equally important), or, when there is a fixed starting state s_0 , $d_0(s_0) = 1$ and $d_0(s) = 0$ elsewhere. The second case is common: “how good is this policy when I start from home?”
- **$d = d_\pi$, the stationary distribution** (Box 8.1). States visited often in the long run get more weight. Note this makes the weights themselves depend on θ , which will complicate the gradient.

Remark (An equivalent form you will meet elsewhere). [Book equation (9.1), p. 194] The literature often writes

$$J(\theta) = \lim_{n \rightarrow \infty} \mathbb{E} \left[\sum_{t=0}^n \gamma^t R_{t+1} \right] = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right].$$

This looks different but equals $\bar{v}_\pi$: condition on S_0 and apply the law of total expectation, $\mathbb{E}[\sum_t \gamma^t R_{t+1}] = \sum_s d(s) \mathbb{E}[\sum_t \gamma^t R_{t+1} \mid S_0 = s] = \sum_s d(s) v_\pi(s)$.

§9.2.2 Metric 2: average reward

► Book: §9.2, pp. 195–197; equations (9.2)–(9.5); Box 9.1, pp. 196–197

Definition (equations (9.2)–(9.3), p. 195).

$$\bar{r}_\pi \doteq \sum_{s \in \mathcal{S}} d_\pi(s) r_\pi(s) = \mathbb{E}_{S \sim d_\pi} [r_\pi(S)], \quad r_\pi(s) \doteq \sum_a \pi(a \mid s, \theta) r(s, a),$$

with $r(s, a) \doteq \mathbb{E}[R \mid s, a] = \sum_r r p(r \mid s, a)$. In vector form $\bar{r}_\pi = d_\pi^\top r_\pi$.

Remark (An equivalent form, equations (9.4)–(9.5), p. 195).

$$J(\theta) = \lim_{n \rightarrow \infty} \frac{1}{n} \mathbb{E} \left[\sum_{t=0}^{n-1} R_{t+1} \right] = \bar{r}_\pi.$$

The proof (Box 9.1, pp. 196–197) is a nice piece of probability: the Cesàro mean lets you replace the average of a convergent sequence by its limit, so $\lim_n \frac{1}{n} \mathbb{E}[\sum_t R_{t+1} \mid S_0 = s_0] = \lim_t \mathbb{E}[R_{t+1} \mid S_0 = s_0]$; then $\mathbb{E}[R_{t+1} \mid S_0 = s_0] = \sum_s r_\pi(s) p^{(t)}(s \mid s_0) \rightarrow \sum_s r_\pi(s) d_\pi(s) = \bar{r}_\pi$ because $p^{(t)}(s \mid s_0) \rightarrow d_\pi(s)$. **The starting state washes out** — long-run average reward does not care where you began.

§9.2.3 The two metrics are the same metric

Lemma 9.1 (Lemma 9.1, p. 200). *In the discounted case $\gamma \in (0, 1)$,*

$$\boxed{\bar{r}_\pi = (1 - \gamma) \bar{v}_\pi.}$$

Proof. Multiply the Bellman equation $v_\pi = r_\pi + \gamma P_\pi v_\pi$ on the left by $d_\pi^\top$ and use $d_\pi^\top P_\pi = d_\pi^\top$:

$$\bar{v}_\pi = \bar{r}_\pi + \gamma d_\pi^\top P_\pi v_\pi = \bar{r}_\pi + \gamma d_\pi^\top v_\pi = \bar{r}_\pi + \gamma \bar{v}_\pi.$$

□

Remark. Since $(1 - \gamma) > 0$, maximising one maximises the other. You may pick whichever is analytically convenient [Book §9.2, p. 198].

Metric	Expression 1	Expression 2	Expression 3
$\bar{v}_\pi$	$\sum_s d(s) v_\pi(s)$	$\mathbb{E}_{S \sim d}[v_\pi(S)]$	$\lim_{n \rightarrow \infty} \mathbb{E}[\sum_{t=0}^n \gamma^t R_{t+1}]$
$\bar{r}_\pi$	$\sum_s d_\pi(s) r_\pi(s)$	$\mathbb{E}_{S \sim d_\pi}[r_\pi(S)]$	$\lim_{n \rightarrow \infty} \frac{1}{n} \mathbb{E}[\sum_{t=0}^{n-1} R_{t+1}]$

(The book's Table 9.2, p. 197.)

§9.3 The policy gradient theorem

► Book: §9.3, pp. 198–200; Theorem 9.1, p. 198; equations (9.8)–(9.11)

Theorem 9.2 (Policy gradient theorem, Theorem 9.1, p. 198).

$$\nabla_\theta J(\theta) = \sum_{s \in \mathcal{S}} \eta(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a \mid s, \theta) q_\pi(s, a), \quad (9.8)$$

and equivalently, in expectation form,

$$\nabla_\theta J(\theta) = \mathbb{E}_{S \sim \eta, A \sim \pi(S, \theta)} [\nabla_\theta \ln \pi(A \mid S, \theta) q_\pi(S, A)]. \quad (9.9)$$

Remark (What Theorem 9.1 is, precisely). [Book §9.3, pp. 198–199] It is a *summary* of Theorems 9.2, 9.3 and 9.5, which handle different metrics ($\bar{v}_\pi^0$, $\bar{v}_\pi$, $\bar{r}_\pi$) and the discounted versus undiscounted cases. Depending on the case, η is either ρ_π or d_π , and the equality may be exact or approximate. The book is explicit that most readers should learn the statement and skip the proofs.

Worked example: why the logarithm appears [Book §9.3, p. 199; equations (9.10)–(9.11)]

Rewrite (9.8) as an expectation over states:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S \sim \eta} \left[\sum_a \nabla_\theta \pi(a \mid S, \theta) q_\pi(S, a) \right].$$

The inner sum is not yet an expectation over actions, because the summand has no factor $\pi(a \mid s, \theta)$. Manufacture one using the derivative of a logarithm:

$$\nabla_\theta \ln \pi(a \mid s, \theta) = \frac{\nabla_\theta \pi(a \mid s, \theta)}{\pi(a \mid s, \theta)} \implies \nabla_\theta \pi(a \mid s, \theta) = \pi(a \mid s, \theta) \nabla_\theta \ln \pi(a \mid s, \theta).$$

Substituting,

$$\nabla_\theta J(\theta) = \mathbb{E}_{S \sim \eta} \left[\sum_a \pi(a \mid S, \theta) \nabla_\theta \ln \pi(a \mid S, \theta) q_\pi(S, a) \right] = \mathbb{E}_{S \sim \eta, A \sim \pi} [\nabla_\theta \ln \pi(A \mid S, \theta) q_\pi(S, A)].$$

That is the entire role of the logarithm: it converts a weighted sum into an expectation, so the gradient can be estimated by sampling [Book §9.6 Q&A, p. 214]. This is the “log-derivative trick”, and it is the same manoeuvre used to derive the score function in maximum likelihood estimation.

Common pitfall (The policy must be strictly positive). $\ln \pi(a \mid s, \theta)$ requires $\pi(a \mid s, \theta) > 0$ for every (s, a) [Book §9.3, p. 199]. The standard construction is the **softmax** [Book equation (9.12), p. 199]:

$$\pi(a \mid s, \theta) = \frac{e^{h(s, a, \theta)}}{\sum_{a' \in \mathcal{A}} e^{h(s, a', \theta)}},$$

where $h(s, a, \theta)$ is a preference for taking a at s . This guarantees $\pi(a \mid s, \theta) \in (0, 1)$ and $\sum_a \pi(a \mid s, \theta) = 1$, and is implemented as a network with a softmax output layer.

An important consequence [Book p. 199]: **the policy is automatically stochastic and therefore automatically exploratory**. There is no ε to tune — exploration is built into the representation. (Economists will recognise the softmax as the choice probability of a multinomial logit model, with h playing the role of deterministic utility.)

§9.3.1 Sketch of the derivation, discounted case

► Book: §9.3.1, pp. 200–205; Lemma 9.2, p. 200; Theorems 9.2–9.3, pp. 201–203

Lemma 9.3 (Gradient of a single state value, Lemma 9.2, p. 200; equation (9.14)).

$$\nabla_{\theta} v_{\pi}(s) = \sum_{s' \in \mathcal{S}} \Pr_{\pi}(s' \mid s) \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi(a \mid s', \theta) q_{\pi}(s', a),$$

where $\Pr_{\pi}(s' \mid s) \doteq \sum_{k=0}^{\infty} \gamma^k [P_{\pi}^k]_{ss'} = [(I - \gamma P_{\pi})^{-1}]_{ss'}$ is the **discounted total probability** of ever reaching s' from s .

Proof idea (Box 9.2, pp. 201–202). Differentiate $v_{\pi}(s) = \sum_a \pi(a \mid s, \theta) q_{\pi}(s, a)$ by the product rule:

$$\nabla_{\theta} v_{\pi}(s) = \sum_a [\nabla_{\theta} \pi(a \mid s, \theta) q_{\pi}(s, a) + \pi(a \mid s, \theta) \nabla_{\theta} q_{\pi}(s, a)].$$

Because $q_{\pi}(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) v_{\pi}(s')$ and $r(s, a)$ does not depend on θ , we get $\nabla_{\theta} q_{\pi}(s, a) = \gamma \sum_{s'} p(s' \mid s, a) \nabla_{\theta} v_{\pi}(s')$. So $\nabla_{\theta} v_{\pi}$ appears on both sides — bootstrapping again. Writing it in matrix form and inverting gives the $(I - \gamma P_{\pi})^{-1}$ factor. $\square$

Theorem 9.4 (Gradient of $\bar{v}_{\pi}^0$, Theorem 9.2, p. 201).

$$\nabla_{\theta} \bar{v}_{\pi}^0 = \mathbb{E}[\nabla_{\theta} \ln \pi(A \mid S, \theta) q_{\pi}(S, A)], \quad S \sim \rho_{\pi}, \quad A \sim \pi(S, \theta),$$

where $\rho_{\pi}(s) = \sum_{s'} d_0(s') \Pr_{\pi}(s \mid s')$ [Book equation (9.19), p. 201].

Theorem 9.5 (Gradients of $\bar{r}_{\pi}$ and $\bar{v}_{\pi}$, Theorem 9.3, p. 203).

$$\nabla_{\theta} \bar{r}_{\pi} = (1 - \gamma) \nabla_{\theta} \bar{v}_{\pi} \approx \sum_s d_{\pi}(s) \sum_a \nabla_{\theta} \pi(a \mid s, \theta) q_{\pi}(s, a) = \mathbb{E}[\nabla_{\theta} \ln \pi(A \mid S, \theta) q_{\pi}(S, A)],$$

with $S \sim d_{\pi}$, $A \sim \pi(S, \theta)$. **The approximation is more accurate the closer γ is to 1.**

Remark (Where the approximation comes from). Differentiating $\bar{v}_{\pi} = \sum_s d_{\pi}(s) v_{\pi}(s)$ produces two terms [Book equation (9.20), p. 204]: $\sum_s \nabla_{\theta} d_{\pi}(s) v_{\pi}(s)$ and $\sum_s d_{\pi}(s) \nabla_{\theta} v_{\pi}(s)$. The second is the one we keep. The first — the effect of θ on where the agent spends its time — is what gets dropped. It is small when $\gamma \rightarrow 1$.

Remark (The undiscounted case). [Book §9.3.2, pp. 205–210; Theorems 9.4–9.5] For $\gamma = 1$ the state value is no longer well defined by the usual sum, and one works instead with the **Poisson equation**, whose solution set is characterised by Theorem 9.4 (p. 205). The conclusion, Theorem 9.5 (p. 209), is that $\nabla_{\theta} \bar{r}_{\pi}$ has the same form as before — but with an *exact* equality rather than an approximation [Book p. 209]. This is why the average-reward metric is worth defining separately: it is valid in both the discounted and undiscounted settings [Book §9.6 Q&A, p. 214].

§9.4 REINFORCE

► [Book: §9.4, pp. 210–212; equations \(9.31\)–\(9.33\); Algorithm 9.1, p. 212](#)

Gradient ascent using the true gradient [\[Book equation \(9.31\), p. 210\]](#):

$$\theta_{t+1} = \theta_t + \alpha \mathbb{E}[\nabla_{\theta} \ln \pi(A | S, \theta_t) q_{\pi}(S, A)].$$

Replace the expectation by one sample (Chapter 6 again) [\[Book equation \(9.32\), p. 211\]](#):

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} \ln \pi(a_t | s_t, \theta_t) q_t(s_t, a_t).$$

When $q_t(s_t, a_t)$ is obtained by Monte Carlo estimation — i.e. the realised discounted return from (s_t, a_t) — this is **REINFORCE**, also called Monte Carlo policy gradient.

§9.4.1 Reading the update

► [Book: §9.4, pp. 211–212; equation \(9.33\)](#)

Using $\nabla_{\theta} \ln \pi = \nabla_{\theta} \pi / \pi$, rewrite the update as

$$\theta_{t+1} = \theta_t + \alpha \underbrace{\frac{q_t(s_t, a_t)}{\pi(a_t | s_t, \theta_t)}}_{\beta_t} \nabla_{\theta} \pi(a_t | s_t, \theta_t) \iff \theta_{t+1} = \theta_t + \alpha \beta_t \nabla_{\theta} \pi(a_t | s_t, \theta_t). \quad (9.33)$$

Interpretation 1: it does the obvious thing. A first-order Taylor expansion gives [\[Book p. 211\]](#)

$$\pi(a_t | s_t, \theta_{t+1}) \approx \pi(a_t | s_t, \theta_t) + \alpha \beta_t \|\nabla_{\theta} \pi(a_t | s_t, \theta_t)\|_2^2.$$

So

$$\beta_t \geq 0 \implies \pi(a_t | s_t, \theta_{t+1}) \geq \pi(a_t | s_t, \theta_t), \quad \beta_t < 0 \implies \pi(a_t | s_t, \theta_{t+1}) < \pi(a_t | s_t, \theta_t),$$

and the size of the change scales with $|\beta_t|$. *Actions that turned out well become more likely; actions that turned out badly become less likely.*

Interpretation 2: exploration and exploitation, automatically. [\[Book p. 212\]](#) Look at $\beta_t = q_t(s_t, a_t) / \pi(a_t | s_t, \theta_t)$:

- β_t is *proportional* to $q_t(s_t, a_t)$: high-value actions get reinforced. That is **exploitation**.
- β_t is *inversely proportional* to $\pi(a_t | s_t, \theta_t)$ (when $q_t > 0$): an action that currently has low probability gets a *larger* boost when it turns out well. That is **exploration**.

Intuition. This is a strictly more elegant answer than ε -greedy. There, exploration was bolted on with a tunable knob that had to be decayed by hand and that degraded optimality (§5.5). Here the trade-off falls out of the algebra of a single ratio.

§9.4.2 How the samples should be drawn

► Book: §9.4, p. 212

- **States:** S should be drawn from η — either d_π or ρ_π . Both describe long-run behaviour under π .
- **Actions:** A should be drawn from $\pi(A | S, \theta)$. **Therefore the policy gradient method is on-policy.** You are estimating an expectation taken under π , so your data must come from π . (Chapter 10 shows how importance sampling relaxes this.)

Remark. The book is honest that the ideal sampling scheme is not followed in practice, because it is too sample-inefficient [Book p. 212]. Algorithm 9.1 instead generates one whole episode and then updates θ once per transition in it.

ALGORITHM — REINFORCE — Monte Carlo policy gradient (Algorithm 9.1, p. 212)

Initialise: $\theta; \gamma \in (0, 1); \alpha > 0$.

Goal: learn a policy maximising $J(\theta)$.

For each episode:

- Generate an episode $\{s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T\}$ following $\pi(\theta)$.
 - For $t = 0, 1, \dots, T - 1$:
 - **Value update:** $q_t(s_t, a_t) = \sum_{k=t+1}^T \gamma^{k-t-1} r_k$
 - **Policy update:** $\theta \leftarrow \theta + \alpha \nabla_\theta \ln \pi(a_t | s_t, \theta) q_t(s_t, a_t)$
-

Worked example: REINFORCE on a two-action state, by hand

Suppose $\mathcal{A} = \{a_1, a_2\}$ and a softmax policy with preferences $h(s, a_1, \theta) = \theta_1$, $h(s, a_2, \theta) = \theta_2$, so

$$\pi(a_1 | s, \theta) = \frac{e^{\theta_1}}{e^{\theta_1} + e^{\theta_2}}.$$

Take $\theta = (0, 0)$, so $\pi(a_1 | s) = \pi(a_2 | s) = 0.5$. The score is

$$\nabla_\theta \ln \pi(a_1 | s, \theta) = (1 - \pi(a_1 | s, \theta), -\pi(a_2 | s, \theta)) = (0.5, -0.5).$$

Suppose we sample a_1 and observe return $q_t = +2$. With $\alpha = 0.1$,

$$\theta \leftarrow (0, 0) + 0.1(2)(0.5, -0.5) = (0.1, -0.1),$$

giving $\pi(a_1 | s) = e^{0.1} / (e^{0.1} + e^{-0.1}) = 0.550$. The probability of the action that worked rose from 0.5 to 0.55; its rival fell to 0.45. Had the return been -2 , the signs would flip and $\pi(a_1 | s)$ would fall to 0.45. Note also that no explicit exploration parameter was needed — the softmax keeps both probabilities strictly positive forever.

§9.5 Chapter 9 summary and self-check

► Book: §9.5, p. 213

Item	The point
Policy as a function	Optimality must be redefined via a scalar metric; policies are updated through θ , not entry by entry.
$\bar{v}_\pi$ and $\bar{r}_\pi$	Two metrics, equivalent in the discounted case via $\bar{r}_\pi = (1 - \gamma)\bar{v}_\pi$.
Theorem 9.1	$\nabla_\theta J = \mathbb{E}[\nabla_\theta \ln \pi(A S, \theta) q_\pi(S, A)]$. The single most important result of the chapter.
The logarithm	A device to turn a weighted sum into an expectation, so it can be sampled.
Softmax policies	Guarantee $\pi > 0$, making $\ln \pi$ valid and exploration automatic.
REINFORCE	$\theta \leftarrow \theta + \alpha \nabla_\theta \ln \pi(a_t s_t, \theta) q_t(s_t, a_t)$ with q_t a Monte Carlo return.

1. Why can optimality not be defined state-by-state once policies are parameterised?
2. Prove $\bar{r}_\pi = (1 - \gamma)\bar{v}_\pi$ in three lines.
3. Derive $\nabla_\theta \pi = \pi \nabla_\theta \ln \pi$ and explain why it is the pivotal step in Theorem 9.1.
4. Why must a softmax (or another strictly positive parameterisation) be used?
5. Explain how $\beta_t = q_t / \pi$ balances exploration and exploitation, and contrast this with ε -greedy.
6. Why is REINFORCE on-policy?
7. Redo the two-action worked example with $\alpha = 0.5$ and $q_t = -1$. What is the new $\pi(a_1 | s)$?

§10 Actor-Critic Methods

READING MAP — Chapter 10 of the textbook

Textbook: Chapter 10, pp. 215–236. Sections: §10.1 the simplest actor-critic (QAC) (p. 216), §10.2 advantage actor-critic (A2C) (p. 217; baseline invariance p. 217, algorithm p. 220), §10.3 off-policy actor-critic (p. 221; importance sampling p. 221, off-policy policy gradient theorem p. 224, algorithm p. 226), §10.4 deterministic actor-critic (p. 227; the DPG theorem p. 227, algorithm p. 234), §10.5 summary (p. 235), §10.6 Q&A (p. 236). Algorithms 10.1 (p. 216), 10.2 (p. 220), 10.3 (p. 226), 10.4 (p. 234). Theorems 10.1–10.4.

Videos: ► L10 P1 — The simplest actor-critic; ► L10 P2 — Advantage actor-critic; ► L10 P3 — Importance sampling & off-policy AC; ► L10 P4 — Deterministic actor-critic.

The synthesis. Chapters 1–8 were value-based, Chapter 9 was policy-based. Actor-critic is both: the *actor* updates the policy, the *critic* estimates the values the actor needs.

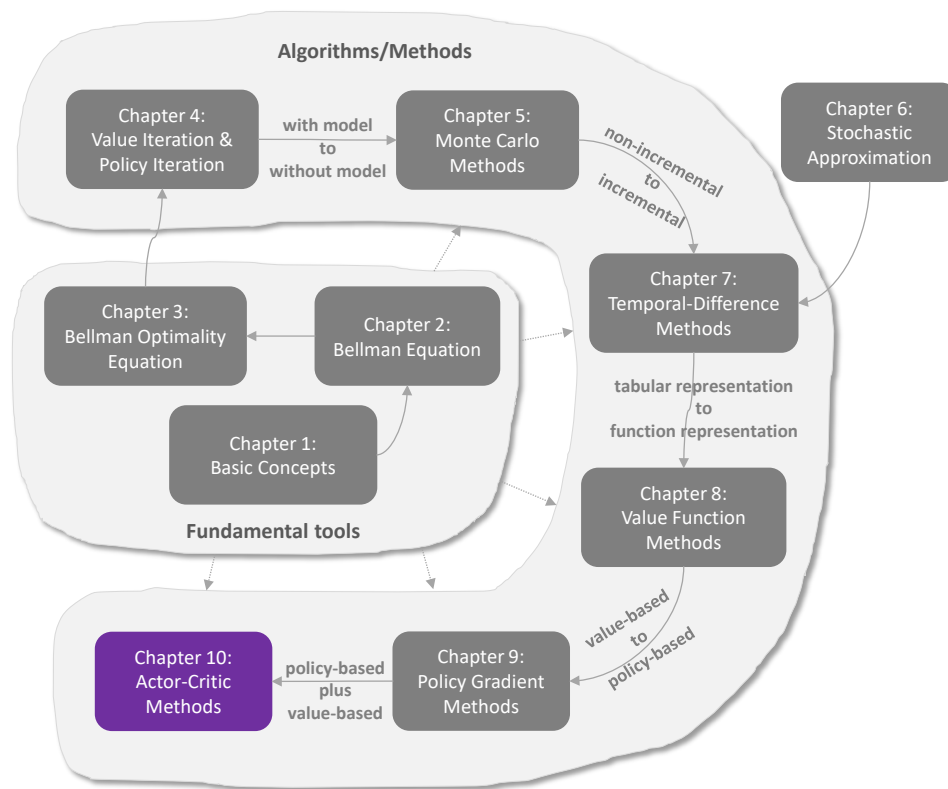


Figure 10.1. Where we are in this book.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§10.1 QAC: the simplest actor-critic

► Book: §10.1, pp. 216–217; equations (10.1)–(10.2); Algorithm 10.1, p. 216

Start from the policy gradient update [Book equations (10.1)–(10.2), p. 216]:

$$\theta_{t+1} = \theta_t + \alpha \mathbb{E}_{S \sim \eta, A \sim \pi} [\nabla_{\theta} \ln \pi(A | S, \theta_t) q_{\pi}(S, A)] \longrightarrow \theta_{t+1} = \theta_t + \alpha \nabla_{\theta} \ln \pi(a_t | s_t, \theta_t) q_t(s_t, a_t).$$

Intuition (The observation that generates the whole chapter). [Book §10.1, p. 216] This one update is *simultaneously* policy-based and value-based. It updates the policy parameter directly — policy-based. But it needs $q_t(s_t, a_t)$, an estimate of an action value, which some value-based method must supply. We know two ways to produce that estimate:

- **Monte Carlo** $\Rightarrow$ REINFORCE (Chapter 9);
- **Temporal difference** $\Rightarrow$ **actor-critic**.

That is the definition. Actor-critic methods *are* policy gradient methods in which the action values come from TD learning with function approximation.

ALGORITHM — Q actor-critic, QAC (Algorithm 10.1, p. 216)

Initialise: a policy $\pi(a | s, \theta_0)$; a value function $q(s, a, w_0)$; step sizes $\alpha_w, \alpha_{\theta} > 0$.
At each time step t of each episode:

- Generate $a_t \sim \pi(\cdot | s_t, \theta_t)$; observe r_{t+1}, s_{t+1} ; generate $a_{t+1} \sim \pi(\cdot | s_{t+1}, \theta_t)$.
- **Actor (policy update):** $\theta_{t+1} = \theta_t + \alpha_{\theta} \nabla_{\theta} \ln \pi(a_t | s_t, \theta_t) q(s_t, a_t, w_t)$
- **Critic (value update, Sarsa with function approximation):** $w_{t+1} = w_t + \alpha_w [r_{t+1} + \gamma q(s_{t+1}, a_{t+1}, w_t) - q(s_t, a_t, w_t)] \nabla_w q(s_t, a_t, w_t)$

Remark. The critic is exactly equation (8.35) — Sarsa with function approximation. The actor is exactly equation (9.32) — the policy gradient update. Nothing new has been invented; two existing pieces have been bolted together [Book §10.1, p. 216].

§10.2 Advantage actor-critic (A2C)

► Book: §10.2, pp. 217–221; equations (10.3)–(10.8); Algorithm 10.2, p. 220

§10.2.1 Baseline invariance

► Book: §10.2.1, pp. 217–220; equation (10.3)

Theorem 10.1 (Baseline invariance, equation (10.3), p. 217). *For any scalar function $b(S)$ of the state alone,*

$$\mathbb{E}_{S \sim \eta, A \sim \pi} [\nabla_{\theta} \ln \pi(A | S, \theta_t) q_{\pi}(S, A)] = \mathbb{E}_{S \sim \eta, A \sim \pi} [\nabla_{\theta} \ln \pi(A | S, \theta_t) (q_{\pi}(S, A) - b(S))].$$

Proof [Book §10.2.1, pp. 217–218]. It suffices that $\mathbb{E}[\nabla_{\theta} \ln \pi(A | S, \theta_t) b(S)] = 0$:

$$\begin{aligned} \mathbb{E}[\nabla_{\theta} \ln \pi b(S)] &= \sum_s \eta(s) \sum_a \pi(a | s, \theta_t) \nabla_{\theta} \ln \pi(a | s, \theta_t) b(s) \\ &= \sum_s \eta(s) b(s) \sum_a \nabla_{\theta} \pi(a | s, \theta_t) = \sum_s \eta(s) b(s) \underbrace{\nabla_{\theta} \sum_a \pi(a | s, \theta_t)}_{=1} = \sum_s \eta(s) b(s) \nabla_{\theta} 1 = 0. \end{aligned}$$

□

Intuition. The baseline is free because probabilities must sum to one, so the gradient of that sum is zero. *Anything that depends only on the state — not on the action — cannot change which action the gradient favours.* It shifts every action’s score by the same amount.

So why bother? Because the *mean* is invariant but the *variance* is not [Book §10.2.1, p. 218]. Let

$$X(S, A) \doteq \nabla_{\theta} \ln \pi(A | S, \theta_t) [q_{\pi}(S, A) - b(S)]. \quad (10.4)$$

The true gradient is $\mathbb{E}[X]$, estimated by a single draw x . If $\text{var}(X)$ is small, any draw is a good estimate; if it is large, a draw can be wildly off. REINFORCE and QAC use $b = 0$, which is an arbitrary and generally poor choice.

Theorem 10.2 (Optimal baseline, equation (10.5), p. 218; Box 10.1, p. 219).

$$b^*(s) = \frac{\mathbb{E}_{A \sim \pi} [\|\nabla_{\theta} \ln \pi(A | s, \theta_t)\|^2 q_{\pi}(s, A)]}{\mathbb{E}_{A \sim \pi} [\|\nabla_{\theta} \ln \pi(A | s, \theta_t)\|^2]}, \quad s \in \mathcal{S}.$$

Proof idea (Box 10.1, p. 219). Minimise $\text{tr}[\text{var}(X)] = \mathbb{E}[X^{\top} X] - \bar{x}^{\top} \bar{x}$. Since $\bar{x}$ is invariant, minimise $\mathbb{E}[\|\nabla_{\theta} \ln \pi\|^2 (q_{\pi}(S, A) - b(S))^2]$. Setting the derivative in $b(s)$ to zero gives a weighted-least-squares condition whose solution is b^* . □

Remark (The practical baseline). b^* is correct but too complicated to use. Drop the weight $\|\nabla_{\theta} \ln \pi\|^2$ and you get [Book p. 218]

$$b^{\dagger}(s) = \mathbb{E}_{A \sim \pi} [q_{\pi}(s, A)] = v_{\pi}(s).$$

The suboptimal-but-usable baseline is the state value. This is a genuinely pleasing result: the natural benchmark for “was this action good?” is “compared to the average action at this state”.

§10.2.2 The advantage function

► Book: §10.2.2, pp. 220–221; equations (10.7)–(10.8)

Definition (Advantage function).

$$\delta_{\pi}(S, A) \doteq q_{\pi}(S, A) - v_{\pi}(S).$$

Because $v_{\pi}(s) = \sum_a \pi(a | s) q_{\pi}(s, a)$ is the mean of the action values, $\delta_{\pi}(s, a) > 0$ means a is better than average at s [Book §10.2.2, p. 220].

The update becomes [Book equations (10.7)–(10.8), pp. 220]

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} \ln \pi(a_t | s_t, \theta_t) [q_t(s_t, a_t) - v_t(s_t)] = \theta_t + \alpha \nabla_{\theta} \ln \pi(a_t | s_t, \theta_t) \delta_t(s_t, a_t).$$

If q_t, v_t come from Monte Carlo, this is *REINFORCE with a baseline*; if from TD, it is **advantage actor-critic (A2C)**.

Intuition. The update now responds to the *relative* value of an action, not its absolute value. That is exactly right: choosing an action at a state is a comparison problem. If every action at a state has value +100, none of them is special, and a baseline-free update would nonetheless push all their probabilities up — wasted motion and needless variance.

§10.2.3 One network, not two

► Book: §10.2.2, p. 220

Estimating $\delta_t = q_t(s_t, a_t) - v_t(s_t)$ would need two networks. Instead approximate the advantage by the TD error:

$$q_t(s_t, a_t) - v_t(s_t) \approx r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t).$$

This is legitimate because

$$q_\pi(s_t, a_t) - v_\pi(s_t) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) - v_\pi(S_t) \mid S_t = s_t, A_t = a_t],$$

directly from the definition of q_π [Book p. 220]. **So the TD error is an unbiased estimate of the advantage** — and only v needs a network. Hence the alternative name *TD actor-critic*.

ALGORITHM — Advantage actor-critic, A2C / TD actor-critic (Algorithm 10.2, p. 220)

Initialise: a policy $\pi(a \mid s, \theta_0)$; a value function $v(s, w_0)$; $\alpha_w, \alpha_\theta > 0$.

At each time step t of each episode:

- Generate $a_t \sim \pi(\cdot \mid s_t, \theta_t)$, observe r_{t+1}, s_{t+1} .
- **Advantage (TD error):** $\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$
- **Actor:** $\theta_{t+1} = \theta_t + \alpha_\theta \delta_t \nabla_\theta \ln \pi(a_t \mid s_t, \theta_t)$
- **Critic:** $w_{t+1} = w_t + \alpha_w \delta_t \nabla_w v(s_t, w_t)$

Remark. Notice how symmetric the two updates are — *the same* δ_t drives both. And note again [Book p. 220] that $\pi(\theta_t)$ is stochastic, hence exploratory, so samples can be generated directly with no ε -greedy machinery. A well known asynchronous variant is A3C.

§10.3 Off-policy actor-critic

► Book: §10.3, pp. 221–227

Everything so far — REINFORCE, QAC, A2C — is **on-policy**, because the true gradient is an expectation with $A \sim \pi(\theta)$, so the samples must come from $\pi(\theta)$ [Book §10.3, p. 221]. What if we already hold data generated by some other policy?

§10.3.1 Importance sampling

► Book: §10.3.1, pp. 221–224; equations (10.9)–(10.10); Figure 10.2, p. 223

Definition (Importance sampling, equations (10.9)–(10.10), pp. 222). *To estimate $\mathbb{E}_{X \sim p_0}[X]$ using samples $\{x_i\}$ drawn from p_1 :*

$$\mathbb{E}_{X \sim p_0}[X] = \sum_x p_0(x)x = \sum_x p_1(x) \frac{p_0(x)}{p_1(x)} x = \mathbb{E}_{X \sim p_1} \left[\frac{p_0(X)}{p_1(X)} X \right] \approx \frac{1}{n} \sum_{i=1}^n \underbrace{\frac{p_0(x_i)}{p_1(x_i)}}_{\text{importance weight}} x_i.$$

Worked example: importance sampling with coins [Book §10.3.1, p. 223; Figure 10.2]

Let $X \in \{+1, -1\}$ with target distribution $p_0(+1) = p_0(-1) = 0.5$, so $\mathbb{E}_{p_0}[X] = 0$. Samples come instead from $p_1(+1) = 0.8$, $p_1(-1) = 0.2$, so $\mathbb{E}_{p_1}[X] = 0.8 - 0.2 = 0.6$.

Naive average. Averaging the raw samples converges to 0.6 — the wrong answer.

Importance-weighted average. The weights are $p_0(+1)/p_1(+1) = 0.5/0.8 = 0.625$ and $p_0(-1)/p_1(-1) = 0.5/0.2 = 2.5$. The weighted average converges to 0 — the right answer. Over-represented samples are shrunk, under-represented ones are magnified.

Common pitfall (Coverage is mandatory). [Book §10.3.1, p. 224] We need $p_1(x) \neq 0$ wherever $p_0(x) \neq 0$. If $p_1(+1) = 1$ and $p_1(-1) = 0$, every sample is $+1$ and the estimator returns

$$\frac{1}{n} \sum_i \frac{p_0(+1)}{p_1(+1)} \cdot 1 = \frac{0.5}{1} = 0.5 \neq 0$$

no matter how much data you collect. **You cannot reweight your way out of never having observed something.**

Remark (Why not just use the definition?). [Book §10.3.1, p. 222] If we know p_0 , why not compute $\sum_x p_0(x)x$ directly? Because that needs $p_0(x)$ at *every* x , which is infeasible when $\mathcal{X}$ is huge or p_0 is a neural network. Importance sampling needs p_0 only at the sampled points.

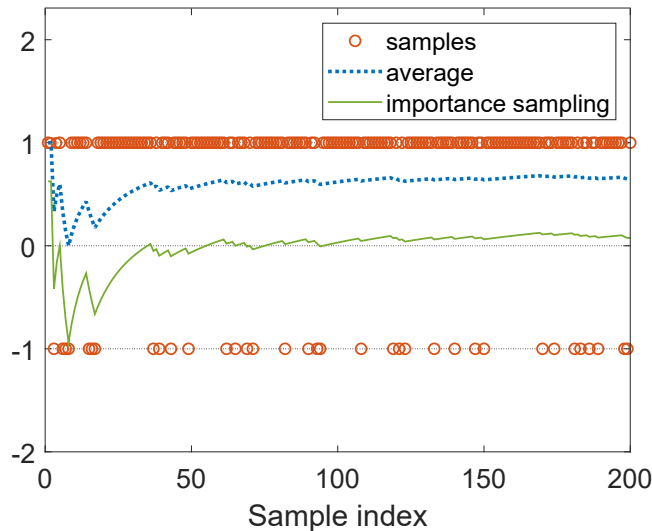


Figure 10.2. An example for demonstrating the importance sampling technique. Here, $X \in \{+1, -1\}$ and $p_0(X = +1) = p_0(X = -1) = 0.5$. The samples are generated according to p_1 where $p_1(X = +1) = 0.8$ and $p_1(X = -1) = 0.2$. The average of the samples converges to $\mathbb{E}_{X \sim p_1}[X] = 0.6$, but the weighted average calculated by the importance sampling technique in (10.10) converges to $\mathbb{E}_{X \sim p_0}[X] = 0$.

Reproduced from S. Zhao, *Mathematical Foundations of Reinforcement Learning* (Springer, 2025).

§10.3.2 The off-policy policy gradient theorem

► Book: §10.3.2, pp. 224–226; Theorem 10.1, p. 224; equation (10.11)

Let β be a behaviour policy and target $J(\theta) = \sum_s d_\beta(s) v_\pi(s) = \mathbb{E}_{S \sim d_\beta}[v_\pi(S)]$.

Theorem 10.3 (Off-policy policy gradient theorem, Theorem 10.1, p. 224). *For $\gamma \in (0, 1)$,*

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{S \sim \rho, A \sim \beta} \left[\underbrace{\frac{\pi(A | S, \theta)}{\beta(A | S)}}_{\text{importance weight}} \nabla_{\theta} \ln \pi(A | S, \theta) q_{\pi}(S, A) \right],$$

where $\rho(s) = \sum_{s'} d_{\beta}(s') \Pr_{\pi}(s | s')$ and $\Pr_{\pi}(s | s') = [(I - \gamma P_{\pi})^{-1}]_{s's}$.

Two differences from Theorem 9.1 [Book p. 224]: the importance weight, and $A \sim \beta$ instead of $A \sim \pi$. That second change is the point — we may now use actions generated by β .

§10.3.3 The algorithm

► Book: §10.3.3, pp. 226–227; Algorithm 10.3, p. 226

Baseline invariance survives the change (the same proof works), so subtract $v_{\pi}(S)$ and replace the advantage by the TD error:

$$\theta_{t+1} = \theta_t + \alpha_{\theta} \frac{\pi(a_t | s_t, \theta_t)}{\beta(a_t | s_t)} \nabla_{\theta} \ln \pi(a_t | s_t, \theta_t) \delta_t(s_t, a_t).$$

ALGORITHM — Off-policy actor-critic via importance sampling (Algorithm 10.3, p. 226)

Initialise: a behaviour policy $\beta(a | s)$; a target policy $\pi(a | s, \theta_0)$; a value function $v(s, w_0)$; $\alpha_w, \alpha_{\theta} > 0$. At each time step t of each episode:

- Generate $a_t \sim \beta(\cdot | s_t)$, observe r_{t+1}, s_{t+1} .
 - **Advantage (TD error):** $\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$
 - **Actor:** $\theta_{t+1} = \theta_t + \alpha_{\theta} \frac{\pi(a_t | s_t, \theta_t)}{\beta(a_t | s_t)} \delta_t \nabla_{\theta} \ln \pi(a_t | s_t, \theta_t)$
 - **Critic:** $w_{t+1} = w_t + \alpha_w \frac{\pi(a_t | s_t, \theta_t)}{\beta(a_t | s_t)} \delta_t \nabla_w v(s_t, w_t)$
-

Remark. Algorithm 10.3 is Algorithm 10.2 with one extra factor in *both* lines. Note that the critic needs converting too, not just the actor: it is also estimating an expectation under π from data drawn under β [Book §10.3.3, p. 226]. Importance sampling is a general device and applies equally to value-based methods [Book §10.6 Q&A, p. 236].

§10.4 Deterministic actor-critic (DPG)

► Book: §10.4, pp. 227–235; Theorem 10.2, p. 228; Algorithm 10.4, p. 234

Every policy so far has been stochastic, because $\ln \pi$ requires $\pi(a | s, \theta) > 0$. Write a deterministic policy as

$$a = \mu(s, \theta),$$

a *mapping* $\mathcal{S} \rightarrow \mathcal{A}$ rather than a distribution [Book §10.4, p. 227]. Two reasons to care: it is **naturally off-policy**, and it **handles continuous action spaces** — where $\max_a$ and $\sum_a$ are not available, so nothing from Chapters 7–8 applies.

Theorem 10.4 (Deterministic policy gradient theorem, Theorem 10.2, p. 228; equation (10.14)).

$$\nabla_{\theta} J(\theta) = \sum_{s \in \mathcal{S}} \eta(s) \nabla_{\theta} \mu(s) (\nabla_a q_{\mu}(s, a))|_{a=\mu(s)} = \mathbb{E}_{S \sim \eta} [\nabla_{\theta} \mu(S) (\nabla_a q_{\mu}(S, a))|_{a=\mu(S)}].$$

Intuition (Read the chain rule). $\nabla_a q_{\mu}$ says “how much better would the value be if I nudged the action?” and $\nabla_{\theta} \mu$ says “how do I nudge the action by moving θ ?”. Multiply them and you get how to move θ to raise the value. Nothing is averaged over actions — there is only one action per state.

Remark (Why it is off-policy, and a notation warning). [Book §10.4.1, p. 228] The gradient contains **no action random variable** A . So estimating it requires no action sampling, and any behaviour policy will do — the deterministic policy gradient method is off-policy by construction.

On notation: the book writes $(\nabla_a q_{\mu}(S, a))|_{a=\mu(S)}$ rather than $\nabla_a q_{\mu}(S, \mu(S))$ deliberately: the latter obscures which argument the derivative is taken with respect to. A clearer compromise is $\nabla_a q_{\mu}(S, a=\mu(S))$.

ALGORITHM — Deterministic actor-critic / DPG (Algorithm 10.4, p. 234)

Initialise: a behaviour policy $\beta(a | s)$; a deterministic target policy $\mu(s, \theta_0)$; a value function $q(s, a, w_0)$; $\alpha_w, \alpha_{\theta} > 0$.

At each time step t of each episode:

- Generate $a_t \sim \beta$, observe r_{t+1}, s_{t+1} .
- **TD error:** $\delta_t = r_{t+1} + \gamma q(s_{t+1}, \mu(s_{t+1}, \theta_t), w_t) - q(s_t, a_t, w_t)$
- **Actor:** $\theta_{t+1} = \theta_t + \alpha_{\theta} \nabla_{\theta} \mu(s_t, \theta_t) (\nabla_a q(s_t, a, w_t))|_{a=\mu(s_t)}$
- **Critic:** $w_{t+1} = w_t + \alpha_w \delta_t \nabla_w q(s_t, a_t, w_t)$

Common pitfall (Why no importance weight here?). [Book §10.4.2, p. 234] This is the subtlest point in the chapter. The critic is off-policy, yet Algorithm 10.4 carries no importance weight, unlike Algorithm 10.3. Why?

The critic’s sample is $(s_t, a_t, r_{t+1}, s_{t+1}, \tilde{a}_{t+1})$ with $\tilde{a}_{t+1} = \mu(s_{t+1})$. Two policies are involved: the one producing a_t (that is β , the behaviour policy, since a_t actually interacts with the environment) and the one producing $\tilde{a}_{t+1}$ — which is μ , because μ is the policy the critic is evaluating. **But $\tilde{a}_{t+1}$ is never executed**; it is only plugged into the TD target. So the target-policy action is computed, not sampled, and there is no distribution mismatch to correct. Compare Q-learning, off-policy for exactly the same structural reason: its $\max_a$ is computed, not sampled [Book §7.4.2, p. 142].

Remark (Practical choices, [Book §10.4.2, p. 234]). For $q(s, a, w)$, the original DPG paper used a linear function $q(s, a, w) = \phi^{\top}(s, a)w$; modern practice uses neural networks, as in **DDPG** (deep deterministic policy gradient). For β , any exploratory policy works; a common choice is μ plus noise — in which case μ is also the behaviour policy and the implementation is effectively on-policy.

§10.5 Chapter 10 summary

► Book: §10.5, pp. 235–236

Algorithm	What it adds
QAC (10.1)	Policy gradient with the q -estimate supplied by TD (Sarsa) instead of Monte Carlo.
A2C (10.2)	Subtracts the baseline $v_\pi(s)$, so the actor responds to the <i>advantage</i> ; the TD error estimates it with a single network.
Off-policy AC (10.3)	Importance weight π/β in both actor and critic, so data from another policy can be used.
DPG (10.4)	Deterministic policy $a = \mu(s, \theta)$; naturally off-policy; handles continuous action spaces. Basis of DDPG.

Remark (Where the subject goes from here). [Book §10.5, pp. 235–236] The book closes by pointing beyond itself: SAC, TRPO, PPO and TD3 are advanced policy-gradient/actor-critic algorithms; multi-agent RL extends the single-agent case; model-based RL fits a model from experience; distributional RL replaces expected values with whole return distributions; and there is a deep literature on the links between RL and control theory. Everything in these ten chapters is the foundation those build on.

§10.6 Chapter 10 self-check

1. In what precise sense is an actor-critic method both policy-based and value-based?
2. Prove baseline invariance in three lines. Which property of probability distributions does the proof rest on?
3. If the baseline does not change the gradient, why use one?
4. Show that the TD error is an unbiased estimate of the advantage, and explain why this saves a network.
5. Compute the importance weights in the coin example and verify that the weighted average converges to 0 rather than 0.6.
6. Give an example where importance sampling fails, and state the condition that rules it out.
7. Why does the deterministic policy gradient contain no action random variable, and why does that make it off-policy?
8. Algorithm 10.3's critic carries an importance weight and Algorithm 10.4's does not, though both are off-policy. Explain the difference.

Where everything fits

A final map of the ten chapters. Read the table downward as a sequence of assumptions being dropped.

Ch.	Contributes	Needs a model?	Representation
1	MDP: state, action, reward, return, policy	—	—
2	State value v_π ; the Bellman equation	yes (to solve it)	table
3	Optimal values v^* ; the BOE; contraction mapping	yes	table
4	Value iteration, policy iteration, truncated PI	yes	table
5	Monte Carlo: MC Basic, Exploring Starts, ε -greedy	no	table
6	Robbins–Monro, Dvoretzky, SGD	—	—
7	TD(0), Sarsa, n -step Sarsa, Q-learning	no	table
8	Function approximation; TD-Linear; DQN	no	function (values)
9	Policy gradient theorem; REINFORCE	no	function (policy)
10	QAC, A2C, off-policy AC, DPG	no	function (both)

The five ideas that recur everywhere

1. **Bootstrapping**, $G_t = R_{t+1} + \gamma G_{t+1}$ [Book §1.6, p. 10]. It generates the Bellman equation (§2.4), the TD target (§7.1), the backward pass in MC Exploring Starts (§5.3.3), and the advantage approximation in A2C (§10.2.2).
2. **Contraction**, $\|f(v_1) - f(v_2)\|_\infty \leq \gamma \|v_1 - v_2\|_\infty$ [Book Theorem 3.2, p. 44]. It gives existence, uniqueness and an algorithm all at once, and the γ reappears as the contraction constant in the convergence proofs of Chapter 7.
3. **Generalised policy iteration**: alternate a value update and a policy update [Book §4.4, p. 74]. Every algorithm in Chapters 4–10 has this shape; the actor and critic of Chapter 10 are its purest expression.
4. **Stochastic approximation**: replace an unknown expectation by a sample, with step sizes satisfying $\sum a_k = \infty$, $\sum a_k^2 < \infty$ [Book Theorem 6.1, p. 106]. This is how every model-free algorithm is built.
5. **Exploration**. It appears as a modelling choice (ε -greedy, §5.4), as a hypothesis of convergence theorems ($\sum_t \alpha_t(s) = \infty$ for every s , §7.1.3), as a condition for the stationary distribution to exist (§8.2.1), and as an automatic by-product of softmax policies (§9.3). It is not an implementation detail; it is a structural requirement.

A suggested study route

1. Watch the ► [30-minute overview](#) first, before reading anything.
2. Read Chapters 1–3 carefully; they are short and everything else assumes them. Do the worked examples with pencil and paper.
3. Chapter 4 is the pivot. If you understand that value iteration, policy iteration and truncated policy iteration are one algorithm with one dial, the rest of the book is a sequence of variations.

4. Chapters 5 and 7 can be read as a pair: MC and TD are the two ways to estimate a value from data, and n -step Sarsa interpolates between them.
5. Chapter 6 is machinery. Read §6.1 and Theorem 6.1 properly; skim §6.3 on first pass.
6. Chapters 8–10 are modern RL. Read §8.1 and §9.1 slowly — the conceptual shift from tables to functions is where most confusion arises.
7. Run the code. The repository accompanying the textbook contains grid-world implementations in both MATLAB and Python; reproducing Figures 4.4, 7.2 and 8.11 yourself is worth more than any amount of re-reading.